

**PEMBACAAN GERAK BIBIR PADA VIDEO SISTEM ISYARAT
BAHASA INDONESIA MENGGUNAKAN *OPTICAL FLOW*
*CONV-LSTM***

SKRIPSI



**NI MADE WIPRA RANUM RATNAYU
NIM: 2208561042**

**PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS UDAYANA**

**JIMBARAN
2026**

**PEMBACAAN GERAK BIBIR PADA VIDEO SISTEM ISYARAT
BAHASA INDONESIA MENGGUNAKAN *OPTICAL FLOW*
*CONV-LSTM***

SKRIPSI



**NI MADE WIPRA RANUM RATNAYU
NIM: 2208561042**

**PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS UDAYANA**

**JIMBARAN
2026**

LEMBAR PENGESAHAN TUGAS AKHIR

Judul : Pembacaan Gerak Bibir pada Sistem Isyarat Bahasa
Indonesia Menggunakan *Optical Flow Conv-LSTM*
Nama : Ni Made Wipra Ranum Ratnayu
NIM : 2208561042
Tanggal Seminar :

Disetujui oleh:

Pembimbing

Ketua Penguji 1

I Dewa Made Bayu Atmaja
Darmawan, S.Kom., M.Cs.
NIP. 198901272012121001

I Putu Gede Hendra Suputra,
S.Kom., M.Kom.
NIP. 198812282014041001

Sekretaris Penguji 2

Dr. Ir. I Ketut Gede Suhartana,
S.Kom., M.Kom., IPM., ASEAN.Eng.
NIP. 198901272012121001

Mengetahui,

Program Studi Informatika
FMIPA UNUD
KPS

Dr. I Made Widiartha, S.Si., M.Kom.
NIP. 198212202008011008

Judul : Pembacaan Gerak Bibir pada Sistem Isyarat Bahasa
Indonesia Menggunakan *Optical Flow Conv-LSTM*
Nama : Ni Made Wipra Ranum Ratnayu (NIM: 2208561042)
Pembimbing : I Dewa Made Bayu Atmaja Darmawan,S.Kom.,M.Cs.

ABSTRAK

Penyandang tunarungu mengandalkan gerak bibir sebagai salah satu media komunikasi, namun akurasi pembacaan gerak bibir secara manual sangat rendah. Penelitian ini mengembangkan sistem pembacaan gerak bibir pada video Sistem Isyarat Bahasa Indonesia (SIBI) menggunakan metode *Optical Flow Conv-LSTM* dan membandingkannya dengan model Conv-LSTM tanpa *optical flow* sebagai *baseline*. Dataset terdiri dari 3.600 video dari 20 subjek yang mencakup 18 label gabungan (alfabet, bilangan, dan kata dasar). *Preprocessing* dilakukan menggunakan *MediaPipe FaceLandmarker* untuk mengekstraksi area bibir sebagai sekuens 30 *frame* berukuran 96×96 piksel. *Optical flow* dihitung menggunakan metode Lucas-Kanade dengan konfigurasi parameter terbaik P17 yang dipilih melalui pengujian OFAT. Model dirancang dalam tiga varian yaitu *baseline* Conv-LSTM, *single-stream optical flow*, dan *dual-stream* dengan mekanisme AttentionFusion. Hasil evaluasi menunjukkan model *baseline* mencapai *test accuracy* 83.06%, model *single-stream* hanya 51.25%, sedangkan model *dual-stream* terbaik mencapai *test accuracy* 86.63% dengan *F1-score* 86.60%, melampaui *baseline* sebesar 3.57%. Penelitian ini membuktikan bahwa pendekatan *dual-stream* dengan AttentionFusion secara efektif meningkatkan akurasi pengenalan gerak bibir pada bahasa SIBI.

Kata Kunci: pembacaan gerak bibir, SIBI, *optical flow*, Lucas-Kanade, Conv-LSTM, *dual-stream*

Judul : Lip Reading in Indonesian Sign Language System Using Optical Flow Conv-LSTM
Nama : Ni Made Wipra Ranum Ratnayu (NIM: 2208561042)
Pembimbing : I Dewa Made Bayu Atmaja Darmawan,S.Kom.,M.Cs.

ABSTRACT

Deaf individuals rely on lip reading as one of their communication methods; however, manual lip reading accuracy is very low. This study develops a lip reading system for Indonesian Sign Language System (SIBI) videos using the Optical Flow Conv-LSTM method, compared against a standard Conv-LSTM model as the baseline. The dataset consists of 3,600 videos from 20 subjects covering 18 combined labels (alphabets, numbers, and basic words). Preprocessing was performed using MediaPipe FaceLandmarker to extract lip regions as sequences of 30 frames at 96×96 pixels. Optical flow was computed using the Lucas-Kanade method with the best P17 parameter configuration selected through OFAT testing. Three model variants were evaluated: baseline Conv-LSTM, single-stream optical flow, and dual-stream with AttentionFusion. Results show that the baseline achieved a test accuracy of 83.06%, the single-stream model only 51.25%, while the best dual-stream configuration achieved 86.63% test accuracy with an F1-score of 86.60%, surpassing the baseline by 3.57%. This study demonstrates that the dual-stream approach with AttentionFusion effectively improves lip reading recognition accuracy for the SIBI language system.

Keywords: lip reading, SIBI, optical flow, Lucas-Kanade, Conv-LSTM, dual-stream

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas berkat dan rahmatnya penelitian dengan judul Pembacaan Gerak Bibir pada Sistem Isyarat Bahasa Indonesia Menggunakan *Optical Flow Conv-LSTM* dapat diselesaikan dengan baik. Penelitian ini dilaksanakan pada periode 2025-2026 di Universitas Udayana.

Selama proses penyusunan penelitian ini, banyak bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu, diucapkan terima kasih dan penghargaan kepada:

1. Bapak Dr. I Dewa Made Bayu Atmaja Darmawan, S.Kom., M.Cs. sebagai Pembimbing dalam penyusunan Laporan Tugas Akhir ini, serta yang telah banyak mengkritisi, memeriksa, dan membantu menyempurnakan penelitian ini;
2. Bapak I Putu Gede Hendra Suputra, S.Kom., M.Kom. sebagai Pembimbing Akademik yang telah banyak memberi arahan dan masukkan agar penelitian ini bisa selesai dengan baik;
3. Bapak dan ibu dosen di Program Studi Informatika Fakultas MIPA Universitas Udayana, yang telah menjadi motivasi, penyemangat, serta memberikan saran dan masukan dalam penelitian ini;
4. Bapak dan Ibu pengurus Tata Usaha Program Studi Informatika yang telah membantu mengurus berkas yang diperlukan untuk ujian penelitian ini.
5. Orang tua dan teman-teman yang selalu memberikan dukungan, doa, dan motivasi sehingga dapat menyelesaikan penelitian ini dengan baik.

Disadari bahwa penelitian ini masih mengandung kelemahan dan kekurangan. Memperhatikan hal ini, masukan dan saran sangat diharapkan. Semoga penelitian ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan.

Bukit Jimbaran, 08 April 2026

Penyusun,

Ni Made Wipra Ranum Ratnayu

DAFTAR ISI

LEMBAR PENGESAHAN TUGAS AKHIR	ii
ABSTRAK	iii
ABSTRACT	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	4
1.5.1 Bagi Praktisi	4
1.5.2 Bagi Peneliti	4
1.5.3 Bagi Keilmuan	5
1.5.4 Bagi Penyandang Tunarungu	5
1.6 Sistematika Penulisan	5
BAB II TINJAUAN PUSTAKA	7
2.1 Tinjauan Teoritis	7
2.1.1 Tunarungu	7
2.1.2 Sistem Isyarat Bahasa Indonesia (SIBI)	7
2.1.3 Konsep Pembacaan Gerak Bibir (<i>Lip Reading</i>)	7

2.1.4	<i>Convolutional Neural Network (CNN)</i>	8
2.1.5	<i>Long Short-Term Memory (LSTM)</i>	9
2.1.6	<i>Optical Flow</i>	10
2.1.7	<i>Convolutional Long Short-Term Memory (Conv-LSTM)</i>	12
2.1.8	<i>Optical Flow Convolutional Long Short-Term Memory (Optical Flow Conv-LSTM)</i>	13
2.1.9	<i>Dual-Stream</i>	14
2.1.10	<i>Attention Function</i>	15
2.1.11	<i>Confusion Matrix</i>	16
2.2	Tinjauan Empiris.....	17
BAB III ANALISIS DAN PERANCANGAN SISTEM		26
3.1	Data dan Metode Pengumpulan Data.....	26
3.1.1	Pengumpulan Data	26
3.1.2	Spesifikasi Data.....	27
3.1.3	Pemilihan Label Berdasarkan Suku Kata.....	27
3.2	Preprosesing Data.....	28
3.3	Optical Flow Lucas-Kanade.....	30
3.4	Perancangan Arsitektur Model.....	30
3.4.1	<i>Baseline Model Conv-LSTM</i>	30
3.4.2	<i>Model Optical Flow Conv-LSTM</i>	31
3.5	Perancangan Pelatihan Model	34
3.6	Skenario Pengujian.....	34
3.6.1	Skenario 1: Perbandingan Metode Optical Flow	34
3.6.2	Skenario 2: OFAT Parameter Lucas-Kanade (Dataset Number).....	36
3.6.3	Skenario 3: Pelatihan Model pada Dataset Gabungan	36

3.6.4	Skenario 4: OFAT Arsitektur Model Conv-LSTM.....	37
3.6.5	Skenario 5: Penerapan Arsitektur Terbaik pada Dual-Stream	37
3.7	Perancangan Evaluasi Model	37
BAB IV HASIL DAN PEMBAHASAN		39
4.1	Hasil Preprocessing Data	39
4.2	Hasil Perhitungan <i>Optical Flow Lucas-Kanade</i> Terpilih	40
4.3	Hasil Skenario 1: Perbandingan Farneback & Lucas-Kanade	42
4.4	Hasil Skenario 2: OFAT Parameter Lucas-Kanade	43
4.5	Hasil Skenario 3: Pelatihan Model pada Dataset Gabungan.....	44
4.5.1	<i>Baseline</i> Conv-LSTM (RGB)	44
4.5.2	Optical Flow Conv-LSTM Single-Stream	48
4.5.3	Dual-Stream Conv-LSTM.....	51
4.6	Hasil Skenario 4: OFAT Arsitektur Model Conv-LSTM	55
4.7	Hasil Skenario 5: Arsitektur Pilihan Terbaik M02 pada LK P17 dan <i>Dual-Stream</i>	55
4.7.1	LK P17 <i>Single-Stream</i> dengan Arsitektur Model Conv-LSTM M02	55
4.7.2	<i>Dual-Stream</i> dengan Arsitektur M02.....	56
4.8	Perbandingan Model <i>Dual-Stream</i>	56
4.9	Rekapitulasi Seluruh Eksperimen	58
4.10	Analisis Berdasarkan Jumlah Suku Kata	58
4.11	Pembahasan.....	59
4.11.1	Analisis Performa Model <i>Baseline</i>	59
4.11.2	Analisis Performa Model dengan <i>Lucas-Kanade</i> LK P17 <i>Single-Stream</i>	59
4.11.3	Analisis Performa Model Dual-Stream.....	60

4.11.4	Kelas dengan Performa Rendah	60
4.11.5	Temuan Utama Penelitian	61
BAB V KESIMPULAN DAN SARAN.....		63
5. 1	Kesimpulan	63
5. 2	Saran.....	64
DAFTAR PUSTAKA		65
LAMPIRAN.....		70
Lampiran 1. Dataset		70
Lampiran 2. Tabel Perbandingan Parameter <i>Optical Flow</i> Lucas-Kanade		72
Lampiran 3. Kode Lengkap Preprosesing Data.....		74
Lampiran 4. Kode Lengkap Optical Flow Lucas-Kanade P17		76
Lampiran 5. Kode Lengkap Model Conv-LSTM Terbaik M02.....		80
Lampiran 6. Kode Lengkap Single-Stream Optical Flow Lucas-Kanade P17..		86
Lampiran 7. Kode Lengkap Dual-Stream Optical Flow Lucas-Kanade P17 & M02		93

DAFTAR TABEL

Tabel 2. 1 Confusion Matrix	16
Tabel 2. 2 Tinjauan Empiris.....	17
Tabel 3. 1 Klasifikasi Label Berdasarkan Jumlah Suku Kata.....	27
Tabel 3. 2 Potongan kode Farneback	34
Tabel 3. 3 Potongan kode implementasi Lucas-Kanade	35
Tabel 3. 4 Matriks Konfigurasi OFAT Parameter Lucas-Kanade	36
Tabel 3. 5 Variasi OFAT Arsitektur Model Conv-LSTM	37
Tabel 4. 1 Distribusi Data Hasil Preprocessing	39
Tabel 4. 2 Konfigurasi Parameter Lucas-Kanade	40
Tabel 4. 3 Perbandingan <i>Farneback</i> vs <i>Lucas-Kanade</i> pada Dataset Number	43
Tabel 4. 4 Hasil Evaluasi <i>Test Set Baseline</i> Conv-LSTM	46
Tabel 4. 5 Nilai Akurasi per Kelas <i>Test Set Baseline Conv-LSTM</i>	46
Tabel 4. 6 Hasil Evaluasi Test Set Lucas-Kanade Single-Stream	49
Tabel 4. 7 Nilai Akurasi per Kelas <i>Test Set Lucas-Kanade Single-Stream</i>	49
Tabel 4. 8 Hasil Evaluasi <i>Test Set Lucas-Kanade Dual-Stream</i>	53
Tabel 4. 9 Nilai Akurasi per Kelas <i>Test Set Lucas-Kanade Dual-Stream</i>	53
Tabel 4. 10 Hasil Lengkap OFAT Arsitektur Model Conv-LSTM	55
Tabel 4. 11 Perbandingan Variasi Arsitektur pada LK P17 Single-Stream.....	56
Tabel 4. 12 Hasil Evaluasi <i>Test Set Dual-Stream</i> RGB + LK P17 (M02).....	56
Tabel 4. 13 Perbandingan Hasil Evaluasi Model Dual-Stream	57
Tabel 4. 14 Rekapitulasi Seluruh Hasil Eksperimen.....	58
Tabel 4. 15 Rata-rata Recall Per Kelompok Suku Kata.....	59

DAFTAR GAMBAR

Gambar 2. 1 Contoh Gambar Pendeteksian Gerak Bibir (Murthy, 2020)	8
Gambar 2. 2 Contoh Gambar Lapisan CNN (Zahara <i>et al.</i> , 2020)	9
Gambar 2. 3 Arsitektur LSTM (Wang <i>et al.</i> , 2020).....	10
Gambar 2. 4 Contoh aplikasi Optical Flow (Nayar Shree, 2021)	11
Gambar 2. 5 Model Conv-LSTM (C. Shi <i>et al.</i> , 2022)	13
Gambar 3. 1 Alur Preprosesing Data	28
Gambar 3. 2 Potongan Video Menjadi Beberapa <i>Frame</i> (Jin Ting <i>et al.</i> , 2021)..	28
Gambar 3. 3 Pendeteksian Bibir Menggunakan Koordinat Landmark (Jin Ting <i>et al.</i> , 2021)	29
Gambar 3. 4 Alur Model Optical Flow Conv-LSTM	31
Gambar 3. 5 Alur Model Dual Stream Optical Flow Conv-LSTM	33
Gambar 4. 1 Contoh Hasil Crop Bibir Setiap Label	40
Gambar 4. 2 Visualisasi 3 Channel Optical Flow (Frame Asli, Channel u, Channel v, dan Magnitude)	41
Gambar 4. 3 Distribusi Raw Flow Sebelum Normalisasi dan Distribusi Setelah Normalisasi	42
Gambar 4. 4 Hasil OFAT Parameter Lucas-Kanade pada Dataset Number	43
Gambar 4. 5 Kurva Training dan Validation Loss/accuracy Model Baseline	45
Gambar 4. 6 Penurunan Learning Rate dan Rata-rata Waktu Training per Epoch Model Baseline.....	45
Gambar 4. 7 Confusion Matrix Baseline	47
Gambar 4. 8 Bar chart akurasi per kelas baseline	47
Gambar 4. 9 Kurva training dan validation loss/accuracy model Lucas-Kanade single-stream	48
Gambar 4. 10 Penurunan Learning Rate dan Rata-rata Waktu Training per Epoch Model Lucas-Kanade single-stream.....	49
Gambar 4. 11 Confusion matrix Lucas-Kanade single-stream	50
Gambar 4. 12 Bar chart akurasi per kelas LK P17 single-stream	51

Gambar 4. 13 Kurva training dan validation loss/accuracy model dual-stream ..	52
Gambar 4. 14 Penurunan Learning Rate dan Rata-rata Waktu Training per Epoch Model Lucas-Kanade dual-stream	52
Gambar 4. 15 Confusion matrix dual-stream.....	54
Gambar 4. 16 Bar chart akurasi per kelas dual-stream	54
Gambar 4. 17 Grafik Batang Perbandingan Model Dual-Stream	57

BAB I

PENDAHULUAN

1.1 Latar Belakang

Gerak bibir merupakan salah satu acuan seseorang untuk memahami maksud dari ucapan orang lain selain dengan suara (Mahboob *et al.*, 2021). Huang *et al.* (2021) menjelaskan pembacaan gerakan bibir merupakan teknologi untuk menganalisis perubahan bentuk bibir pembicara dan mengenali informasi gerakan bibir yang membantu orang dengan gangguan pendengaran memahami apa yang dikatakan orang lain. Membaca gerak bibir dapat diartikan sebagai suatu keterampilan seseorang untuk memahami kata, kalimat, ataupun ucapan orang lain hanya dengan melihat gerakan pada bibir tanpa harus mendengar suaranya (Murthy, 2020). Kadang kala, kondisi ribut dan bising dapat mengganggu pemahaman ucapan orang, maka dari itu, pengenalan gerak bibir mampu membantu meningkatkan pemahaman mengenai ucapan seseorang (Aripin & Setiawan, 2024). Bagi kebanyakan orang, gerak bibir dapat membantu memahami maksud kata atau kalimat yang diucapkan seseorang dalam kondisi yang ramai, bahkan tidak hanya bagi orang dengan pendengaran normal, tetapi juga bagi penyandang tunarungu, kemampuan memahami gerak bibir sangat krusial.

Penyandang tunarungu merupakan seseorang yang mempunyai masalah atau gangguan pada pendengaran, dimana diharapkan orang tuna rungu juga bisa berkomunikasi dengan efektif, baik itu sesama orang tuna rungu atau orang tuna rungu dengan masyarakat umum dengan menggunakan media berbicara, membaca gerak bibir, dan berisyarat secara terpadu (Samto *et al.*, 2020). Secara umum, komunikasi penyandang tunarungu dilakukan dengan menggunakan bahasa isyarat baik itu dari gerak tangan, tubuh ataupun ekspresi (Syulistyo *et al.*, 2020). Namun, komunikasi dengan bahasa isyarat juga melibatkan gerak bibir (*lip reading*). Gerak bibir berperan penting bagi penyandang tunarungu karena melalui gerak bibir seseorang dapat menangkap pesan dari ucapan orang lain (OZCAN & BASTURK, 2019). Namun, Easton dan Basala dalam (X. Chen *et al.*, 2020) menjelaskan bahkan

seseorang yang telah terlatih dalam membaca gerakan bibir bahkan hanya mampu mendapatkan akurasi $17 \pm 12\%$ untuk 30 kata tunggal dan $21 \pm 11\%$ untuk 30 kata majemuk.

Penelitian terkait pembacaan gerakan bibir telah dilakukan untuk beberapa bahasa lisan dan metode yang berbeda. Atila dan Sabaz (2022) membuat pembacaan gerak bibir untuk bahasa Turki dengan *CNN* dan *Bi-LSTM*. Penelitian ini menghasilkan akurasi 84,5% dan 88,55% dengan menggunakan model *ResNet-18* dan *Bi-LSTM*. Dweik *et al.* (2022) mengusulkan sistem komputasi pembaca gerakan bibir yang mampu mengenali sepuluh kata dalam bahasa Arab dengan menggunakan tiga metode *deep learning*, yaitu: *CNN*, *TD-CNN-LSTM*, dan *TD-CNN-BiLSTM* untuk melatih, memvalidasi, dan menguji sistem. Penelitian sebelumnya dari Chen *et al.* (2020) memperkenalkan pembacaan bibir dalam bahasa Mandarin dari kalimat yang ada pada berita, pidato, dan acara *talkshow*. Penelitian ini menggunakan *Hanyu Pinyin* (transkripsi fonemik bahasa Cina) sebagai label. Pendeteksian pembacaan gerakan bibir dilakukan dengan metode konvolusi 3D menggunakan *DenseNet* dan *resBi-LSTM* dalam *Hanyu Pinyin* mendapat hasil terburuk hanya 64,35%. Sedangkan dalam bahasa Indonesia sendiri terdapat penelitian dari Aripin dan Setiawan (2024) tentang lipreading menggunakan dataset bahasa Indonesia dengan metode *Long-term Recurrent Convolutional Network (LRCN)* dengan mempertimbangkan delapan jenis bentuk bibir yang berbeda dan menggunakan *MediaPipe Face Mesh*. Hasil eksperimen menunjukkan efektivitas pendekatan, dengan model *LRCN* dengan tiga lapisan konvolusional (*LRCN 3Conv*) mencapai akurasi 95,42% untuk data uji kata dan 95,63% untuk frasa, mengungguli metode *Convolutional Long Short-Term Memory (Conv-LSTM)*. Namun, *Convolutional Long Short-Term Memory (Conv-LSTM)* juga termasuk baik mendapatkan akurasi 90,42% untuk kata dan 90,62% untuk frasa.

Penelitian Kurniawan dan Kaswidjanti, (2024) melakukan pengenalan gerakan isyarat SIBI namun tidak menggunakan pembacaan Gerakan bibir sebagai

fiturnya. Pembacaan gerakan bibir seharusnya dapat lebih membantu tunarungu dalam berkomunikasi. Begitupun pendeteksian gerak bibir sudah banyak terdapat dengan berbagai bahasa lisan, namun saat ini belum ada pembacaan gerak bibir terkhusus untuk orang yang mengalami gangguan pendengaran seperti orang tunarungu. Oleh karena itu, usulan penelitian ini bertujuan untuk mengembangkan sistem pembacaan gerak bibir menggunakan metode *Optical Flow Conv-LSTM*, yang diharapkan dapat memberikan hasil yang lebih optimal dalam membantu penyandang tunarungu memahami informasi yang disampaikan dalam bahasa SIBI.

1.2 Rumusan Masalah

Berdasarkan latar belakang penelitian ini, adapun rumusan masalah yang diajukan sebagai acuan dalam penelitian ini yaitu: bagaimana perbandingan hasil akurasi yang didapat dengan menggunakan model *Conv-LSTM* dan *Optical Flow Conv-LSTM* untuk mendeteksi Gerakan bibir pada video SIBI?

1.3 Batasan Masalah

Adapun batasan yang ditetapkan dalam penelitian ini sebagai berikut:

- a. Penelitian ini hanya akan fokus pada pengenalan gerak bibir dalam bahasa SIBI (Sistem Isyarat Bahasa Indonesia) yang datanya diambil langsung dengan gerakan SIBI.
- b. Penelitian ini menggunakan metode *Con-LSTM* dengan menggunakan *Optical Flow* pada tahap *preprocessing* agar mendapat detail perubahan gerak lebih optimal.
- c. Metode pembanding utama adalah metode *Convolutional Long Short-Term Memory (Conv-LSTM)* tanpa tahap *preprocessing* menggunakan *Optical Flow*.
- d. Data diambil dari guru di SLB dan siswa penyandang tunarungu.
- e. Evaluasi performa akan fokus pada metrik akurasi, presisi, *recall*, dan *F1-score* untuk pengenalan gerak bibir.
- f. Dataset yang digunakan meliputi *alphabet* (a, b, c, d, e, dan f), bilangan (1, 2, 3, 8, 9, dan 10), dan 6 kata dasar dalam bahasa SIBI (Sistem Isyarat

Bahasa Indonesia) yaitu kata buku, dia, saya, keliling, kelompok, dan sekarang.

- g. Data terekam (tidak *real time*).
- h. Output program yang diharapkan berupa teks.

1.4 Tujuan Penelitian

Adapun tujuan dari penelitian ini yaitu sebagai berikut:

- a. Mengembangkan model komputasi pembacaan gerak bibir untuk bahasa SIBI (Sistem Isyarat Bahasa Indonesia) agar lebih efektif dan akurat.
- b. Membandingkan efektivitas dari hasil akurasi dari pendekatan model *Conv-LSTM* dengan model *Optical Flow Conv-LSTM* dalam konteks pembacaan gerak bibir untuk bahasa SIBI.
- c. Menghasilkan rekomendasi untuk pengembangan lebih lanjut dan implementasi teknologi pembacaan gerak bibir dalam aplikasi yang dapat digunakan oleh penyandang tunarungu di Indonesia.

1.5 Manfaat Penelitian

1.5.1 Bagi Praktisi

- 1. Membantu mengembangkan alat bantu yang efektif bagi orang tunarungu untuk memahami percakapan melalui teknologi pengenalan gerak bibir, sehingga mereka dapat berkomunikasi lebih mudah dan efektif dalam berbagai situasi.
- 2. Hasil penelitian dapat digunakan untuk mengembangkan aplikasi mobile atau perangkat lunak lainnya yang dapat diimplementasikan dalam kehidupan sehari-hari orang tunarungu, meningkatkan kualitas hidup mereka.

1.5.2 Bagi Peneliti

- 1. Penelitian ini menambah pengetahuan di bidang pengenalan gerak bibir dan aplikasi *deep learning* dalam konteks bahasa Isyarat Indonesia, serta memberikan wawasan baru dan dasar untuk penelitian lanjutan.

2. Pengembangan dataset yang beragam dan representatif untuk bahasa SIBI dapat digunakan oleh peneliti lain untuk berbagai keperluan penelitian lebih lanjut.

1.5.3 Bagi Keilmuan

Diharapkan penelitian ini nantinya dapat menjadi referensi pada penelitian lain yang akan datang tentang pengenalan gerak bibir untuk orang tunarungu.

1.5.4 Bagi Penyandang Tunarungu

Diharapkan dapat membantu berkomunikasi dengan baik bagi penyandang tunarungu saat dibuatkan sebuah aplikasi khusus dari penelitian ini.

1.6 Sistematika Penulisan

Penulisan tugas akhir ini menggunakan sistematika yang telah ditentukan guna memastikan kejelasannya. Sistematika ini membagi laporan menjadi lima bab sebagai berikut.

1. BAB I PENDAHULUAN

Bab ini menguraikan latar belakang permasalahan yang akan diteliti, merumuskan inti permasalahan yang ingin dipecahkan, menjabarkan batasan dari masalah yang akan diselesaikan, menentukan tujuan dan manfaat dari penelitian, serta memaparkan sistematika penulisan laporan penelitian.

2. BAB II TINJAUAN PUSTAKA

Bab ini membahas mengenai beberapa hasil penelitian lain yang relevan digunakan dalam penelitian ini dan teori-teori yang menjadi landasan dalam melakukan penelitian diantaranya teori mengenai tuna rungu, Sistem Isyarat Bahasa Indonesia (SIBI), konsep pembacaan Gerak bibir (*lip reading*), *Convolutional Neural Network (CNN)*, *Long Short-Term Memory (LSTM)*, *Optical Flow*, *Convolutional Long Short-Term Memory (Conv-LSTM)*, *Optical Flow Convolutional Long Short-Term Memory (Optical Flow Conv-LSTM)*, *K-Fold Cross-Validation*, dan *Confusion Matrix*.

3. BAB III ANALISIS DAN PERANCANGAN SISTEM

Bab ini membahas mengenai metode pengumpulan data, menguraikan spesifikasi dan analisis serta metode persiapan data yang digunakan. Selain itu, dalam bab ini juga dibahas mengenai desain metode komputasi yang akan dikembangkan dan metode evaluasi yang digunakan untuk mengembangkan rancangan tersebut.

4. BAB IV HASIL DAN PEMBAHASAN

Bab ini menguraikan secara detail mengenai hasil implementasi dari rancangan sistem yang telah dibuat sebelumnya pada bagian analisis dan perancangan sistem serta pembahasan atau analisis yang dijabarkan untuk menjawab rumusan masalah yang telah ditentukan pada bagian pendahuluan.

5. BAB V KESIMPULAN DAN SARAN

Bab ini memaparkan kesimpulan singkat dari hasil pembahasan mengenai solusi permasalahan yang telah dirumuskan dan saran mengenai pengembangan dan pemanfaatan penelitian lebih lanjut.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Teoritis

2.1.1 Tunarungu

Penyandang tunarungu adalah individu yang memiliki gangguan pada pendengaran yang dapat berupa kehilangan pendengaran sebagian atau total. Menurut (Samto *et al.*, 2020), tunarungu merujuk pada orang-orang yang mengalami kesulitan dalam mendengar, yang mengakibatkan keterbatasan dalam menerima informasi secara auditori. Menurut (Caraka *et al.*, 2024), ketulian diakibatkan oleh 2 faktor yaitu faktor bawaan, seperti kecenderungan genetik dan infeksi selama kehamilan seperti *rubella* dan faktor yang didapat termasuk infeksi seperti meningitis, infeksi telinga kronis, dan paparan suara keras, yang semuanya dapat merusak struktur telinga dan mengakibatkan gangguan pendengaran.

2.1.2 Sistem Isyarat Bahasa Indonesia (SIBI)

Bahasa isyarat adalah sistem komunikasi visual yang menggunakan gerakan tangan dan ekspresi wajah. Setiap negara memiliki versi bahasa isyaratnya sendiri, seperti *ASL (American Sign Language)* di Amerika Serikat dan SIBI (Sistem Isyarat Bahasa Indonesia) di Indonesia. Berdasarkan Rakun *et al.* (2013) pada tahun 1994, Departemen Kebudayaan dan Pendidikan Indonesia merilis kamus SIBI (Sistem Isyarat Bahasa Indonesia). SIBI menjadi sistem bahasa isyarat resmi untuk bahasa Indonesia. Bahasa isyarat memungkinkan penyandang tunarungu berkomunikasi secara efektif dengan sesama tunarungu maupun dengan masyarakat umum yang mengerti isyarat tersebut (Samto *et al.*, 2020).

2.1.3 Konsep Pembacaan Gerak Bibir (*Lip Reading*)

Lip reading adalah metode komunikasi dengan membaca gerakan bibir seorang pembicara (Jin Ting *et al.*, 2021). Pembacaan gerak bibir (*lip reading*) adalah keterampilan untuk memahami kata-kata, kalimat, atau ucapan seseorang melalui pengamatan pergerakan bibir dan wajah mereka tanpa mendengar suara yang dihasilkan. Menurut Murthy (2020), pembacaan gerak bibir adalah proses

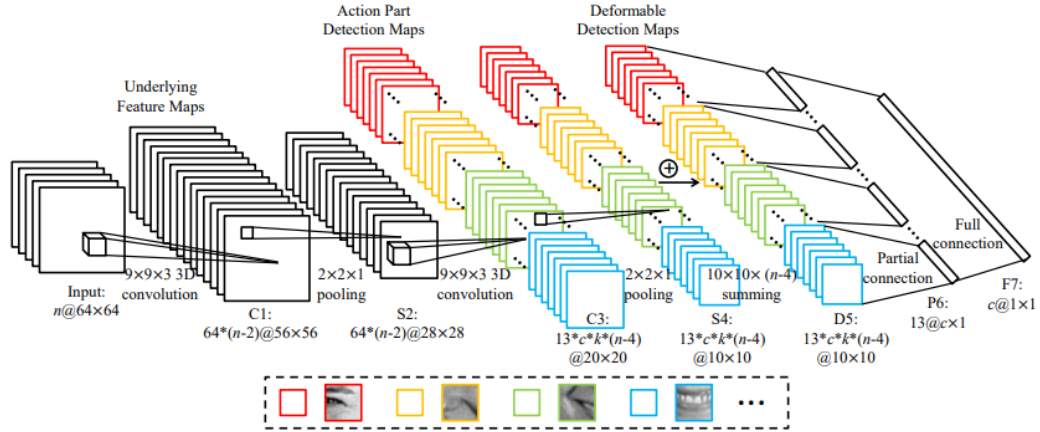
visual yang memungkinkan seseorang untuk menginterpretasikan informasi yang disampaikan secara verbal melalui ekspresi dan pergerakan bibir. Konsep pembacaan gerak bibir dari video tanpa suara dipaparkan dalam penelitian Prajwal *et al.*, 2021 yang berfokus pada visual.



Gambar 2. 1 Contoh Gambar Pendeteksian Gerak Bibir (Murthy, 2020)ibir
(Murthy, 2020)

2.1.4 *Convolutional Neural Network (CNN)*

Convolutional Neural Network (CNN) adalah jenis jaringan saraf tiruan yang dirancang khusus untuk memproses data dalam bentuk *grid*, seperti gambar. *CNN* mengotomatiskan dan meningkatkan proses ekstraksi fitur dari data masukan dengan menggunakan lapisan konvolusi. Arsitektur *CNN* umumnya terdiri dari beberapa lapisan, yaitu Lapisan Konvolusi (*Convolutional Layer*), Lapisan Aktivasi (*Activation Layer*), Lapisan Pooling (*Pooling Layer*), dan Lapisan Sepenuhnya Terhubung (*Fully Connected Layer*). Lapisan *CNN* memiliki susunan neuron 3 dimensi lebar (Zahara *et al.*, 2020).



Gambar 2. 2 Contoh Gambar Lapisan CNN (Zahara *et al.*, 2020)

Pada Gambar 2. 2 menampilkan *CNN* yang memiliki puluhan hingga ratusan lapisan yang masing-masing belajar mendeteksi gambar. Pemrosesan gambar berlaku untuk setiap gambar pelatihan pada resolusi yang berbeda, dan output dari setiap gambar diproses dan digunakan sebagai input ke lapisan berikutnya.

2.1.5 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) adalah jenis arsitektur jaringan saraf tiruan yang termasuk dalam kategori *Recurrent Neural Network (RNN)*. *LSTM* dikembangkan oleh Hochreiter dan Schmidhuber pada tahun 1997 untuk mengatasi masalah *vanishing gradient* yang sering terjadi pada *RNN* konvensional ketika mempelajari dependensi jangka panjang dalam data sekuensial (Hochreiter & Schmidhuber, 1997). *LSTM* dirancang untuk mengingat informasi selama periode waktu yang panjang dan mengabaikan informasi yang tidak relevan. *LSTM* terdiri dari satuan atau sel *LSTM*, yang memiliki tiga gerbang utama dua status: gerbang masukan (*input gate*), gerbang keluaran (*output gate*), gerbang lupa (*forget gate*), status sel (*cell state*), dan status tersembunyi (*hidden state*).

Komponen utama *LSTM*:

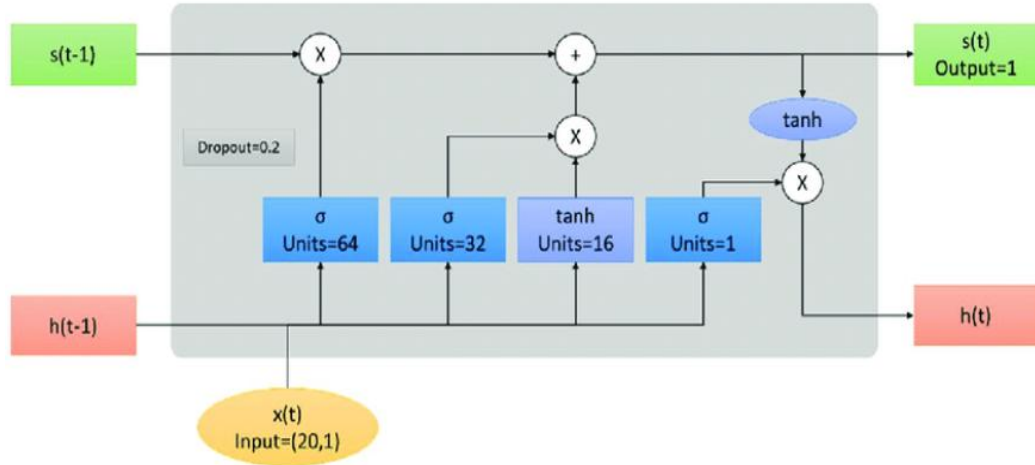
$$\text{Input Gate: } i_t = \sigma(W_i * [H_{t-1}, x_t] + b_i) \quad (6.1)$$

$$\text{Forget Gate: } f_t = \sigma(W_f * [H_{t-1}, x_t] + b_f) \quad (6.2)$$

$$\text{Output Gate: } o_t = \sigma(W_o * [H_{t-1}, x_t] + b_o) \quad (6.3)$$

$$\text{Cell State: } C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (6.4)$$

$$\text{Hidden State: } h_t = o_t * \tanh(C_t) \quad (6.5)$$



Gambar 2. 3 Arsitektur LSTM (Wang *et al.*, 2020)

2.1.6 Optical Flow

Optical flow adalah pola pergerakan yang tampak dari objek, permukaan, dan tepi dalam suatu pemandangan visual yang disebabkan oleh gerakan relatif antara pengamat dan pemandangan itu sendiri, maupun gerakan objek itu sendiri (Horn & Schunck, 1981). Secara teknis, optical flow merepresentasikan estimasi pergerakan piksel antara dua frame berurutan dalam sebuah gambar atau video sebagai medan vektor dua dimensi, di mana setiap vektor menggambarkan perpindahan piksel pada sumbu horizontal (u) dan vertikal (v) (Fleet & Weiss, 2006). Optical flow memberikan informasi tentang kecepatan dan arah gerakan setiap titik dalam gambar, yang dapat dimanfaatkan untuk berbagai aplikasi seperti pengenalan gerakan, pelacakan objek, segmentasi gerak, dan pemahaman adegan visual (Alfarano *et al.*, 2024).

Secara umum, metode komputasi optical flow dibagi menjadi dua kategori utama, yaitu dense optical flow dan sparse optical flow. Dense optical flow menghitung vektor pergerakan untuk setiap piksel dalam frame, seperti pada

metode Farnebäck yang menggunakan pendekatan aproksimasi polinomial (Farnebäck, 2003). Sebaliknya, sparse optical flow hanya menghitung pergerakan pada titik-titik fitur tertentu yang telah dipilih terlebih dahulu, sehingga lebih efisien secara komputasi. Salah satu metode sparse optical flow yang paling banyak digunakan adalah metode Lucas-Kanade, yang mengasumsikan bahwa pergerakan piksel dalam suatu lingkungan lokal bersifat konstan dan menyelesaikan persamaan optical flow menggunakan pendekatan least squares (Lucas & Kanade, 1981). Dalam konteks pembacaan gerak bibir (lip reading), optical flow terbukti mampu menangkap pola pergerakan halus pada region bibir antar frame yang sulit dideteksi hanya dari informasi penampilan statis, sehingga meningkatkan kemampuan model dalam membedakan kelas gerak bibir yang secara visual serupa [6].



Gambar 2. 4 Contoh aplikasi *Optical Flow* (Nayar Shree, 2021)

Salah satu metode umum yang sering digunakan untuk menghitung *Optical Flow* yaitu metode Lucas-Kanade (Al-Qudah & Yang, 2023). Metode ini mengasumsikan bahwa pergerakan objek di sekitar piksel yang diamati adalah kecil dan hampir seragam. Intensitas sebuah piksel di lokasi tertentu dalam suatu *frame* tetap konstan pada *frame* berikutnya. Persamaan dasar untuk *Optical Flow* adalah:

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t) \quad (6.6)$$

Dengan menggunakan deret *Taylor* untuk mendekati intensitas di *frame* berikutnya, didapatkan:

$$I(x + \Delta x, y + \Delta y, t + \Delta t) \approx I(x, y, t) + \frac{\partial I}{\partial x} \Delta x + \frac{\partial I}{\partial y} \Delta y + \frac{\partial I}{\partial t} \Delta t \quad (6.7)$$

Dengan Intensitas tetap:

$$\frac{\partial I}{\partial x} u + \frac{\partial I}{\partial y} v + \frac{\partial I}{\partial t} = 0 \quad (6.8)$$

Dimana u dan v adalah komponen horizontal dan vertikal dari *Optical Flow*.

2.1.7 Convolutional Long Short-Term Memory (Conv-LSTM)

Convolutional Long Short-Term Memory (Conv-LSTM) adalah variasi dari *Long Short-Term Memory (LSTM)* yang menggabungkan operasi konvolusional ke dalam struktur *LSTM*. *Conv-LSTM* diperkenalkan oleh (X. Shi *et al.*, n.d.) sebagai metode untuk menangani data sekuensial yang memiliki dimensi spasial, seperti video dan citra satelit. Dengan menggunakan lapisan konvolusional, *Conv-LSTM* dapat secara efektif menangkap pola spasial dan temporal dalam data. *Conv-LSTM* memodifikasi unit *LSTM* standar dengan menggantikan operasi matriks dalam *LSTM* dengan operasi konvolusi. Berikut adalah elemen kunci dari *Conv-LSTM*:

1. Sel *LSTM* dengan Konvolusi: Alih-alih menggunakan operasi matriks untuk memproses *input* dan menghasilkan *output*, *Conv-LSTM* menggunakan operasi konvolusi untuk menangkap informasi spasial dalam data.
2. Gerbang Konvolusional: Sama seperti *LSTM* standar, *Conv-LSTM* memiliki gerbang masukan (*input gate*), gerbang lupa (*forget gate*), dan gerbang keluaran (*output gate*). Namun, dalam *Conv-LSTM*, gerbang-gerbang ini menggunakan operasi konvolusi untuk memproses data.

Struktur *Conv-LSTM* diilustrasikan pada Gambar 2.5:

Input Gate (Gerbang Masukan):

$$i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + W_{ci} * C_{t-1} + b_i) \quad (6.9)$$

Forget Gate (Gerbang Lupa):

$$f_t = \sigma(W_{xf} * X_t + W_{hf} * H_{t-1} + W_{cf} \circ C_{t-1} + b_f) \quad (6.10)$$

Cell State (Keadaan Sel):

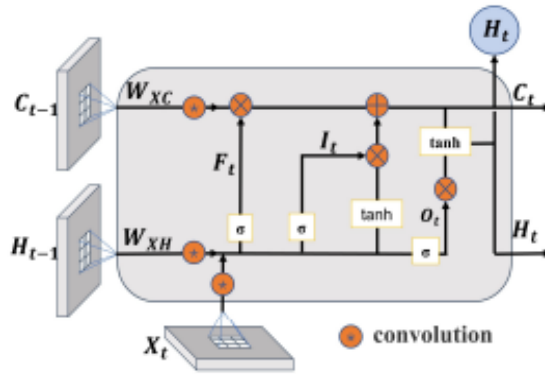
$$C_t = f_t \circ C_{t-1} + i_t \circ \tanh(W_{xc} * X_t + W_{hc} * H_{t-1} + b_c) \quad (6.11)$$

Output Gate (Gerbang Keluaran):

$$o_t = \sigma(W_{xo} * X_t + W_{ho} * H_{t-1} + W_{co} \circ C_{t-1} + b_o) \quad (6.12)$$

Hidden State (Keadaan Tersembunyi):

$$H_t = o_t \circ \tanh(C_t) \quad (6.13)$$



Gambar 2. 5 Model *Conv-LSTM* (C. Shi *et al.*, 2022)

2.1.8 *Optical Flow Convolutional Long Short-Term Memory (Optical Flow Conv-LSTM)*

Optical Flow Convolutional Long Short-Term Memory (Optical Flow Conv-LSTM) adalah varian dari *Convolutional Long Short-Term Memory (Conv-LSTM)* yang menggabungkan konsep *Optical Flow* untuk menangani data sekuensial dengan dimensi spasial dan temporal. *Optical Flow* memberikan informasi tentang pergerakan objek dalam suatu adegan, sementara *Conv-LSTM* memodelkan hubungan temporal dalam data sekuensial. Kombinasi ini memungkinkan *Optical Flow Conv-LSTM* untuk secara efektif menangkap pola gerakan dan perubahan temporal dalam data visual, seperti video. Konsep dasar *Optical Flow Conv-LSTM* yaitu menggabungkan kekuatan dari tiga komponen:

- a. *Optical Flow*: Memperkirakan pergerakan piksel dalam data video, menangkap informasi tentang pergerakan objek atau kamera.
- b. *Convolutional Neural Network (CNN)*: Mengelola informasi spasial, seperti pola dalam gerakan yang diekstrak dari *Optical Flow*.
- c. *Long Short-Term Memory (LSTM)*: Menangkap ketergantungan temporal dalam data, yaitu bagaimana pola gerakan tersebut berubah dan berinteraksi sepanjang waktu.

2.1.9 *Dual-Stream*

Arsitektur *dual-stream* adalah pendekatan arsitektur jaringan saraf yang memproses dua jalur input secara paralel dan independen, kemudian menggabungkan representasi fitur dari kedua jalur tersebut untuk menghasilkan keputusan akhir. Salah satu penelitian dari S. Chen *et al.*, 2021 melakukan pendekatan dengan metode *two-stream* yang memberikan hasil unggul untuk pengenalan aksi.

Keunggulan utama pendekatan dual-stream dibandingkan pendekatan *single-stream* adalah kemampuannya untuk memanfaatkan modalitas informasi yang berbeda secara bersamaan tanpa saling menghilangkan karakteristik masing-masing. Penelitian Zhao *et al.*, 2020 melakukan percobaan *two-stream* menggunakan LSTM untuk mengekstraksi fitur spasial dan temporal dalam *frame* RGB. Dalam konteks pembacaan gerak bibir terdapat juga penelitian dari Weng & Kitani, 2019 yang melakukan pengenalan gerakan bibir tingkat kata menggunakan konsep *two-stream*. Selain itu, informasi visual RGB memberikan konteks bentuk dan tekstur bibir, sementara *optical flow* memberikan informasi dinamika pergerakan antar *frame*. Kombinasi kedua modalitas ini memungkinkan model untuk membuat keputusan klasifikasi yang lebih akurat dan robust. Setiap *stream* pada arsitektur ini umumnya terdiri dari *encoder* yang berdiri sendiri, dan representasi fitur dari masing-masing *encoder* tersebut kemudian digabungkan melalui mekanisme *fusion* sebelum diteruskan ke lapisan klasifikasi.

2.1.10 *Attention Function*

Mekanisme atensi (*attention mechanism*) merupakan teknik penting dalam *deep learning* yang dapat diterapkan pada berbagai model dan domain tugas secara adaptif (Brauwers & Frasincar, 2022). Brauwers & Frasincar, 2022 mendefinisikan atensi sebagai mekanisme yang memungkinkan model untuk secara dinamis menentukan bobot relevansi terhadap bagian-bagian tertentu dari *input*, di mana berbagai jenis mekanisme atensi dapat diklasifikasikan menggunakan taksonomi yang beragam berdasarkan cara mereka menghitung dan mengalokasikan bobot perhatian tersebut. Dalam ranah visi komputer, Hassanin *et al.*, 2024 menjelaskan bahwa atensi bekerja dengan cara mengamplifikasi detail-detail krusial sehingga model lebih terfokus pada aspek-aspek esensial dari data, dan desain atensi yang sama dapat dengan mudah diintegrasikan ke dalam jaringan besar maupun digunakan untuk berbagai modalitas data. Sejalan dengan hal ini, Hafiz *et al.*, 2021 menegaskan bahwa penggabungan mekanisme atensi ke dalam arsitektur CNN telah menjadi pendekatan yang dominan dalam *machine vision modern*, di mana modul atensi dapat disisipkan sebagai komponen tambahan pada arsitektur *deep learning* klasik untuk meningkatkan kemampuan seleksi fitur secara lokal maupun global.

Dalam konteks model *dual-stream* pada penelitian ini, mekanisme atensi diimplementasikan melalui modul *AttentionFusion* yang berfungsi menggabungkan vektor fitur dari *stream* RGB dan *stream optical flow* secara adaptif. Berbeda dengan metode *fusion* sederhana seperti penjumlahan atau *concatenation* yang memberikan bobot setara pada semua fitur, *AttentionFusion* menghitung dua buah *gate* secara terpisah menggunakan lapisan *linear* dan fungsi *sigmoid* untuk menentukan kontribusi optimal dari setiap *stream*. Dengan mekanisme ini, model dapat secara otomatis memprioritaskan informasi RGB ketika kualitas *optical flow* kurang memadai, atau sebaliknya, sehingga menghasilkan representasi gabungan yang lebih informatif dan *robust* untuk klasifikasi gerak bibir.

2.1.11 Confusion Matrix

Confusion Matrix memberikan gambaran tentang jumlah prediksi yang dilakukan oleh model klasifikasi dengan jumlah data terprediksi benar dan salah. Dalam kasus klasifikasi biner, *Confusion Matrix* memiliki beberapa komponen, yaitu:

- True Positive (TP)*: Jumlah data *positif* yang diprediksi *positif* oleh model.
- True Negative (TN)*: Jumlah data *negatif* yang diprediksi *negatif* oleh model.
- False Positive (FP)*: Jumlah data *negatif* yang diprediksi *positif* oleh model. Ini juga dikenal sebagai "*Type I Error*".
- False Negative (FN)*: Jumlah data *positif* yang diprediksi *negatif* oleh model. Ini juga dikenal sebagai "*Type II Error*".

Tabel 2. 1 *Confusion Matrix*

	Prediksi <i>Positif</i>	Prediksi <i>Negatif</i>
Aktual <i>Positif</i>	TP	FN
Aktual <i>Negatif</i>	FP	TN

Dari confusion matrix tersebut, dapat diperoleh kinerja algoritma dengan menghitung nilai *accuracy*, *precision*, *recall*, dan *F1-score* dengan rumus berikut:

$$accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (6.14)$$

$$precision = \frac{TP}{TP+FP} \quad (6.15)$$

$$recall = \frac{TP}{TP+FN} \quad (6.16)$$

$$F1 \text{ score} = 2 * \frac{precision * recall}{precision+recall} \quad (6.17)$$

2.2 Tinjauan Empiris

Adapun penelitian terdahulu terkait *Lip Reading*, Bahasa Isyarat SIBI, dan juga metode lain. Berikut penjabarannya diuraikan pada Tabel 2.2:

Tabel 2. 2 Tinjauan Empiris

No	Keterangan	Penjelasan
1.	Judul, Penulis, Tahun	<i>A Novel Machine Lip Reading Model</i> (Huang <i>et al.</i> , 2021)
	Permasalahan atau Tujuan	Permasalahan yang dibahas dalam penelitian ini yaitu kesulitan membaca gerak bibir bagi manusia, khususnya bagi penyandang tunarungu dan tantangan untuk mengembangkan model pembacaan gerak bibir dengan akurasi yang tinggi. Adapun tujuan dari jurnal ini yaitu untuk membantu orang-orang yang mengalami gangguan dalam Indera pendengaran agar bisa memahami perkataan orang lain.
	Metode	Metode yang digunakan adalah <i>Transformer Network</i> untuk mendapatkan <i>lip reading</i> dengan akurasi tinggi.
	Hasil	Mode ini mendapatkan akurasi sebesar 45,81% pada <i>lip reading level</i> kata pada <i>GRID corpus</i> .
2.	Judul, Penulis, Tahun	<i>Lip reading with Hahn Convolutional Neural Networks</i> (Mesbah <i>et al.</i> , 2019)
	Permasalahan atau Tujuan	Jurnal ini membahas mengenai tantangan dalam membaca bibir, terkhusus untuk meningkatkan akurasi klasifikasi dan mengurangi kerumitan dalam mengenali gerakan bibir dari gambar video, tujuan penelitian ini untuk mengembangkan sistem pengenalan suara dengan hanya memanfaatkan video

		saja. Target yang dituju untuk penerapan sistem ini itu dunia medis.
	Metode	Metode yang digunakan dalam pembangunan sistem dalam jurnal ini yaitu <i>Hans Convolutional Neural Network (HCNN)</i> yang dimana merupakan metode baru yang didasari dengan hans moment pada lapisan pertama dalam CNN.
	Hasil	Dengan metode ini berdasarkan 3 data set yaitu AVLetters dengan accuracy maks 59,23%, OuluVS2 dengan <i>accuracy</i> maks 93,72% dan BBC LRW dengan <i>accuracy</i> maks 90,86% menghasilkan perbandingan yang signifikan dengan karya-karya lain dalam literatur.
3.	Judul, Penulis, Tahun	<i>Turkish lip-reading using Bi-LSTM and deep learning models</i> (Atila & Sabaz, 2022)
	Permasalahan atau Tujuan	Karena lip reading sudah berkembang sangat pesat, maka penelitian ini mencoba mendeteksi perkataan seseorang dari <i>frame</i> video tanpa suara. Pada penelitian <i>lip reading</i> lainnya seperti Cina, Korea, Inggris, dan Jerman mengatakan penelitian ini sangat susah karena dipengaruhi oleh faktor lain seperti pencahayaan, jarak, dan juga jenis kelamin.
	Metode	Di awal pengembangan penelitian lip reading banyak yang menggunakan metode <i>Machine Learning</i> , namun seiring berjalannya waktu berkembang dengan adanya metode <i>Deep Learning</i> yang menghasilkan lebih bagus untuk hasil penelitiannya, maka dari itu dalam penelitian ini yaitu <i>Turki Lip Reading</i> digunakan metode <i>Deep Learning</i> agar bisa mendapat hasil yang optimal. Dalam penelitian ini digunakan 2

		dataset baru yaitu 111 kata dan 113 dengan kalimat yang sudah dibuat dengan Teknik image processing. Untuk ekstraksi fitur dari <i>frame</i> video menggunakan CNN dan untuk klasifikasi digunakan Bi-LSTM.
	Hasil	Hasil dari penelitian ini dengan ResNet-18 dan Bi-LSTM mendapatkan hasil yang terbaik dengan akurasi 84,5% dan 88,55%, dimana didapatkan juga hasil pengenalan dengan Kalimat lebih baik dari pengenalan dengan kata saja.
4.	Judul, Penulis, Tahun	<i>Read my lips: Artificial intelligence word-level arabic lipreading system</i> (Dweik et al., 2022)
	Permasalahan atau Tujuan	Dalam penelitian ini dijelaskan tujuan dari Lip reading yaitu untuk memfasilitasi seseorang yang mempunyai gangguan pendengaran dan berbicara serta membantu memahami perkataan dalam kondisi bising. Di sini khusus membahas mengenai pengenalan 10 kata dalam Bahasa Arab dengan gerakan mulut.
	Metode	Metode yang digunakan dalam penelitian ini yaitu <i>Three Deep Learning</i> dan <i>Neural Network Architectures</i> , dimana menggunakan 1051 data set.
	Hasil	Akurasi pengujian tertinggi (yaitu 82,84%) dicapai dengan memanfaatkan model pemungutan suara.
5.	Judul, Penulis, Tahun	<i>A Comprehensive Dataset for Machine-Learning-based Lip-Reading Algorithm</i> (Jin Ting et al., 2021)
	Permasalahan atau Tujuan	Pada penelitian ini menjelaskan kualitas data mempengaruhi sistem dalam Lip Reading, maka dari itu penelitian ini lebih menjabarkan bagaimana pembuatan dataset untuk Lip Reading.

	Metode	Cara yang digunakan dalam penelitian ini dengan mengekstraksi video asli dengan Scikit-Video, lalu deteksi wajahnya dilakukan dengan penerapan dlib. Kemudian untuk bibir ditangkap dengan memproses titik fitur untuk mencapai titik potong. Lalu untuk augmentasi dilakukan untuk memperbesar dataset.
	Hasil	Dataset yang dihasilkan memiliki 33 pembicara, masing-masing dengan 7.000 gambar gerakan bibir mereka.
6.	Judul, Penulis, Tahun	<i>Detection of human emotions through facial expressions using hybrid convolutional neural network-recurrent neural network algorithm</i> (Manalu & Rifai, 2024)
	Permasalahan atau Tujuan	Dalam penelitian ini membahas tentang FER. Tujuan penelitian lebih pengembangan metode model <i>hibrida CNN-RNN</i> .
	Metode	Metode yang digunakan sudah jelas yaitu <i>Convolutional Neural Network - Recurrent Neural Network</i> (CNN-RNN), dimana digunakan menggunakan dataset <i>Emotional Wearable Dataset 2020</i> yang terdiri dari empat ekspresi yaitu hiburan, antusiasme, kekaguman, dan kesukaan - belum pernah dieksplorasi dalam dataset sebelumnya. Model yang digunakan untuk klasifikasi yaitu <i>MobileNetV2-RNN</i> , <i>InceptionV3-RNN</i> , dan <i>CNN-RNN</i>
	Hasil	Model CNN-RNN khusus mencapai tingkat akurasi 63%, sedangkan model pembelajaran Transfer <i>MobileNetV2-RNN</i> dan <i>InceptionV3-RNN</i> masing-masing menghasilkan 59% dan 66%. Model yang

		dikembangkan menunjukkan peningkatan efisiensi dalam membedakan emosi
7.	Judul, Penulis, Tahun	<i>Lip Reading Using Convolutional Neural Networks with and without Pre-Trained Models</i> (OZCAN & BASTURK, 2019)
	Permasalahan atau Tujuan	Pada penelitian ini mengambil penelitian Lip Reading dari data Video.
	Metode	Metode yang digunakan dengan pengaplikasian <i>Convolutional Neural Networks</i> (CNNs) dengan <i>dataset AvLetters Standar</i> dan juga <i>dataset AvLetters</i> yang telah ditambah digunakan untuk tahap pelatihan dan pengujian.
	Hasil	Hasil eksperimen menunjukkan bahwa augmentasi data adalah cara yang baik untuk mendapatkan lebih banyak hasil yang sukses
8.	Judul, Penulis, Tahun	<i>The Facial Emotion Recognition (FER-2013) Dataset for Prediction System of Micro-Expressions Face Using the Convolutional Neural Network (CNN) Algorithm based Raspberry Pi</i> (Zahara et al., 2020)
	Permasalahan atau Tujuan	Pada penelitian ini menjelaskan saat memprediksi ekspresi atau emosi beberapa orang yang terkadang kurang jelas. Penelitian ini mengusulkan perancangan sistem yang dapat memprediksi dan mengenali emosi pada wajah.
	Metode	Metode yang digunakan untuk ekstraksi fitur menggunakan algoritma <i>Convolution Neural Network</i> (CNN) secara <i>real time</i> dengan <i>library OpenCV</i> yaitu TensorFlow dan Keras. Penelitian ini diimplementasikan pada <i>Raspberry Pi</i> terdiri dari tiga

		proses utama, yaitu: deteksi wajah, ekstraksi fitur wajah, dan klasifikasi emosi wajah.
	Hasil	Hasil prediksi ekspresi wajah pada penelitian dengan metode <i>Convolutional Neural Network</i> (CNN) menggunakan <i>Facial Emotion Recognition</i> (FER-2013) adalah sebesar 65,97%
9.	Judul, Penulis, Tahun	Lip-reading with Densely Connected Temporal Convolutional Networks (Ma <i>et al.</i> , 2021)
	Permasalahan atau Tujuan	Karena dianggap kurang cukup baik metode <i>Temporal Convolutional Networks</i> (TCN) dalam membaca gerak bibir, maka pada penelitian ini memperkenalkan fitur jaringan temporal yang lebih bagus dengan menggunakan <i>Squeezeand-Excitation block</i> .
	Metode	Metode yang digunakan yaitu Temporal Convolutional Networks (TCN) dengan menggunakan Squeezeand-Excitation block.
	Hasil	Metode DC-TCN kami telah mencapai akurasi 88,36% pada dataset Lip Reading in the Wild (LRW) dan 43,65% pada dataset LRW-1000, yang telah melampaui semua metode awal dan merupakan yang tercanggih di kedua dataset.
10.	Judul, Penulis, Tahun	Lipreading with DenseNet and resBi-LSTM (X. Chen <i>et al.</i> , 2020)
	Permasalahan atau Tujuan	Penelitian ini memperkenalkan metode sederhana untuk membuat dataset Lip Reading dalam Bahasa Mandarin pada Kalimat dalam berita, pidato, dan talkshow.
	Metode	Metode yang digunakan dalam penelitian ini yaitu metode sederhana yaitu 3D Convolutional Layer dengan DenseNet dan Residual Bidirectional Long

		Short-Term Memory dimana menggunakan 1705 yang ada dan Hanyu Pinyin (transkripsi fonemik bahasa Mandarin) sebagai label dan secara keseluruhan yang memiliki 349 kelas.
	Hasil	Untuk akurasi Hanyu Pinyin dan karakter Cina, jaringan yang diusulkan memiliki peningkatan 13,91% dan 14,99% secara absolut oleh LipNet, sementara peningkatan absolut 4,68% dan 4,74% dibandingkan dengan metode ResNet.
11.	Judul, Penulis, Tahun	Lip reading using CNN and LSTM (Garg <i>et al.</i> , n.d.)
	Permasalahan atau Tujuan	Merancang metode untuk memprediksi kata dan frasa pada video tanpa suara dimana melihat dari kasus nyata dimana dalam kondisi bising tidak dapat mendengar informasi yang disampaikan secara jelas
	Metode	Penelitian ini menggunakan 3 metode yaitu pertama dengan menggabungkan beberapa gambar menjadi satu gambar, kedua dengan nearest-neighbor interpolation, ketiga dengan VGGNet untuk mengekstrak satu set fitur, dan kemudian melewati setiap urutan fitur melalui beberapa lapisan LSTM
	Hasil	Hasil yang didapat model gambar gabungan yang menggunakan <i>nearest-neighbor interpolation</i> berkinerja baik, mencapai akurasi validasi 76%
12.	Judul, Penulis, Tahun	SIBI <i>Dynamic Gesture Translation Using MediaPipe and Long Short-Term Memory in Real-Time</i> (Kurniawan & Kaswidjanti, 2024)
	Permasalahan atau Tujuan	Permasalahannya diangkat dari kinerja dengan data statis dan menghadapi keterbatasan dalam menangani

		data berurutan dan dinamis seperti gerakan isyarat Sistem Bahasa Isyarat Indonesia (SIBI).
	Metode	Metode yang digunakan yaitu <i>Long Short-Term Memory</i> (LSTM) dimana memiliki arsitektur fleksibel dan dapat menyesuaikan secara dinamis untuk mengakomodasi berbagai panjang urutan input dan pemrosesan data menggunakan kerangka MediaPipe untuk mengekstrak landmark tangan dari setiap titik data.
	Hasil	Kombinasi ini menghasilkan akurasi kategoris 99,85%, kehilangan 0,0059, akurasi kategoris validasi 100%, dan kehilangan validasi 0,0064 setelah 150 periode pelatihan.
13.	Judul, Penulis, Tahun	IMPLEMENTASI ALGORITMA <i>CONVOLUTIONAL NEURAL NETWORK</i> UNTUK MENERJEMAHKAN BAHASA ISYARAT (Amri <i>et al.</i> , 2024)
	Permasalahan atau Tujuan	Penelitian ini bertujuan untuk memfasilitasi komunikasi yang efektif dengan individu dengan gangguan pendengaran.
	Metode	Penelitian ini menjelaskan implementasi algoritma <i>Convolutional Neural Network</i> (CNN) menggunakan YOLOV5 untuk menerjemahkan Bahasa Isyarat ke dalam teks.
	Hasil	Hasil eksperimen menunjukkan kemampuan YOLOV5 dalam menerjemahkan Bahasa Isyarat dengan tingkat akurasi yang memuaskan, dimana pengujian model didapat akurasi tertinggi sebesar 94%

14.	Judul, Penulis, Tahun	<i>Indonesian Lip-Reading Detection and Recognition Based on Lip Shape Using Face Mesh and Long-Term Recurrent Convolutional Network</i> (Aripin & Setiawan, 2024)
	Permasalahan atau Tujuan	Komunikasi terhambat karena kebisingan lingkungan sehingga diperlukan metode alternatif seperti pembacaan bibir, namun selain itu pembacaan Gerakan bibir dipengaruhi oleh bentuk bibir sehingga diperlukan analisis lebih terperinci. Penelitian ini dilakukan selain untuk mengatasi masalah tersebut juga bertujuan untuk menambah dataset untuk pengembangan dalam bahasa Indonesia. Sistem ini mengusulkan sistem pembacaan gerak bibir yang ditingkatkan dan dilatih menggunakan LRCN dengan mempertimbangkan 8 jenis bentuk bibir yang berbeda.
	Metode	Metode yang digunakan dalam penelitian ini yaitu <i>Long-Term Recurrent Convolutional Network (LRCN)</i> dengan menggunakan <i>MediaPipe Face Mesh</i> untuk mendeteksi <i>Landmark</i> bibir dan dibandingkan dengan metode <i>Convolutional Long Short-Term Memory (Conv-LSTM)</i> .
	Hasil	Dengan metode LRCN mendapat akurasi sebesar 95,42% untuk data uji kata dan 95,63% untuk frasa mengungguli metode Conv-LSTM yaitu 90,67% untuk data uji kata dan 91% untuk data uji frasa.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Data dan Metode Pengumpulan Data

Data yang digunakan dalam penelitian ini merupakan data primer berupa video yang direkam dari seseorang yang sudah terlatih berbahasa isyarat SIBI (guru SLB dan siswa penyandang tunarungu).

3.1.1 Pengumpulan Data

Pengumpulan data merupakan langkah penting dalam penelitian ini karena kualitas dan keakuratan data sangat mempengaruhi hasil akhir dari model pengenalan gerak bibir. Video direkam dengan resolusi HD 1920x1080 dan *framerate* 30 fps. Video hasil rekaman dilakukan pengolahan awal dengan mengubah resolusi, menyamakan *framerate*, dan menghilangkan suara. Adapun peralatan yang digunakan dalam pengambilan data ini sebagai berikut:

- Kamera: Logitech C920 Webcam Pro Full HD 1080p.
- Laptop: IdeaPad 14ALC7
- Prosesor: AMD RYZEN 3 5000 Series dengan AMD RADEON Graphics
- Hardisk: Samsung SSD

Data yang digunakan diambil dari 10 guru dan 10 siswa sehingga keseluruhan data dari 20 orang. Setiap orang membuat video dengan data pilihan seperti alphabet (a, b, c, d, e, dan f), bilangan (1, 2, 3, 8, 9, dan 10), dan 6 kata dasar yaitu kata buku, dia, saya, keliling, kelompok, dan sekarang. Label data yang diambil diatur sedemikian rupa untuk memastikan variasi data, durasi/panjang pendeknya pelafalan bibir dari label yang digunakan, dan berdasarkan jumlah suku kata yang diwakilkan di seluruh label terpilih. Dari total label yang diambil kemudian direplikasi/diulang sebanyak 10 kali untuk setiap orangnya, sehingga total data yang didapat yaitu 3600 dataset untuk 18 label data.

3.1.2 Spesifikasi Data

Dari total 3600 data angka dibagi menjadi 70% data untuk training, 20% data untuk validasi, dan 10% data untuk testing. Data training di ambil dari setiap subjek dari pengulangan 1-7, kemudian untuk data validasi di ambil dari pengulangan 8 dan 9, dan untuk data testing diambil dari pengulangan ke 10. Hal ini di atur sedemikian rupa untuk memastikan setiap pembagian data rata tanpa ada kehilangan data dan memastikan data yang digunakan sesuai porsi.

3.1.3 Pemilihan Label Berdasarkan Suku Kata

Pemilihan 18 label pada penelitian ini tidak dilakukan secara acak, melainkan mempertimbangkan variasi jumlah suku kata sebagai representasi perbedaan durasi dan kompleksitas gerak bibir. Prinsipnya adalah memastikan dataset mencakup label dengan pola gerak bibir yang pendek, menengah, dan panjang secara proporsional.

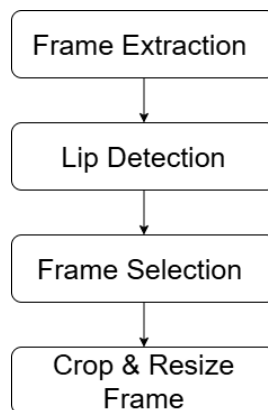
Tabel 3. 1 Klasifikasi Label Berdasarkan Jumlah Suku Kata

Kategori	Label	Pelafalan	Suku Kata	Jumlah Suku Kata
Alphabet	a	/a/	a	1
Alphabet	b	/be/	be	1
Alphabet	c	/ce/	ce	1
Alphabet	d	/de/	de	1
Alphabet	e	/e/	e	1
Alphabet	f	/ef/	ef	1
Angka	1	satu	sa-tu	2
Angka	2	dua	du-a	2
Angka	3	tiga	ti-ga	2
Kata	buku	buku	bu-ku	2
Kata	dia	dia	di-a	2
Kata	saya	saya	sa-ya	2
Angka	8	delapan	de-la-pan	3
Angka	9	sembilan	sem-bi-lan	3
Angka	10	sepuluh	se-pu-luh	3
Kata	keliling	keliling	ke-li-ling	3
Kata	kelompok	kelompok	ke-lom-pok	3
Kata	sekarang	sekarang	se-ka-rang	3

Distribusi label per jumlah suku kata dirancang seimbang yaitu, 6 label 1 suku kata (semua alphabet), 6 label 2 suku kata (angka 1,2,3 dan kata buku, dia, saya), dan 6 label 3 suku kata (angka 8,9,10 dan kata keliling, kelompok, sekarang). Keseimbangan ini penting agar model tidak bias terhadap satu pola durasi tertentu.

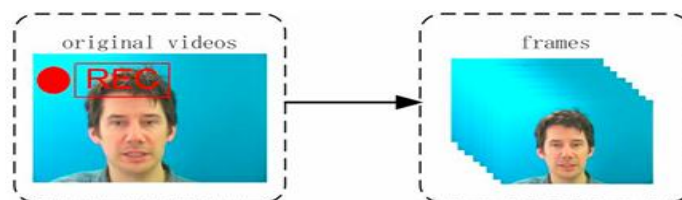
3.2 Preprocessing Data

Tahap preprocessing bertujuan untuk mengolah data video mentah menjadi representasi fitur yang siap digunakan dalam proses pelatihan model. Karena input berupa video dengan resolusi dan durasi yang bervariasi, diperlukan beberapa tahapan untuk menyeragamkan dan mengekstraksi informasi yang relevan, yaitu hanya area bibir. Pada tahapan ini data video angka melalui proses seperti yang ditampilkan pada Gambar 3.1.



Gambar 3. 1 Alur Preprocessing Data

Tahap pertama yang dilakukan adalah ekstraksi *frame*. Video terdiri dari banyak *frame* (gambar) yang ditampilkan dalam urutan cepat untuk menciptakan ilusi gerak. Setiap *frame* adalah gambar statis. Ekstraksi *frame* melibatkan mengakses setiap *frame* secara berurutan atau pada interval tertentu.



Gambar 3. 2 Potongan Video Menjadi Beberapa *Frame* (Jin Ting *et al.*, 2021)

Tahap kedua adalah deteksi area bibir menggunakan *MediaPipe FaceLandmarker* (Tasks API versi 0.10). Library ini dipilih karena mampu mendeteksi 478 titik landmark wajah secara akurat, termasuk titik-titik yang merepresentasikan kontur bibir. Deteksi dilakukan menggunakan 40 titik landmark bibir yang terdiri dari 20 titik OUTER_LIP dan 20 titik INNER_LIP. Dari hasil deteksi, koordinat *bounding box* area bibir dihitung berdasarkan nilai minimum dan maksimum koordinat landmark, kemudian diperluas dengan margin 30% ke semua sisi agar area bibir tidak terpotong.

Tahap ketiga adalah seleksi *frame* berbasis skor gerak (*motion score*). Tidak semua *frame* dalam sebuah video mengandung gerakan bibir yang signifikan. Mekanisme seleksi ini bekerja dengan menghitung selisih piksel antar *frame* pada *thumbnail* 32x32 piksel *grayscale* untuk efisiensi komputasi. Dari seluruh *frame* yang berhasil terdeteksi, dipilih 30 *frame* dengan skor gerak tertinggi kemudian diurutkan kembali secara temporal, sehingga informasi urutan gerak bibir tetap terjaga.

Tahap keempat adalah *crop* dan *resize*. *Frame-frame* terpilih di-*crop* berdasarkan koordinat *bounding box* bibir, lalu di-*resize* ke ukuran 96x96 piksel. Hasil akhir dari setiap video berupa *array numpy* dengan dimensi [30, 96, 96, 3] yang merepresentasikan 30 *frame crop* bibir dalam format RGB. Array ini disimpan dalam format *.npy* dan diorganisasikan ke dalam struktur folder sesuai partisi data (train/val/test) serta label kelasnya. Gambar 3.3 menampilkan contoh *crop* dan *resize* dari penelitian (Jin Ting *et al.*, 2021).



Gambar 3. 3 Pendeteksian Bibir Menggunakan Koordinat *Landmark* (Jin Ting *et al.*, 2021)

3.3 Optical Flow Lucas-Kanade

Proses *Optical Flow* di sini digunakan untuk proses pengecekan model *Optical flow Conv-LSTM*. Dari hasil preprosesing yang sudah dilakukan sebelumnya, *array lip crop* RGB berukuran $[30, 96, 96, 3]$ digunakan sebagai input untuk tahap perhitungan *optical flow*. Metode yang digunakan adalah Lucas-Kanade, yaitu metode *sparse optical flow* yang menghitung pergerakan titik-titik fitur tertentu antar *frame* yang berdekatan.

Proses dimulai dengan mendeteksi titik-titik fitur (*feature points*) pada setiap *frame* menggunakan fungsi `goodFeaturesToTrack`, kemudian pergerakan titik-titik tersebut dilacak ke *frame* berikutnya menggunakan `calcOpticalFlowPyrLK`. Hasil dari Lucas-Kanade bersifat *sparse* (hanya pada titik tertentu), sehingga perlu dilakukan interpolasi ke seluruh piksel menggunakan `scipy.griddata` sehingga dihasilkan *dense flow map*. Output akhir dari proses ini berupa *3-channel map* yang merepresentasikan komponen horizontal (u), vertikal (v), dan magnitudo pergerakan piksel. Karena *optical flow* dihitung antar dua *frame* yang berurutan, dari 30 *frame* RGB dihasilkan 29 *frame optical flow* dengan dimensi *array* $[29, 96, 96, 3]$.

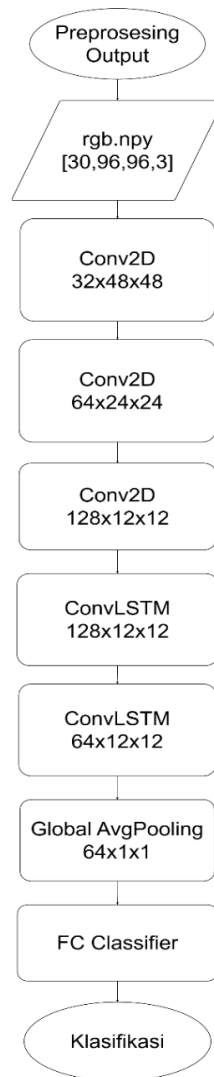
3.4 Perancangan Arsitektur Model

Penelitian ini merancang dua model utama yang dibandingkan performanya, yaitu model Conv-LSTM sebagai *baseline* dan model Optical Flow Conv-LSTM sebagai model utama.

3.4.1 Baseline Model Conv-LSTM

Model Conv-LSTM *baseline* menerima input berupa sekuens 30 *frame lip crop* RGB $[30, 96, 96, 3]$. Setiap *frame* diproses oleh *backbone* CNN yang terdiri dari tiga blok konvolusi dengan jumlah filter 32, 64, dan 128 secara berurutan, mereduksi dimensi spasial dari 96×96 menjadi 12×12 . Representasi spasial per *frame* kemudian diproses secara temporal oleh dua lapisan ConvLSTM (128 dan 64 *hidden channel*), diikuti *Global Average Pooling* dan dua lapisan *fully connected*

untuk menghasilkan prediksi kelas akhir. Alur desain model *baseline* dapat dilihat pada Gambar 3.4.



Gambar 3. 4 Alur Model *Optical Flow Conv-LSTM*

3.4.2 Model *Optical Flow Conv-LSTM*

Model *Optical Flow Conv-LSTM* dirancang dalam dua varian. Pertama, varian *single-stream* yang hanya menggunakan *optical flow map* [29, 96, 96, 3] sebagai input dengan arsitektur *backbone* yang identik dengan model *baseline*. Pada varian ini, model belajar mengenali pola gerakan bibir semata-mata dari informasi pergerakan piksel yang dihasilkan oleh Lucas-Kanade, tanpa melibatkan informasi

visual RGB. Kedua, varian *dual-stream* yang memproses RGB dan *optical flow* secara paralel melalui dua modul StreamEncoder yang independen, di mana masing-masing StreamEncoder memiliki susunan *backbone* CNN dan ConvLSTM yang sama seperti model *baseline*, namun tanpa lapisan *classifier* di ujungnya sehingga hanya menghasilkan vektor fitur berukuran 64 dimensi.

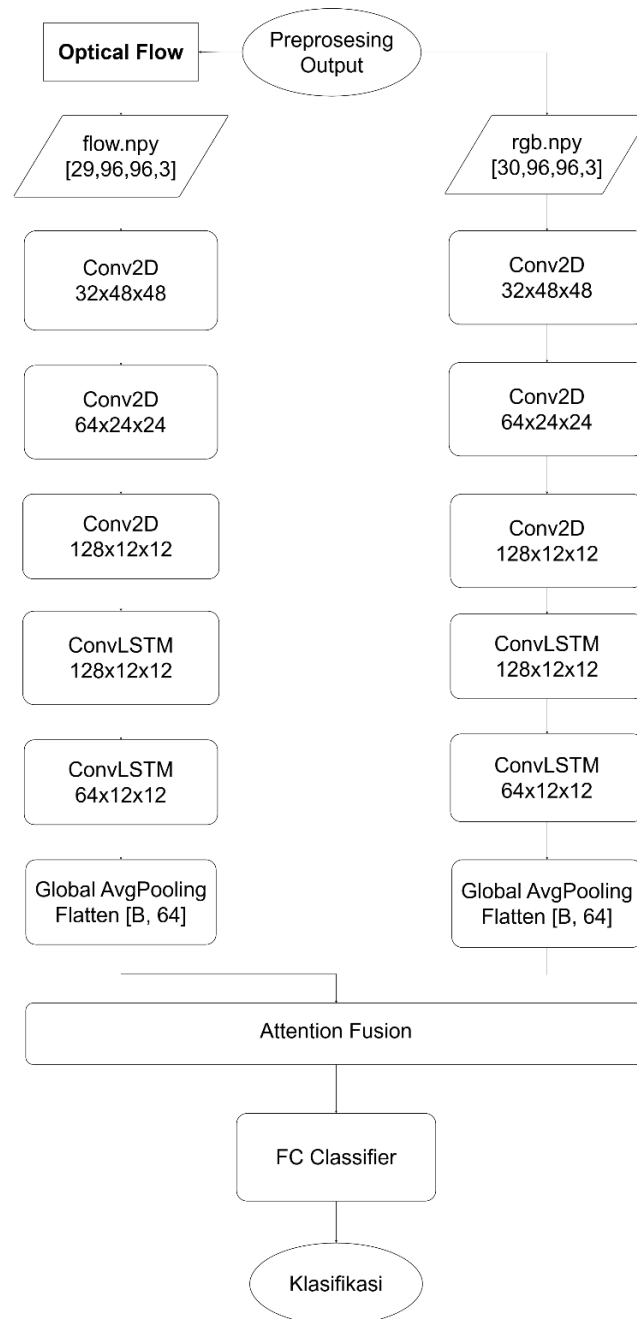
Representasi fitur dari kedua stream kemudian tidak langsung dijumlahkan atau digabung begitu saja, melainkan melewati mekanisme AttentionFusion untuk menentukan seberapa besar kontribusi masing-masing stream terhadap keputusan akhir model. Cara kerja AttentionFusion dimulai dengan menggabungkan kedua vektor fitur f_{rgb} dan f_{flow} menjadi satu vektor gabungan. Dari vektor gabungan ini, dua buah gate dihitung secara terpisah menggunakan lapisan *linear* dan fungsi *sigmoid*:

$$\begin{aligned} g_{rgb} &= \sigma(W_{rgb} \cdot [f_{rgb} || f_{flow}]) \\ g_{flow} &= \sigma(W_{flow} \cdot [f_{rgb} || f_{flow}]) \\ f_{fused} &= g_{rgb} \odot f_{rgb} + g_{flow} \odot f_{flow} \end{aligned}$$

di mana $f_{rgb} || f_{flow}$ adalah hasil penggabungan (*concatenation*) kedua vektor fitur, W_{rgb} dan W_{flow} adalah bobot yang dipelajari selama proses pelatihan, σ adalah fungsi *sigmoid* yang menghasilkan nilai antara 0 dan 1, serta $\odot$ adalah perkalian elemen per elemen. Vektor hasil fusi f_{fused} inilah yang kemudian diteruskan ke lapisan *fully connected* untuk menghasilkan prediksi kelas akhir.

Nilai *gate* yang dihasilkan *sigmoid* berada di antara 0 dan 1, sehingga dapat diinterpretasikan sebagai seberapa penting fitur dari masing-masing stream untuk sampel yang sedang diproses. Sebagai contoh, apabila kualitas *optical flow* pada suatu video kurang baik, model akan secara otomatis mengecilkan nilai g_{flow} dan lebih mengandalkan fitur dari *stream* RGB. Dengan demikian, mekanisme AttentionFusion membuat model lebih robust dibandingkan apabila kedua *stream*

digabungkan secara sederhana. Alur model untuk varian *dual-stream* secara keseluruhan dapat dilihat pada Gambar 3.5.



Gambar 3. 5 Alur Model Dual Stream *Optical Flow Conv-LSTM*

3.5 Perancangan Pelatihan Model

Pelatihan model dilakukan dengan konfigurasi *hyperparameter* yang seragam untuk memastikan perbandingan yang adil antar model. *Optimizer* yang digunakan adalah Adam dengan *learning rate* awal 0,001 dan *weight decay* $1e-5$. Pelatihan dilakukan maksimal 100 epoch dengan *batch size* 8. Penyesuaian *learning rate* dilakukan secara otomatis menggunakan ReduceLROnPlateau dengan faktor 0,5 dan *patience* 8 epoch apabila tidak terjadi peningkatan pada *validation loss*. Fungsi *loss* yang digunakan adalah CrossEntropyLoss untuk klasifikasi multi-kelas.

3.6 Skenario Pengujian

Penelitian ini dilakukan dalam beberapa tahap pengujian yang terstruktur sebelum menghasilkan model akhir pada dataset Gabungan. Seluruh pengujian menggunakan pendekatan *One Factor At A Time* (OFAT), yaitu hanya mengubah satu variabel pada setiap pengujian agar pengaruh perubahan tersebut dapat diisolasi dan dianalisis secara adil.

3.6.1 Skenario 1: Perbandingan Metode Optical Flow

Pengujian pertama dilakukan pada dataset *Number* (10 kelas: 1–10) dengan tujuan menentukan metode perhitungan *optical flow* yang paling sesuai sebelum diimplementasikan pada dataset gabungan.

a. Farneback Dense Optical Flow

Metode Farneback merupakan metode *dense optical flow* yang menghitung vektor pergerakan untuk setiap piksel secara langsung. Parameter yang digunakan adalah *pyr_scale*=0.5, *levels*=3, *winsize*=15, *iterations*=3, *poly_n*=5, dan *poly_sigma*=1.2. Keunggulan metode ini adalah menghasilkan *flow map* yang sangat padat tanpa memerlukan *interpolasi*, namun memiliki beban komputasi lebih tinggi. Dapat dilihat pada Tabel 3. 2 menampilkan potongan kode yang merepresentasikan metode Farneback beserta parameter yang digunakan.

Tabel 3. 2 Potongan kode Farneback

```

FB_PARAMS = dict(
    pyr_scale=0.5, levels=3, winsize=15,
    iterations=3, poly_n=5, poly_sigma=1.2, flags=0
)

def compute_flow(frames):
    flows = []
    for t in range(len(frames) - 1):
        flow = cv2.calcOpticalFlowFarneback(
            frames[t], frames[t+1], None,
            **FB_PARAMS)
        u, v = flow[..., 0], flow[..., 1]
        flows.append(np.stack([u, v, np.sqrt(u**2+v**2)],
            axis=-1))
    return np.array(flows, dtype=np.float32)

```

b. Lucas-Kanade Sparse Optical Flow

Metode *Lucas-Kanade* menghitung pergerakan pada titik-titik fitur yang terdeteksi secara *sparse*, kemudian diinterpolasi ke seluruh piksel menggunakan *scipy.griddata*. Konfigurasi awal yang digunakan adalah parameter *maxCorners=200*, *qualityLevel=0.015*, *blockSize=3*.

Tabel 3. 3 Potongan kode implementasi Lucas-Kanade

```

FEAT_PARAMS = dict(maxCorners=200, qualityLevel=0.015,
    minDistance=5, blockSize=3)
LK_PARAMS = dict(winSize=(15,15), maxLevel=3,
    criteria=(cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT, 30,
    0.01))

def compute_lk_flow(frames):
    flows = []
    for t in range(len(frames) - 1):
        p0 = cv2.goodFeaturesToTrack(frames[t], mask=None,
            **FEAT_PARAMS)
        p1, status, _ = cv2.calcOpticalFlowPyrLK(
            frames[t], frames[t+1],
            p0, None, **LK_PARAMS)
        mask = status.ravel() == 1
        old, new = p0[mask].reshape(-1,2),
        p1[mask].reshape(-1,2)
        flows.append(sparse_to_dense(old, new - old, H, W))
    return np.array(flows, dtype=np.float32)

```


3.6.2 Skenario 2: OFAT Parameter Lucas-Kanade (Dataset Number)

Setelah metode optimal dipilih dari Skenario 1, dilakukan pengujian OFAT terhadap parameter-parameter kunci yang mempengaruhi jumlah dan kualitas titik fitur yang dideteksi. Total 18 konfigurasi parameter diuji, dengan setiap konfigurasi melatih ulang model Conv-LSTM dari awal menggunakan data *optical flow* yang dihasilkan oleh parameter tersebut.

Tabel 3. 4 Matriks Konfigurasi OFAT Parameter Lucas-Kanade

<i>ID</i>	<i>maxCorners</i>	<i>quality Level</i>	<i>minDistance</i>	<i>block Size</i>	<i>winSize</i>	<i>maxLevel</i>
Base LK	200	0.01	5	5	15x15	3
P1	100	0.01	5	5	15x15	3
P2	300	0.01	5	5	15x15	3
P3	200	0.005	5	5	15x15	3
P4	200	0.02	5	5	15x15	3
P5	200	0.01	7	5	15x15	3
P6	200	0.01	9	5	15x15	3
P7	200	0.01	5	5	15x15	1
P8	200	0.01	5	5	15x15	2
P9	100	0.005	5	5	15x15	3
P10	100	0.02	5	5	15x15	3
P11	150	0.01	5	5	15x15	3
P12	150	0.005	5	6	15x15	3
P13	150	0.02	5	6	15x15	3
P14	250	0.01	5	6	15x15	3
P15	250	0.005	5	6	15x15	3
P16	250	0.02	5	6	15x15	3
P17	300	0.005	5	6	15x15	3
P18	300	0.02	5	6	15x15	3

3.6.3 Skenario 3: Pelatihan Model pada Dataset Gabungan

Setelah parameter di skenario 2 terpilih sebagai konfigurasi terbaik, dilakukan pelatihan ulang tiga varian model pada dataset Gabungan 18 kelas: *Baseline* RGB Conv-LSTM, LK *Single-Stream*, dan *Dual-Stream* RGB + LK terpilih. Skenario ini merupakan eksperimen utama yang hasilnya menjadi perbandingan akhir penelitian.

3.6.4 Skenario 4: OFAT Arsitektur Model Conv-LSTM

Sebagai pengujian tambahan untuk menganalisis sensitivitas model terhadap variasi arsitektur, dilakukan OFAT pada komponen arsitektur model *baseline*. Enam variasi (M01–M06) dirancang dengan mengubah satu komponen pada setiap variasi.

Tabel 3. 5 Variasi OFAT Arsitektur Model Conv-LSTM

<i>ID</i>	<i>CNN Channels</i>	<i>LSTM Hidden</i>	<i>Dropout</i>	<i>FC Hidden</i>	<i>Yang Diubah</i>
M01	32, 64, 128	128, 64	0.5	256	-
M02	32, 64, 128	128, 128	0.5	256	LSTM hidden layer-2 x2
M03	32, 64, 128	256, 128	0.5	256	LSTM hidden keduanya naik
M04	32, 64, 128	128, 128, 64	0.5	256	Tambah 1 ConvLSTM layer
M05	64, 128, 256	128, 64	0.5	256	CNN chanel lebih besar
M06	32, 64, 128	128, 64	0.3	256	Dropout dikurangi

3.6.5 Skenario 5: Penerapan Arsitektur Terbaik pada Dual-Stream

Dari hasil Skenario 4, arsitektur terbaik diidentifikasi dan diterapkan kembali pada konfigurasi *dual-stream*. Tujuannya adalah mengetahui apakah perbaikan arsitektur yang terbukti meningkatkan performa pada model *single-stream* RGB juga memberikan peningkatan serupa ketika dikombinasikan dengan *stream optical flow* parameter terpilih. Pengujian ini dilakukan pada model parameter terpilih+arsitektur terbaik untuk *single-stream* maupun *dual-stream*.

3.7 Perancangan Evaluasi Model

Evaluasi model dilakukan pada data test yang sepenuhnya terpisah dari proses pelatihan. Metrik yang digunakan meliputi akurasi (6.14) keseluruhan, *presisi* (6.15), *recall* (6.16), dan *F1-score* (6.17) per kelas, yang dihitung dari *confusion matrix* hasil prediksi. Perbandingan dilakukan antar tiga varian model (RGB

baseline, optical flow single-stream, dan dual-stream) untuk menganalisis kontribusi masing-masing komponen terhadap performa sistem secara keseluruhan.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Preprocessing Data

Tahap preprocessing dilakukan terhadap seluruh 3.600 video dari 18 label yang terdiri dari kategori angka, alphabet, dan kata. Setiap video diproses melalui pipeline yang telah dirancang pada Bab 3, yaitu ekstraksi *frame*, deteksi bibir menggunakan MediaPipe FaceLandmarker (*Tasks API*), seleksi 30 *frame* berbasis *motion score*, dan *cropping* area bibir ke ukuran 96×96 piksel. Hasil preprocessing menunjukkan tingkat keberhasilan 100%, di mana seluruh 3.600 video berhasil menghasilkan file .npy dengan *shape* [30, 96, 96, 3] tanpa ada data yang gagal diproses. Distribusi data hasil preprocessing dapat dilihat pada Tabel 4.1.

Tabel 4. 1 Distribusi Data Hasil Preprocessing

Split	Jumlah Video	Jumlah Label	Video per Label
Train	2520	18	140
Val	720	18	40
Test	360	18	20
Total	3600	18	200

Pada proses deteksi bibir, digunakan sebanyak 40 titik landmark bibir yang terdiri dari 20 titik OUTER_LIP dan 20 titik INNER_LIP dari MediaPipe FaceLandmarker. Setiap *frame* terlebih dahulu di-resize ke 224×224 piksel sebelum dimasukkan ke detektor, hal ini bertujuan untuk menyeragamkan skala deteksi meskipun resolusi video asli bervariasi antara 800×800 hingga 1000×1000 piksel. Setelah *bounding box* bibir diperoleh, area tersebut diperluas dengan margin 30% ke semua sisi untuk memastikan seluruh area gerak bibir tertangkap dan tidak terpotong.

Mekanisme seleksi *frame* berbasis motion score bekerja dengan menghitung selisih piksel antar *frame* pada *thumbnail* 32×32 piksel *grayscale*. Pendekatan *thumbnail* ini dipilih untuk efisiensi komputasi. Dari seluruh *frame* yang berhasil

terdeteksi, dipilih 30 *frame* dengan skor gerak tertinggi kemudian diurutkan kembali secara temporal, sehingga informasi urutan gerak bibir tetap terjaga.

Hasil tersebut secara visual dapat dilihat pada Gambar 4.1 yang menampilkan contoh hasil *crop* bibir dari beberapa label berbeda. Terlihat bahwa setiap potongan bibir berhasil diekstraksi pada area yang konsisten dengan ukuran 96×96 piksel.



Gambar 4. 1 Contoh Hasil *Crop* Bibir Setiap Label

4.2 Hasil Perhitungan *Optical Flow Lucas-Kanade* Terpilih

Setelah preprocessing selesai, *array lip crop* RGB [30, 96, 96, 3] digunakan sebagai input untuk tahap perhitungan *optical flow* menggunakan metode Lucas-Kanade. Pada penelitian ini digunakan konfigurasi parameter terbaik yang sebelumnya sudah di uji, yaitu *maxCorners*=300, *qualityLevel*=0.005, *minDistance*=5, *blockSize*=6, *winSize*=15×15, dan *maxLevel*=3. Konfigurasi lengkap parameternya ditampilkan pada Tabel 4.2.

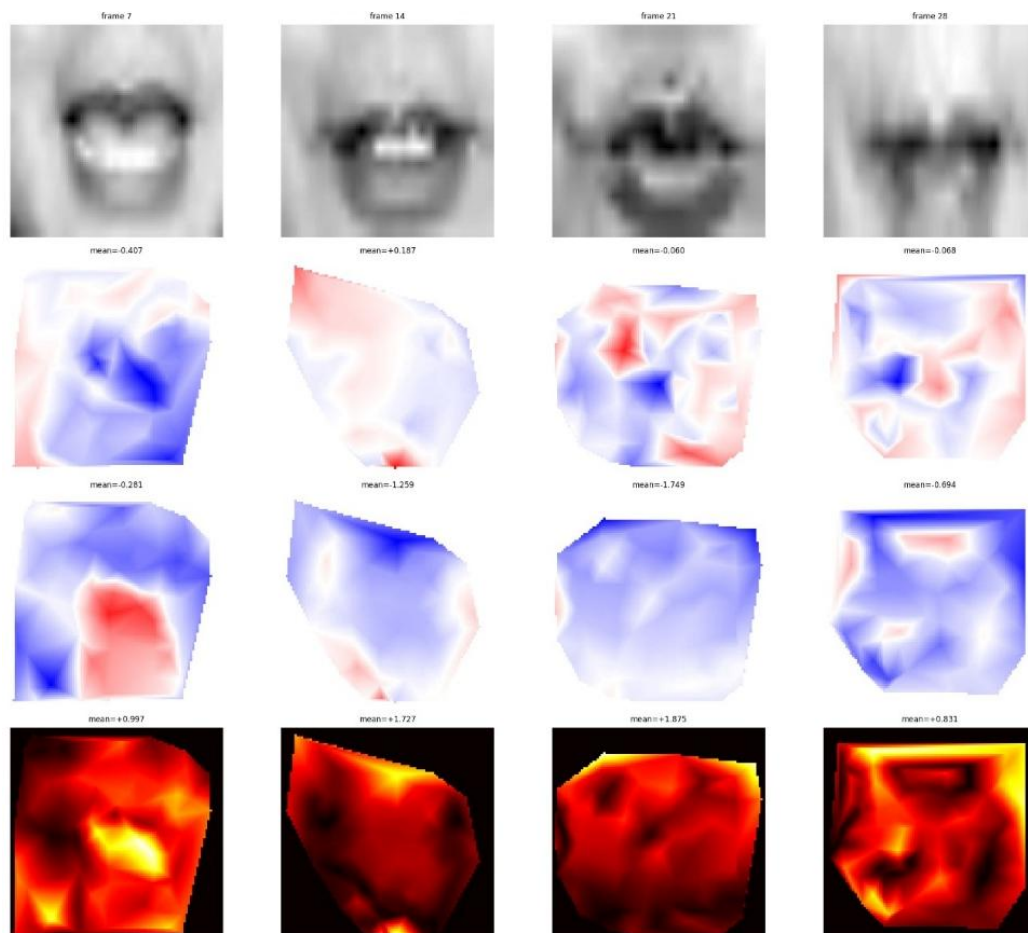
Tabel 4. 2 Konfigurasi Parameter Lucas-Kanade

Parameter	Nilai	Keterangan
<i>maxCorners</i>	300	Jumlah maksimum titik fitur yang dideteksi
<i>qualityLevel</i>	0.005	Ambang batas kualitas sudut minimum
<i>minDistance</i>	5	Jarak minimum antar titik fitur (piksel)
<i>blockSize</i>	6	Ukuran blok untuk komputasi turunan
<i>winSize</i>	15x15	Ukuran jendela pencarian aliran
<i>maxLevel</i>	3	Kedalaman piramida gambar
<i>Interpolasi</i>	linier	Metode griddata untuk dense flow

Proses perhitungan *optical flow* menghasilkan 29 *frame flow* untuk setiap video, karena *optical flow* dihitung antar dua *frame* yang berurutan sehingga dari

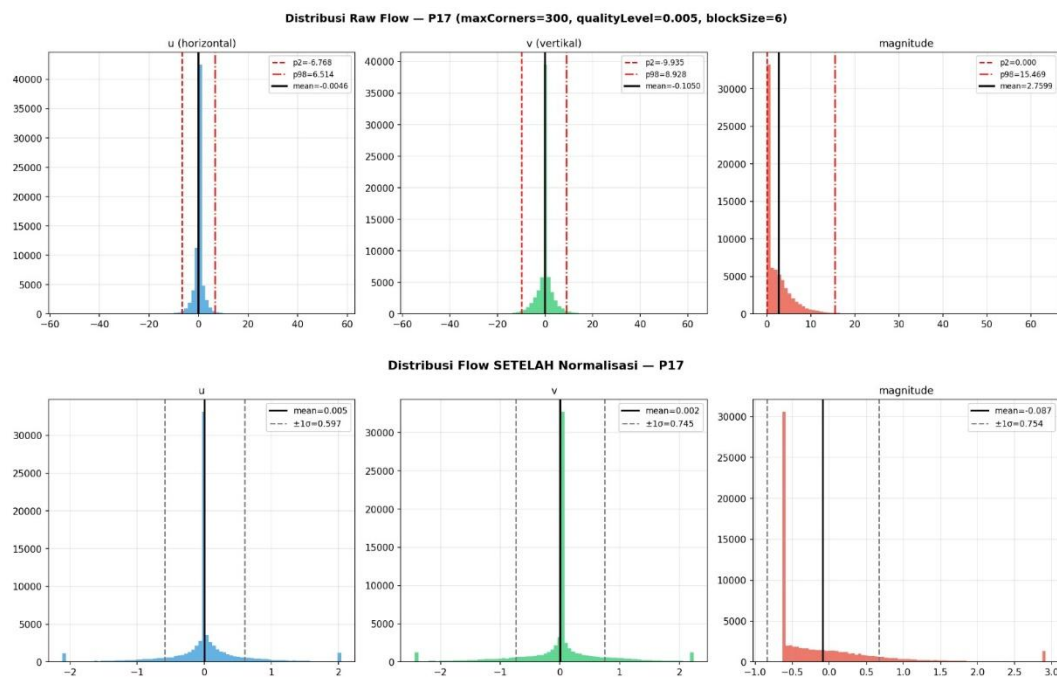
30 *frame* RGB dihasilkan 29 pasangan. Setiap *frame flow* memiliki 3 *channel* yang merepresentasikan komponen horizontal (u), vertikal (v), dan magnitudo pergerakan piksel seperti yang terlihat pada Gambar 2.4.

Normalisasi dilakukan menggunakan statistik global yang dihitung dari seluruh data training. Proses normalisasi terdiri dari dua tahap: pertama, nilai di-clip pada rentang persentil p2–p98 untuk membuang nilai ekstrem, kemudian dilakukan *z-score* normalisasi menggunakan *mean* dan standar deviasi global dari data training. Pendekatan ini dipilih karena normalisasi per-video atau per-*frame* akan menghilangkan informasi relatif besarnya gerakan antar sampel yang justru penting untuk membedakan antar kelas.



Gambar 4. 2 Visualisasi 3 Channel Optical Flow (*Frame Asli*, *Channel u*, *Channel v*, dan *Magnitude*)

Pada Gambar 4.3 (atas) menunjukkan distribusi nilai *raw flow* sebelum normalisasi, tampak bahwa distribusi nilai pada ketiga kanal (u, v, dan magnitudo) sangat bervariasi dan memiliki rentang yang lebar. Gambar 4.3 (bawah) menunjukkan distribusi setelah normalisasi global menggunakan *z-score*, terlihat bahwa distribusi menjadi lebih terpusat di sekitar nol dengan rentang yang terkendali, yang memudahkan proses pelatihan model.



Gambar 4. 3 Distribusi *Raw Flow* Sebelum Normalisasi dan Distribusi Setelah Normalisasi

4.3 Hasil Skenario 1: Perbandingan Farneback & Lucas-Kanade

Pengujian perbandingan dua metode *optical flow* dilakukan pada dataset Number (10 kelas) menggunakan model Conv-LSTM yang identik. Kedua metode menghasilkan *array flow* dengan *shape* [30, 96, 96, 3]. Hasil evaluasi ditampilkan pada Tabel 4.3, dimana hasil test akurasi terbaik dipilih menjadi metode yang diterapkan pada penelitian ini.

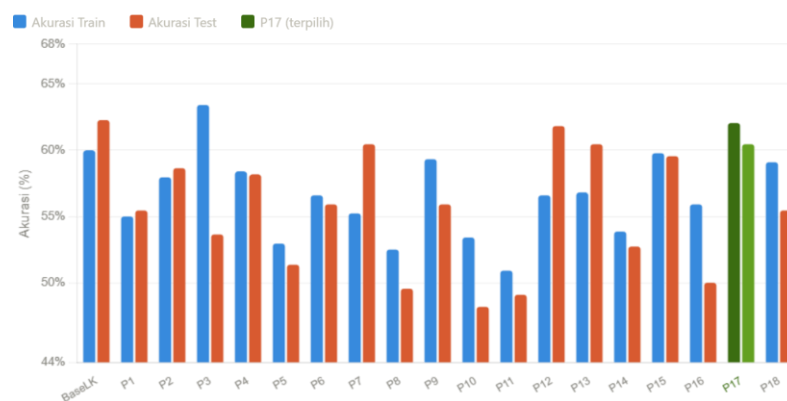
Tabel 4. 3 Perbandingan *Farneback* vs *Lucas-Kanade* pada Dataset Number

Metode	<i>Best Val Acc</i>	<i>Best Epoch</i>	<i>Test Acc</i>	<i>F1 Macro</i>	Waktu/ <i>Epoch</i>
Farneback	61.59%	80/100	57.73%	57.41%	~71 detik
Lucas-Kanade	60.00%	96/100	62.27%	61.15%	~77 detik

Meskipun *best validation accuracy* *Farneback* sedikit lebih tinggi (61.59% vs 60.00%), *test accuracy* *Lucas-Kanade* unggul signifikan yaitu 62.27% vs 57.73% (selisih +4.54%). Hal ini menunjukkan bahwa representasi *sparse-to-dense* dari *Lucas-Kanade* lebih mampu melakukan generalisasi pada data yang belum pernah dilihat. *Farneback* juga menunjukkan lebih banyak fluktuasi pada *validation accuracy* dan konvergensi yang baru signifikan di sekitar epoch 30-40, sedangkan *Lucas-Kanade* mulai konvergen lebih awal. Berdasarkan hasil ini, *Lucas-Kanade* dipilih sebagai metode *optical flow* untuk seluruh eksperimen selanjutnya.

4.4 Hasil Skenario 2: OFAT Parameter *Lucas-Kanade*

Hasil lengkap pengujian 18 konfigurasi parameter *Lucas-Kanade* pada dataset Number secara umum dapat dilihat pada Gambar 4. 4, namun untuk lebih lengkapnya dapat dilihat pada tabel Lampiran 2. Metrik utama yang digunakan untuk pemilihan parameter terbaik adalah *training accuracy* karena OFAT bertujuan memilih parameter yang memberikan kemampuan ekstraksi fitur optimal.

Gambar 4.4 | Perbandingan Akurasi OFAT Parameter *Lucas-Kanade* pada Dataset NumberGambar 4. 4 Hasil OFAT Parameter *Lucas-Kanade* pada Dataset Number

Parameter P17 dipilih sebagai konfigurasi terbaik karena memiliki *training accuracy* tertinggi (62.05%) dengan *test accuracy* yang kompetitif (60.45%). Tiga pola penting yang ditemukan dari hasil OFAT ini adalah: (1) menurunkan *qualityLevel* dari 0.01 menjadi 0.005 secara konsisten meningkatkan kemampuan deteksi titik fitur di area bibir, (2) menaikkan *blockSize* dari 3 ke 6 memberikan estimasi *gradien* yang lebih stabil, dan (3) *maxCorners*=300 menghasilkan *flow map* yang lebih kaya informasi dibanding *maxCorners*=200.

4.5 Hasil Skenario 3: Pelatihan Model pada Dataset Gabungan

Pelatihan dilakukan terhadap tiga varian model pada dataset gabungan 18 kelas dengan total 2.520 data training, 720 data validasi, dan 360 data testing. Semua model dilatih dengan konfigurasi *hyperparameter* dasar yang sama, yaitu optimizer Adam dengan *learning rate* awal 0,001, *weight decay* 1e-5, *batch size* 8, dan maksimal 100 *epoch*. Perbedaan konfigurasi antar model hanya pada nilai *patience scheduler* dan penggunaan *gradient clipping*, yang disesuaikan dengan karakteristik konvergensi masing-masing model.

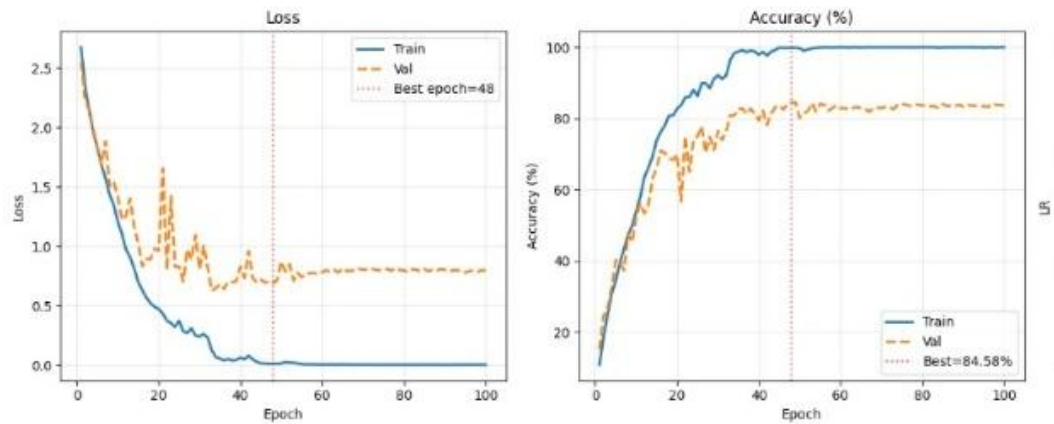
4.5.1 Baseline Conv-LSTM (RGB)

Model *baseline* menerima input berupa sekuens 30 *frame* lip crop RGB dengan *shape* [B, 3, 30, 96, 96]. Pelatihan model *baseline* selesai dalam waktu 98.1 menit untuk 100 *epoch*, dengan rata-rata sekitar 59 detik per *epoch*.

Proses konvergensi model *baseline* berlangsung relatif stabil. Pada *epoch* awal, model sudah menunjukkan peningkatan yang konsisten di mana pada *epoch* 1 *validation accuracy* baru mencapai 15.14%, kemudian meningkat pesat hingga *epoch* 16 mencapai 70.97%. Model terus membaik dan mencapai *validation accuracy* terbaik sebesar 84.58% pada *epoch* 48. Setelah *epoch* tersebut, scheduler ReduceLROnPlateau mulai menurunkan *learning rate* secara bertahap dari 1e-3 hingga akhirnya mencapai 1e-6 di *epoch* 100, namun performa validasi tidak lagi meningkat secara signifikan.

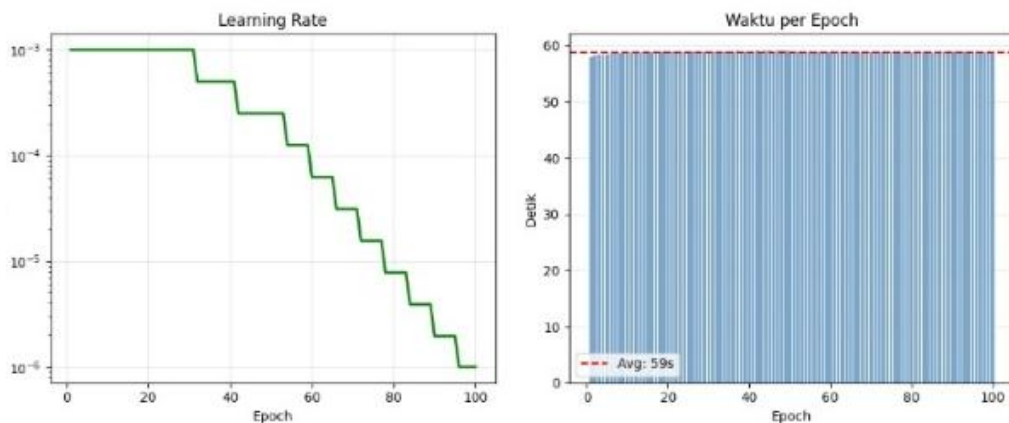
Gambar 4.5 menampilkan kurva *training* dan *validation loss/accuracy* model *baseline* Conv-LSTM selama 100 *epoch*. Kurva menunjukkan proses konvergensi

yang relatif stabil dengan validation accuracy terbaik 84,58% tercapai pada epoch 48.



Gambar 4. 5 Kurva *Training* dan *Validation Loss/accuracy* Model *Baseline*

Gambar 4. 6 menampilkan grafik penurunan *learning rate* yang dikelola oleh *scheduler ReduceLROnPlateau* beserta rata-rata waktu pelatihan per *epoch* model *baseline*. Penurunan *learning rate* terjadi secara bertahap seiring dengan stagnannya performa validasi.



Gambar 4. 6 Penurunan Learning Rate dan Rata-rata Waktu Training per Epoch
Model *Baseline*

Evaluasi pada data *test* menghasilkan *accuracy* sebesar 83.06% dengan detail metrik yang dapat dilihat pada Tabel 4. 4.

Tabel 4. 4 Hasil Evaluasi *Test Set Baseline Conv-LSTM*

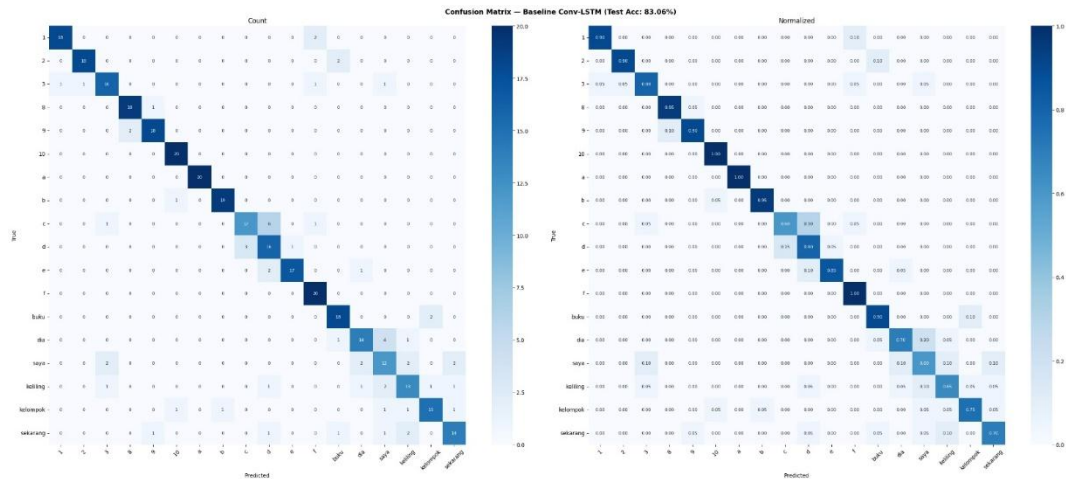
Metrik	Nilai
Test Loss	0.8074
Test Accuracy	83.06%
Precision	83.41%
Recall	83.06%
F1-score	82.96%
F1-score	82.96%
Best Epoch	48/100

Hasil per kelas menunjukkan variasi yang cukup signifikan. Kelas ‘a’, ‘10’, dan ‘b’ mencapai performa tertinggi dengan F1-score masing-masing 1.000, 0.952, dan 0.950. Di sisi lain, kelas ‘saya’ dan ‘keliling’ menjadi kelas dengan performa terendah dengan *F1-score* 0.585 dan 0.667. Detail akurasi per kelas dapat dilihat pada Tabel 4. 5.

Tabel 4. 5 Nilai Akurasi per Kelas *Test Set Baseline Conv-LSTM*

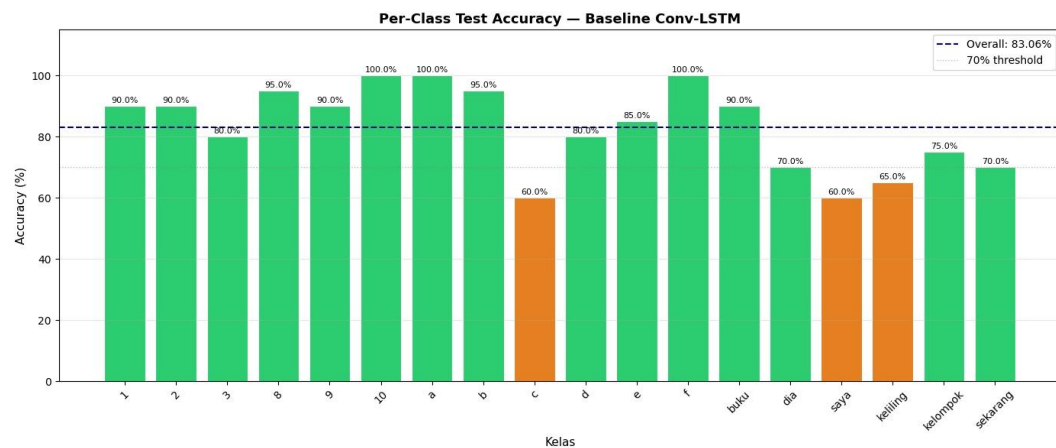
Kelas	Precision	Recall	F1-score	Suport
1	0.947	0.900	0.923	20
2	0.947	0.900	0.923	20
3	0.800	0.800	0.800	20
8	0.905	0.950	0.927	20
9	0.900	0.900	0.900	20
10	0.909	1.000	0.950	20
a	1.000	1.000	1.000	20
b	0.950	0.950	0.950	20
c	0.800	0.600	0.686	20
d	0.615	0.800	0.696	20
e	0.944	0.850	0.895	20
f	0.833	1.000	0.909	20
buku	0.818	0.900	0.857	20
dia	0.778	0.700	0.737	20
saya	0.571	0.600	0.585	20
keliling	0.684	0.650	0.667	20
kelompok	0.833	0.750	0.789	20
sekarang	0.778	0.700	0.737	20

Distribusi prediksi model *baseline* pada data uji secara lengkap ditampilkan dalam bentuk *confusion matrix* pada Gambar 4.7. *Confusion matrix* ini memperlihatkan pola kesalahan klasifikasi antar kelas secara detail.



Gambar 4. 7 *Confusion Matrix Baseline*

Gambar 4. 8 menampilkan akurasi per kelas model baseline dalam bentuk diagram batang. Terlihat bahwa sebagian besar kelas mencapai akurasi di atas 80%, dengan beberapa kelas seperti ‘c’, ‘saya’, dan ‘keliling’ menunjukkan akurasi yang lebih rendah.



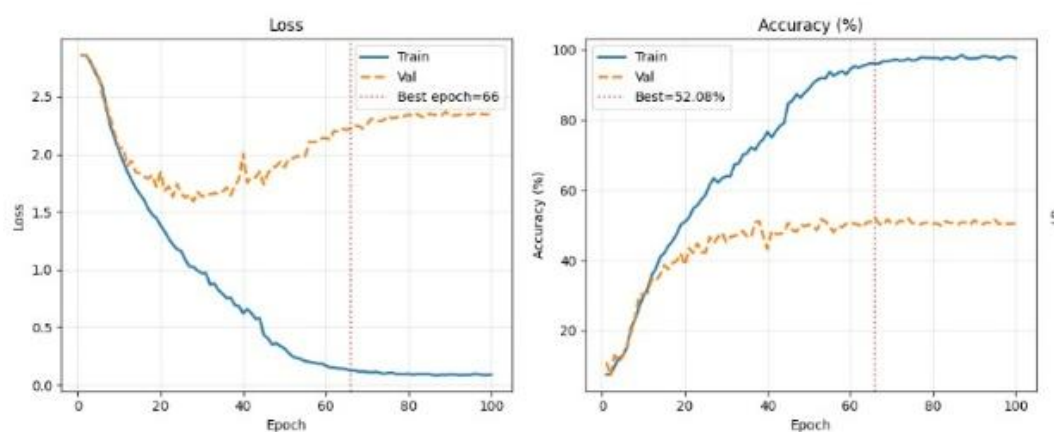
Gambar 4. 8 *Bar chart* akurasi per kelas *baseline*

4.5.2 Optical Flow Conv-LSTM Single-Stream

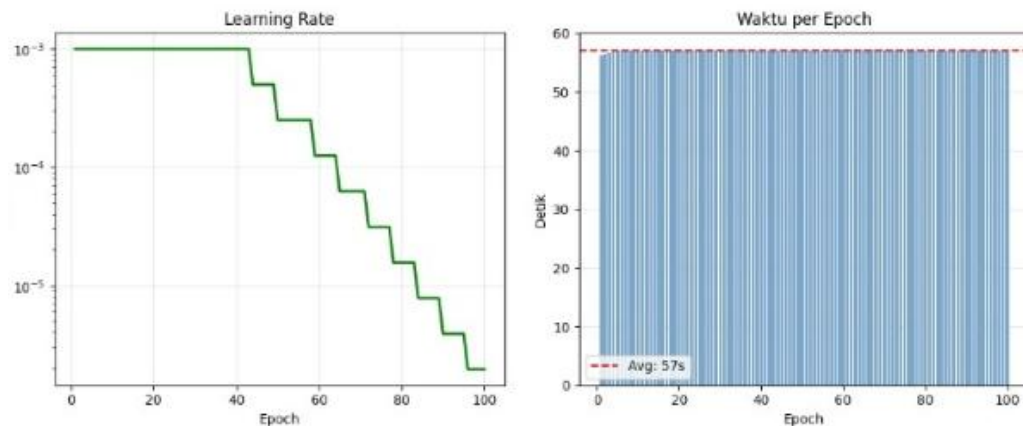
Model *single-stream optical flow* menerima input berupa sekuens 29 *frame optical flow* dengan *shape* [B, 3, 29, 96, 96], di mana 3 *channel* merepresentasikan komponen *u*, *v*, dan *magnitude* dari hasil *Lucas-Kanade*. Data *optical flow* yang dimasukkan ke model sudah dalam keadaan ternormalisasi (*clip p2-p98 + z-score*), sehingga nilai input tidak berada di rentang [0,1] seperti RGB, melainkan dalam skala *z-score*.

Pelatihan model *single-stream* selesai dalam waktu 95,1 menit untuk 100 *epoch*, dengan rata-rata sekitar 57 detik per *epoch*. Proses konvergensi model ini jauh lebih lambat dan tidak stabil dibandingkan model *baseline*. Pada *epoch* 1, *validation accuracy* hanya mencapai 10.97%, dan setelah 100 *epoch* penuh, *validation accuracy* terbaik yang dicapai hanya sebesar 52.08% pada *epoch* 66. Perbedaan yang cukup besar antara *training accuracy* (97.66% di *epoch* 100) dan *validation accuracy* (50.56% di *epoch* 100) mengindikasikan terjadinya *overfitting* yang signifikan.

Dinamika pelatihan model *single-stream* Lucas-Kanade P17 dapat dilihat pada Gambar 4. 9. Kurva *training* dan *validation* menunjukkan pola yang sangat berbeda dari model *baseline*, mencerminkan adanya *overfitting* yang signifikan.



Gambar 4. 9 Kurva *training* dan *validation loss/accuracy* model *Lucas-Kanade single-stream*



Gambar 4. 10 Penurunan Learning Rate dan Rata-rata Waktu Training per Epoch Model *Lucas-Kanade single-stream*

Evaluasi pada data test menghasilkan *accuracy* sebesar 51.25% dengan detail metrik yang ditampilkan pada Tabel 4.6.

Tabel 4. 6 Hasil Evaluasi Test Set Lucas-Kanade Single-Stream

Metrik	Nilai
<i>Test Loss</i>	2.3354
<i>Test Accuracy</i>	51.25%
<i>Precision</i>	50.66%
<i>Recall</i>	51.26%
<i>F1-score</i>	50.47%
<i>F1-score</i>	50.47%
<i>Best Epoch</i>	66/100

Hasil per kelas menunjukkan ketimpangan yang cukup besar. Kelas ‘b’ menjadi kelas terbaik dengan *F1-score* 0.780, sementara kelas ‘c’ dan ‘saya’, menjadi kelas terburuk dengan *F1-score* masing-masing hanya 0.071 dan 0.270. Detail per kelas dapat dilihat pada Tabel 4. 7.

Tabel 4. 7 Nilai Akurasi per Kelas *Test Set Lucas-Kanade Single-Stream*

Kelas	<i>Precision</i>	<i>Recall</i>	<i>F1-score</i>	<i>Suport</i>
1	0.565	0.650	0.605	20
2	0.688	0.550	0.611	20

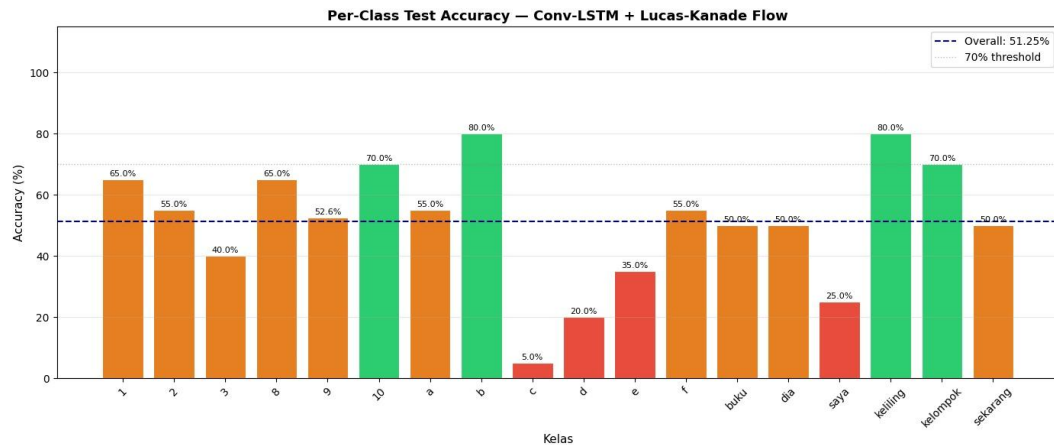
3	0.348	0.400	0.372	20
8	0.722	0.650	0.684	20
9	0.500	0.526	0.513	20
10	0.538	0.700	0.609	20
a	0.550	0.550	0.550	20
b	0.762	0.800	0.780	20
c	0.125	0.050	0.071	20
d	0.167	0.200	0.128	20
e	0.350	0.350	0.350	20
f	0.688	0.550	0.611	20
buku	0.588	0.500	0.541	20
dia	0.625	0.500	0.556	20
saya	0.294	0.250	0.270	20
keliling	0.615	0.800	0.696	20
kelompok	0.538	0.700	0.609	20
sekarang	0.455	0.500	0.476	20

Gambar 4. 11 menampilkan *confusion matrix* model *single-stream* Lucas-Kanade P17. Distribusi kesalahan pada *confusion matrix* ini memperlihatkan kesulitan model dalam membedakan kelas-kelas dengan pola gerak bibir yang serupa.



Gambar 4. 11 *Confusion matrix Lucas-Kanade single-stream*

Gambar 4.12 menampilkan akurasi per kelas model *single-stream* Lucas-Kanade P17 dalam bentuk diagram batang. Sebagian besar kelas menunjukkan akurasi yang rendah, terutama pada label dengan 1 dan 2 suku kata.



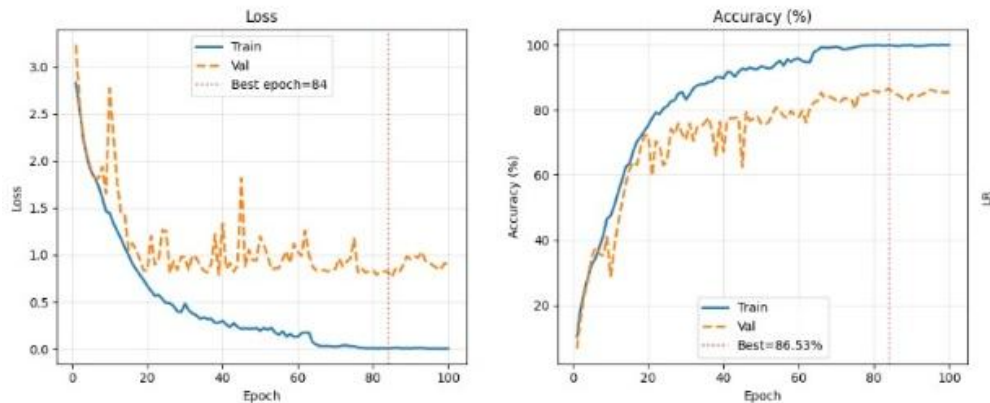
Gambar 4. 12 Bar chart akurasi per kelas LK P17 *single-stream*

4.5.3 Dual-Stream Conv-LSTM

Model *dual-stream* memproses dua input secara paralel: sekuens RGB [B, 3, 30, 96, 96] melalui StreamEncoder pertama, dan sekuens optical flow Lucas-Kanade [B, 3, 29, 96, 96] melalui StreamEncoder kedua. Kedua StreamEncoder memiliki arsitektur identik dengan model *baseline* namun tanpa lapisan *classifier*, sehingga masing-masing menghasilkan vektor fitur berukuran 64 dimensi. Kedua vektor fitur kemudian digabungkan melalui mekanisme AttentionFusion sebelum masuk ke lapisan *fully connected* untuk menghasilkan prediksi kelas.

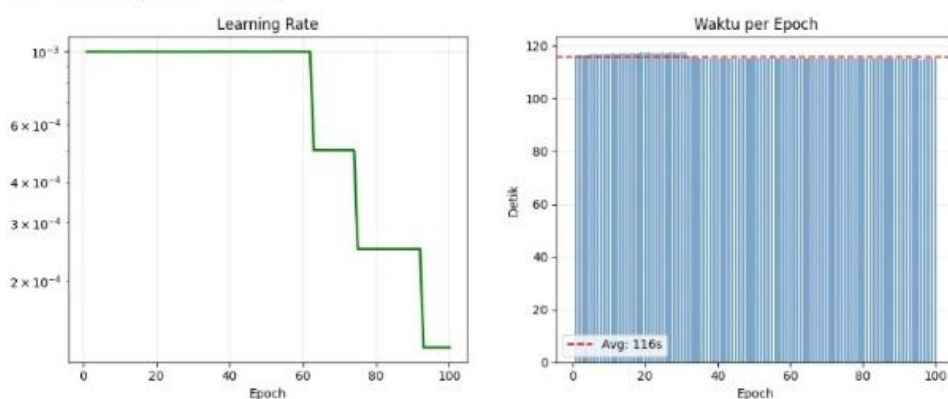
Pelatihan model *dual-stream* membutuhkan waktu paling lama, yaitu 192.9 menit untuk 100 *epoch*, dengan rata-rata sekitar 115–117 detik per *epoch*. Waktu per *epoch* yang hampir dua kali lipat dibandingkan model *single-stream* ini wajar mengingat model *dual-stream* memproses dua *stream* data secara paralel sekaligus. Pada *epoch* 1, *validation accuracy* mencapai 6.67%, dan secara konsisten meningkat hingga mencapai *validation accuracy* terbaik sebesar 86.53% pada *epoch* 84. Nilai *patience scheduler* yang lebih besar (*patience*=8 vs *patience*=5 pada model *baseline*) terbukti sesuai untuk mengakomodasi konvergensi *dual-stream* yang lebih lambat namun lebih stabil.

Gambar 4. 13 menunjukkan proses konvergensi yang lebih lambat namun stabil, konsisten dengan kompleksitas arsitektur yang lebih tinggi pada pelatihan model *dual-stream*.



Gambar 4. 13 Kurva *training* dan *validation loss/accuracy* model *dual-stream*

Gambar 4. 14 menampilkan proses penurunan *learning rate* pada proses *training* yang diawal proses pembelajaran dapat belajar dengan *learning rate* yang sudah ditentukan, kemudian menurun setelah dalam rentang tidak ada peningkatan acc validation. Selain itu rata-rata waktu yang dibutuhkan untuk model belajar dalam satu *epoch* juga di tampilkan di sini, terlihat konsistensi proses pembelajaran yang dibutuhkan.



Gambar 4. 14 Penurunan Learning Rate dan Rata-rata Waktu Training per Epoch Model *Lucas-Kanade dual-stream*

Evaluasi pada data *test* menghasilkan *accuracy* sebesar 86.07% dengan detail metrik yang ditampilkan pada Tabel 4. 8.

Tabel 4. 8 Hasil Evaluasi *Test Set Lucas-Kanade Dual-Stream*

Metrik	Nilai
<i>Test Loss</i>	0.8598
<i>Test Accuracy</i>	86.07%
<i>Precision</i>	86.70%
<i>Recall</i>	86.04%
<i>F1-score</i>	85.93%
<i>F1-score</i>	85.94%
<i>Best Epoch</i>	84/100

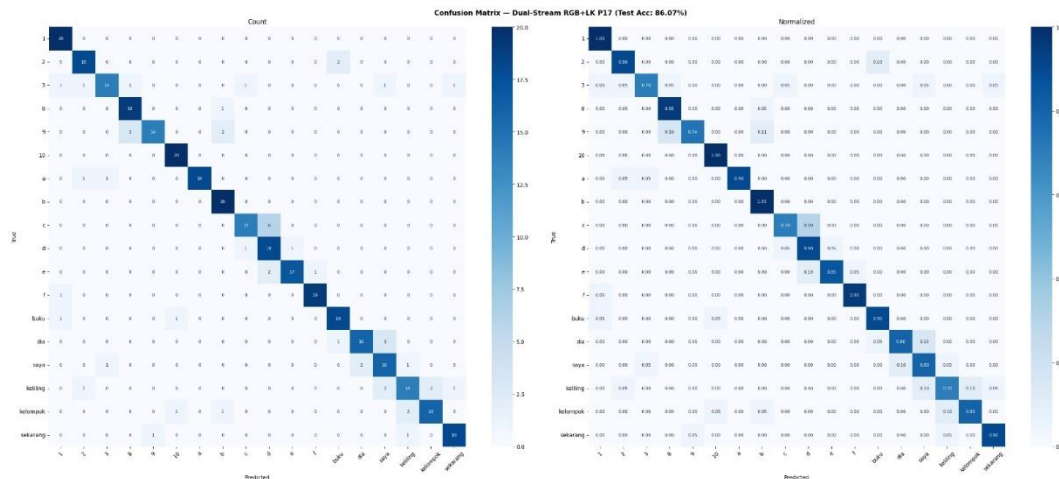
Performa per kelas model *dual-stream* menunjukkan peningkatan yang merata dibandingkan dua model sebelumnya. Kelas '10' dan 'b' mencapai *recall* sempurna 1.000, sementara kelas 'c' dan 'keliling' yang sebelumnya menjadi titik lemah pada model *baseline* berhasil meningkat. Kelas 'saya' masih menjadi yang paling rendah dengan *F1-score* 0.762, namun ini sudah jauh lebih baik dibandingkan 0.585 pada *baseline*. Detail per kelas dapat dilihat pada Tabel 4. 9.

Tabel 4. 9 Nilai Akurasi per Kelas *Test Set Lucas-Kanade Dual-Stream*

Kelas	<i>Precision</i>	<i>Recall</i>	<i>F1-score</i>	<i>Suport</i>
1	0.870	1.000	0.930	20
2	0.857	0.900	0.878	20
3	0.875	0.700	0.778	20
8	0.826	0.950	0.884	20
9	0.933	0.737	0.824	20
10	0.909	1.000	0.952	20
a	1.000	0.900	0.947	20
b	0.833	1.000	0.909	20
c	0.875	0.700	0.778	20
d	0.692	0.900	0.783	20
e	0.944	0.850	0.895	20
f	0.950	0.950	0.950	20
buku	0.857	0.900	0.878	20
dia	0.889	0.800	0.842	20
saya	0.727	0.800	0.762	20
keliling	0.778	0.700	0.737	20

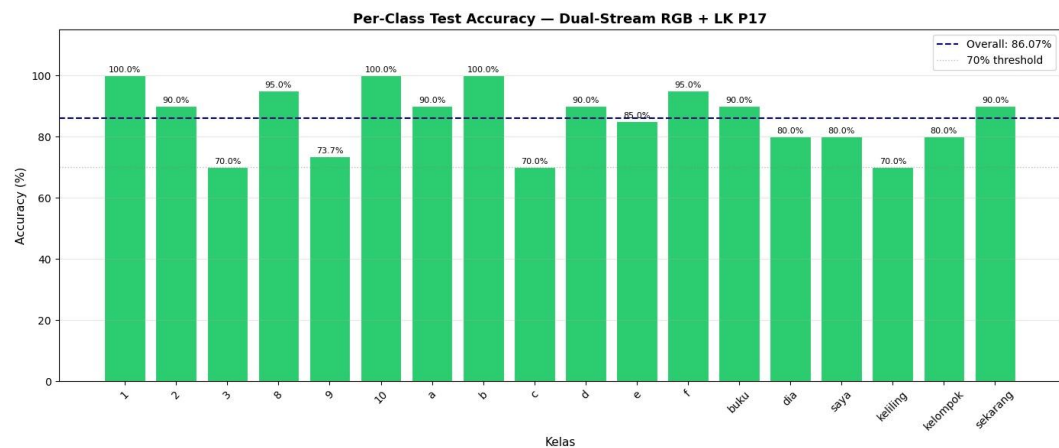
kelompok	0.889	0.800	0.842	20
sekarang	0.900	0.900	0.900	20

Gambar 4. 15 menampilkan *confusion matrix* model *dual-stream* LK P17 (M01). Terlihat peningkatan yang jelas dibandingkan model *single-stream*, terutama pada kelas-kelas yang sebelumnya sulit diklasifikasikan.



Gambar 4. 15 *Confusion matrix dual-stream*

Gambar 4.16 menampilkan perbandingan akurasi per kelas model *dual-stream* dalam bentuk diagram batang. Model ini secara merata memberikan akurasi yang lebih tinggi dibandingkan dua varian sebelumnya pada hampir seluruh kelas.



Gambar 4. 16 *Bar chart* akurasi per kelas *dual-stream*

4.6 Hasil Skenario 4: OFAT Arsitektur Model Conv-LSTM

Pengujian enam variasi arsitektur dilakukan pada dataset Gabungan 18 kelas.

Tabel 4. 10 menyajikan hasil lengkap seluruh variasi.

Tabel 4. 10 Hasil Lengkap OFAT Arsitektur Model Conv-LSTM

ID	<i>LSTM Hidden</i>	<i>Dropout</i>	<i>Val Acc</i>	<i>Test Acc</i>	<i>F1 (w)</i>	<i>Best Epoch</i>	Waktu
M01	128, 64	0.5	84.58%	83.06%	82.96%	48	98.1 mnt
M02	128, 128	0.5	85.97%	85.00%	84.83%	59	116.6 mnt
M03	256, 128	0.5	79.44%	80.83%	80.76%	98	181.2 mnt
M04	128,128,64	0.5	84.03%	85.00%	85.00%	99	130.5 mnt
M05	128, 64	0.5	86.25%	81.67%	81.62%	61	161.1 mnt
M06	128, 64	0.3	85.97%	84.72%	84.75%	93	98.0 mnt

M02 dan M04 sama-sama mencapai *test accuracy* 85.00%, namun M02 dipilih sebagai arsitektur terbaik karena konvergensi lebih cepat (*epoch* 59 vs 99) dan waktu training yang lebih efisien (116.6 vs 130.5 menit). M03 menghasilkan performa di bawah *baseline* karena *overfitting* dengan kapasitas model yang terlalu besar. M05 menunjukkan *val accuracy* tertinggi (86. 25%) tetapi *test accuracy* justru turun (81.67%) akibat *overfitting* spasial. M06 memberikan peningkatan moderat dengan *dropout* yang lebih kecil.

4.7 Hasil Skenario 5: Arsitektur Pilihan Terbaik M02 pada LK P17 dan Dual-Stream

4.7.1 LK P17 *Single-Stream* dengan Arsitektur Model Conv-LSTM M02

Penerapan arsitektur M02 pada model *single-stream optical flow* menghasilkan *test accuracy* 53.48%, hanya naik +2.23% dibanding P17-M01 (51.25%). Ini mengkonfirmasi bahwa *bottleneck* pada model *single-stream* bukan di kapasitas arsitektur, melainkan di keterbatasan representasi fitur *optical flow* itu

sendiri. Untuk konteks perbandingan, model P17-M04 (3 layer LSTM) justru menghasilkan *test accuracy* yang lebih rendah yaitu 43.73% akibat *overfitting* yang lebih parah.

Tabel 4. 11 Perbandingan Variasi Arsitektur pada LK P17 *Single-Stream*

Konfigurasi	<i>LSTM Hidden</i>	<i>Val Acc</i>	<i>Test Acc</i>	<i>F1 (w)</i>	<i>Best Epoch</i>
P17-M01	128, 64	52.08%	51.25%	50.47%	66
P17-M01	128, 128	51.53%	53.48%	54.03%	97
P17-M01	128,128,64	45.42%	43.73%	44.27%	89

4.7.2 *Dual-Stream* dengan Arsitektur M02

Penerapan arsitektur M02 (LSTM hidden 128, 128) pada kedua stream model *dual-stream* menghasilkan konfigurasi terbaik dalam keseluruhan penelitian. Model ini selesai dilatih dalam 225.4 menit dan mencapai *best val accuracy* 87.22% pada epoch 83. Hasil evaluasi test ditampilkan pada Tabel 4.12.

Tabel 4. 12 Hasil Evaluasi *Test Set Dual-Stream* RGB + LK P17 (M02)

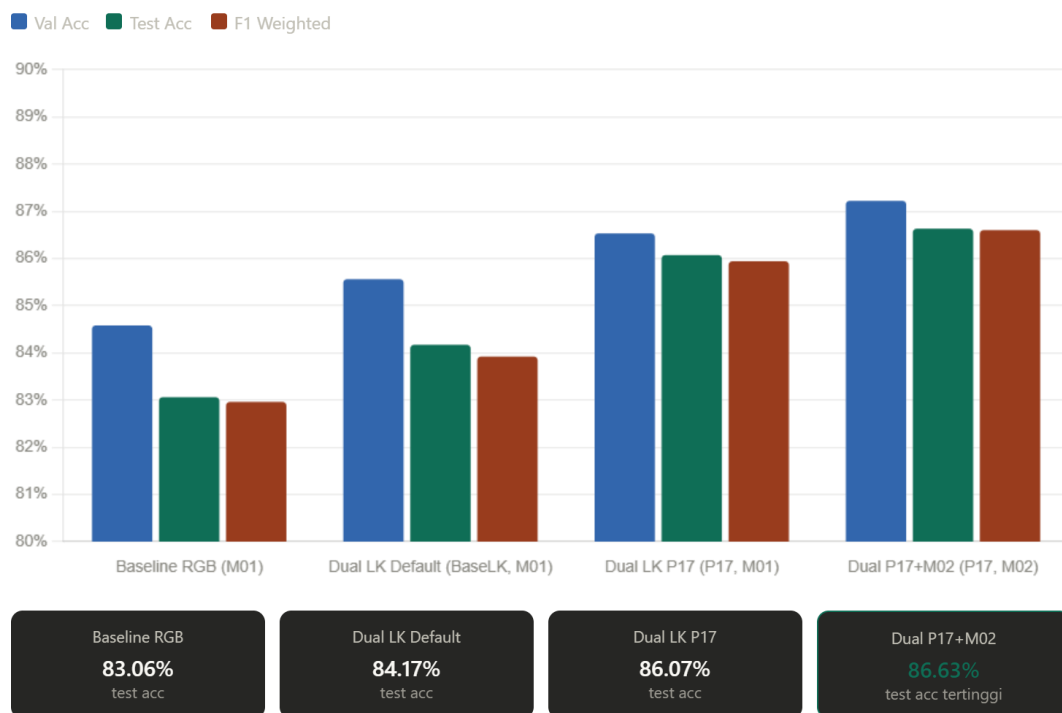
Metrik	Nilai
<i>Test Loss</i>	0.8598
<i>Test Accuracy</i>	86. 63%
<i>Precision (macro)</i>	86.85%
<i>Recall (macro)</i>	86.63%
<i>F1-score (macro)</i>	86.61%
<i>F1-score (weighted)</i>	86.60%
<i>Best Val Acc</i>	87.22%
<i>Best Epoch</i>	83/100
<i>Waktu Training</i>	225.4 menit

4.8 Perbandingan Model *Dual-Stream*

Tabel 4. 13 dan Gambar 4. 17 menampilkan perbandingan langsung ketiga varian *dual-stream* yang diuji dalam penelitian ini, menunjukkan dampak masing-masing perbaikan secara terpisah. Percobaan terbaik didapat dari penggunaan Dual dengan parameter LK P17 dan M02.

Tabel 4. 13 Perbandingan Hasil Evaluasi Model Dual-Stream

Model	LK Params	Arsitektur	Val Acc	Test Acc	F1 (w)	Best Epoch	Waktu
Dual LK Default	BaseLK	M01 (128,64)	85.56 %	84.17 %	83.92 %	97	192.1 mnt
Dual LK P17	P17	M01 (128,64)	86.53 %	86.07 %	85.94 %	84	192.9 mnt
Dual P17+M02	P17	M02 (128,128)	87.22 %	86.63 %	86.60 %	83	225.4 mnt
Baseline RGB	—	M01	84.58 %	83.06 %	82.96 %	48	98.1 mnt

Gambar 4. 17 Grafik Batang Perbandingan Model *Dual-Stream*

Dari perbandingan ini terlihat dua faktor perbaikan yang masing-masing berkontribusi secara terpisah. Penggantian parameter LK Default ke P17 menghasilkan peningkatan +1.90% (84.17% menjadi 86.07%), sedangkan

penggantian arsitektur M01 ke M02 memberikan tambahan +0.56% (86.07% menjadi 86.63%). Total perbaikan kumulatif dari *baseline* adalah +3.57%.

4.9 Rekapitulasi Seluruh Eksperimen

Tabel 4.14 menampilkan rekapitulasi hasil seluruh eksperimen yang dilakukan dalam penelitian ini secara berurutan.

Tabel 4. 14 Rekapitulasi Seluruh Hasil Eksperimen

No	Konfigurasi	Input	Val Acc	Test Acc	F1(w)	Best Epoch
1	<i>Baseline RGB (M01)</i>	RGB	84.58%	83.06%	82.96%	48
2	<i>LK P17 Single (M01)</i>	Flow	52.08%	51.25%	50.47%	66
3	<i>Dual-Stream LK Default</i>	RGB+Flow	85.56%	84.17%	83.92%	97
4	<i>Dual-Stream LK P17 (M01)</i>	RGB+Flow	86.53%	86.07%	85.94%	84
5	<i>RGB M02</i>	RGB	85.97%	85.00%	84.83%	59
6	<i>P17-M02 Single</i>	Flow	51.53%	53.48%	54.03%	97
7	<i>P17-M04 Single</i>	Flow	45.42%	43.73%	44.27%	89
8	<i>Dual-Stream P17+M02</i>	RGB+Flow	87.22%	86.63%	86.60%	83

4.10 Analisis Berdasarkan Jumlah Suku Kata

Untuk melihat konsistensi performa model terhadap kompleksitas gerak bibir, hasil *test accuracy* dikelompokkan berdasarkan jumlah suku kata dari masing-masing label. Tabel 4. 15 menampilkan rata-rata *recall* per kelompok suku kata untuk tiga model utama.

Tabel 4. 15 Rata-rata Recall Per Kelompok Suku Kata

Suku Kata	Label	<i>Baseline (M01)</i>	<i>LK P17 Single</i>	<i>Dual P17+M02</i>
1 suku kata	a, b, c, d, e, f	86.67%	41.67%	89.17%
2 suku kata	1,2,3, buku, dia, saya	80.00%	47.50%	87.50%
3 suku kata	8,9,10, keliling, kelompok, sekarang	82.50%	64.60%	86.28%
Rata-rata keseluruhan	—	83.06%	51.25%	86.63%

Beberapa temuan penting dari analisis ini: (1) Label 1 suku kata paling sulit bagi *LK single-stream* (41.67%) karena gerak bibir yang sangat singkat menghasilkan perbedaan antar *frame* yang minimal, (2) Label 3 suku kata justru paling baik ditangani *LK single-stream* (64.60%) karena gerakan yang lebih panjang dan distinktif, (3) *Dual-stream* P17+M02 unggul merata di semua kelompok, dengan peningkatan terbesar pada 2 suku kata (+7.50% dari *baseline*) dan 1 suku kata (+2.50%).

4.11 Pembahasan

4.11.1 Analisis Performa Model *Baseline*

Model *baseline* Conv-LSTM berhasil mencapai test accuracy 83.06% pada dataset Gabungan 18 kelas. Arsitektur Conv-LSTM terbukti mampu mengekstraksi fitur spasial dan temporal dari sekuens *lip crop* RGB secara efektif. Konvergensi di *epoch* 48 dari 100 *epoch* mengindikasikan bahwa model tidak memerlukan waktu pelatihan yang sangat panjang untuk mencapai performa optimal pada data RGB, konsisten dengan karakteristik input RGB yang kaya informasi visual.

4.11.2 Analisis Performa Model dengan *Lucas-Kanade* LK P17 *Single-Stream*

Model *single-stream* menghasilkan *test accuracy* hanya 51.25%, jauh di bawah *baseline*. Terdapat tiga penjelasan yang mendasari hasil ini. Pertama, *backbone* CNN yang dirancang untuk fitur visual RGB kurang optimal untuk peta

optical flow yang memiliki distribusi nilai sangat berbeda. Kedua, pola *optical flow* area bibir memiliki kemiripan tinggi antar beberapa kelas, terutama label dengan bentuk mulut serupa seperti c, d, dan e. Ketiga, gap besar antara *training accuracy* (97.66%) dan *validation accuracy* (50.56%) mengindikasikan *overfitting* yang signifikan. Percobaan meningkatkan kapasitas arsitektur (M02, M04) tidak memberikan perbaikan berarti, mengkonfirmasi bahwa *bottleneck* ada pada kualitas representasi *optical flow* itu sendiri, bukan pada kapasitas model.

4.11.3 Analisis Performa Model Dual-Stream

Model *dual-stream* P17+M02 berhasil mencapai *test accuracy* tertinggi 86.63%, melampaui *baseline* sebesar +3.57%. Mekanisme AttentionFusion berperan krusial dengan menggunakan *gate sigmoid* yang belajar secara adaptif menentukan bobot masing-masing *stream*. Konvergensi yang lebih lambat (*epoch* 83) dibanding *baseline* (*epoch* 48) wajar mengingat model harus mengkoordinasikan dua jalur input sekaligus. Penggunaan *patience*=8 dan *gradient clipping* terbukti menjaga stabilitas pelatihan.

Peningkatan paling signifikan pada kelas saya (+20.0% dari *baseline* ke *dual-P17*), sekarang (+20.0%), dan l (+10.0%) mengindikasikan bahwa kombinasi informasi visual dan gerak paling efektif untuk label-label yang memiliki pola temporal tertentu. Beberapa kelas mengalami penurunan kecil (seperti 9 dan 3) yang dapat disebabkan oleh *noise* dari *stream optical flow* pada kelas tersebut yang mengganggu representasi RGB yang sudah baik.

4.11.4 Kelas dengan Performa Rendah

Terlepas dari model yang digunakan, terdapat beberapa kelas yang secara konsisten menunjukkan performa di bawah rata-rata, yaitu kelas 'saya', 'keliling', 'c', dan 'd'. Beberapa faktor yang kemungkinan menjadi penyebabnya adalah:

Pertama, kemiripan gerak bibir antar kelas. Kelas 'saya' dan 'dia' misalnya, memiliki pola gerak bibir yang relatif serupa karena keduanya merupakan kata ganti yang diucapkan dengan gerakan singkat. Begitu pula huruf 'c' dan 'e' yang secara visual memiliki bentuk mulut yang mirip.

Kedua, variasi antar subjek yang tinggi. Data diambil dari 20 orang berbeda (10 guru dan 10 siswa) dengan latar belakang dan kebiasaan pengucapan isyarat yang bervariasi. Untuk kelas-kelas yang memiliki pola gerak bibir kurang distinktif, variasi antar subjek ini bisa menjadi sumber kebingungan bagi model.

4.11.5 Temuan Utama Penelitian

Berdasarkan seluruh eksperimen yang dilakukan, terdapat beberapa temuan utama. Pertama, *Lucas-Kanade* lebih unggul dari *Farneback* untuk representasi *optical flow* bibir dengan selisih +4.54% *test accuracy*. Kedua, optimasi parameter LK (P17) memberikan dampak lebih besar (+1.90%) dibanding optimasi arsitektur model (+0.56%) terhadap performa *dual-stream*, menunjukkan bahwa kualitas representasi input lebih kritis dari kapasitas model. Ketiga, model *dual-stream* dengan AttentionFusion secara konsisten mengalahkan model *single-stream* manapun, mengkonfirmasi bahwa kombinasi informasi visual RGB dan informasi gerak *optical flow* memberikan representasi yang lebih komprehensif untuk pengenalan gerak bibir SIBI. Keempat, konfigurasi *Dual-Stream* P17+M02 dengan *test accuracy* 86,63% merupakan konfigurasi terbaik dari seluruh eksperimen yang dilakukan dalam penelitian ini.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Penelitian ini bertujuan untuk mengembangkan sistem pembacaan gerak bibir pada video SIBI menggunakan metode *Optical Flow Conv-LSTM* dan membandingkan efektivitasnya dengan model Conv-LSTM tanpa *optical flow* sebagai *baseline*. Berdasarkan serangkaian eksperimen yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut.

Pertama, model Conv-LSTM *baseline* yang menggunakan input RGB secara langsung mampu mencapai *test accuracy* sebesar 83.06% dengan *F1-score* 82.96% pada dataset gabungan 18 kelas yang terdiri dari label angka, alfabet, dan kata dalam bahasa SIBI. Hasil ini menunjukkan bahwa arsitektur Conv-LSTM telah mampu mengekstraksi fitur spasial dan temporal dari sekuens *lip crop* secara efektif sebagai titik acuan perbandingan.

Kedua, penambahan *optical flow Lucas-Kanade* pada varian *single-stream* tidak memberikan peningkatan performa, bahkan menghasilkan *test accuracy* yang jauh lebih rendah yaitu 51.25% (*F1-score* 50.47%). Hal ini disebabkan oleh keterbatasan representasi gerak bibir secara mandiri tanpa konteks visual RGB, di mana *backbone* CNN yang dirancang untuk fitur spasial kurang optimal untuk mempelajari pola dari peta *optical flow* semata. Percobaan peningkatan kapasitas arsitektur (M02 dan M04) juga tidak mampu mengatasi keterbatasan fundamental ini.

Ketiga, varian *dual-stream* yang memproses RGB dan *optical flow* secara paralel melalui mekanisme AttentionFusion berhasil melampaui semua konfigurasi lainnya. Konfigurasi terbaik yaitu *Dual-Stream* LK P17 dengan arsitektur M02 (LSTM hidden 128, 128) mencapai *test accuracy* sebesar 86.63% dan *F1-score* sebesar 86.60%, melampaui model *baseline* Conv-LSTM sebesar +3.57%.

Keempat, dari seluruh skenario pengujian ditemukan bahwa optimasi parameter Lucas-Kanade (*konfigurasi P17: maxCorners=300, qualityLevel=0.005, blockSize=6*) memberikan dampak peningkatan yang lebih signifikan (+1.90%) dibandingkan optimasi arsitektur model (+0.56%), menunjukkan bahwa kualitas representasi input *optical flow* lebih kritis dari kapasitas model dalam konteks penelitian ini.

Kelima, analisis berdasarkan jumlah suku kata menunjukkan bahwa model *dual-stream* P17+M02 unggul secara merata pada semua kelompok, dengan rata-rata recall 89.17% untuk label 1 suku kata, 87.50% untuk label 2 suku kata, dan 86.28% untuk label 3 suku kata. Label yang secara konsisten menunjukkan performa paling rendah di semua model adalah kelas "saya", "keliling", "c", dan "d", yang disebabkan oleh kemiripan pola gerak bibir antar kelas dan tingginya variasi antar subjek.

Dengan demikian, menjawab rumusan masalah penelitian ini yaitu model *Optical Flow Conv-LSTM* dalam konfigurasi *dual-stream* terbukti lebih unggul dibandingkan model *Conv-LSTM* tanpa *optical flow*, dengan peningkatan *test accuracy* sebesar 3.57% (dari 83.06% menjadi 86.63%). Namun penting dicatat bahwa keunggulan ini hanya tercapai melalui pendekatan *dual-stream* yang mempertahankan informasi RGB sekaligus menambahkan informasi gerak, bukan melalui penggantian penuh ke *optical flow*.

5. 2 Saran

Untuk pengembangan lebih lanjut, pengujian dengan subjek baru di luar dataset perlu dilakukan agar kemampuan generalisasi model dapat diukur lebih objektif. Selain itu, dataset juga perlu diperluas baik dari segi jumlah label maupun jumlah subjek agar model dapat belajar dari variasi pengguna yang lebih beragam. Terakhir, untuk implementasi nyata bagi penyandang tunarungu, disarankan menggunakan *backbone* yang lebih ringan seperti MobileNet agar sistem dapat berjalan secara *real-time* pada perangkat *mobile*.

DAFTAR PUSTAKA

- Alfarano, A., Maiano, L., Papa, L., & Amerini, I. (2024). Estimating optical flow: A comprehensive review of the state of the art. *Computer Vision and Image Understanding*, 249. <https://doi.org/10.1016/j.cviu.2024.104160>
- Al-Qudah, S., & Yang, M. (2023). Large Displacement Detection Using Improved Lucas–Kanade Optical Flow. *Sensors*, 23(6). <https://doi.org/10.3390/s23063152>
- Amri, I., Jurnal, K. :, & Saintek, M. (2024). *Implementasi Algoritma Convolutional Neural Network untuk Menerjemahkan Bahasa Isyarat*. 2(9), 70–87. <https://ejournal.warunayama.org/kohesi>
- Aripin, & Setiawan, A. (2024). Indonesian Lip-Reading Detection and Recognition Based on Lip Shape Using Face Mesh and Long-Term Recurrent Convolutional Network. *Applied Computational Intelligence and Soft Computing*, 2024, 1–13. <https://doi.org/10.1155/2024/6479124>
- Atila, Ü., & Sabaz, F. (2022). Turkish lip-reading using Bi-LSTM and deep learning models. *Engineering Science and Technology, an International Journal*, 35. <https://doi.org/10.1016/j.jestch.2022.101206>
- Brauwers, G., & Frasincar, F. (2022). *A General Survey on Attention Mechanisms in Deep Learning*. <https://doi.org/10.1109/TKDE.2021.3126456>
- Caraka, R. E., Supardi, K., Kurniawan, R., Kim, Y., Gio, P. U., Yuniarto, B., Mubarok, F. Z., & Pardamean, B. (2024). Empowering deaf communication: a novel LSTM model for recognizing Indonesian sign language. *Universal Access in the Information Society*. <https://doi.org/10.1007/s10209-024-01095-1>

- Chen, S., Wang, Y., & Lei, Z. (2021). Two Stream Convolution Fusion Network based on Attention Mechanism. *Journal of Physics: Conference Series*, 1920(1). <https://doi.org/10.1088/1742-6596/1920/1/012070>
- Chen, X., Du, J., & Zhang, H. (2020). Lipreading with DenseNet and resBi-LSTM. *Signal, Image and Video Processing*, 14(5), 981–989. <https://doi.org/10.1007/s11760-019-01630-1>
- Dweik, W., Altorman, S., & Ashour, S. (2022). Read my lips: Artificial intelligence word-level arabic lipreading system. *Egyptian Informatics Journal*, 23(4), 1–12. <https://doi.org/10.1016/j.eij.2022.06.001>
- Farnebäck, G. (2003). Two-Frame Motion Estimation Based on Polynomial Expansion. In J. Bigun & T. Gustavsson (Eds.), *Image Analysis* (pp. 363–370). Springer Berlin Heidelberg.
- Fleet, D., & Weiss, Y. (2006). Optical Flow Estimation. In N. Paragios, Y. Chen, & O. Faugeras (Eds.), *Handbook of Mathematical Models in Computer Vision* (pp. 237–257). Springer US. https://doi.org/10.1007/0-387-28831-7_15
- Garg, A., Noyola, J., & Bagadia, S. (n.d.). *Lip reading using CNN and LSTM*.
- Hafiz, A. M., Parah, S. A., & Bhat, R. U. A. (2021). *Attention mechanisms and deep learning for machine vision: A survey of the state of the art*. <http://arxiv.org/abs/2106.07550>
- Hassanin, M., Anwar, S., Radwan, I., Khan, F. S., & Mian, A. (2024). *Visual Attention Methods in Deep Learning: An In-Depth Survey*. <http://arxiv.org/abs/2204.07756>
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9, 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Horn, B. K. P., & Schunck, B. G. (1981). Determining optical flow. *Artificial Intelligence*, 17(1), 185–203. [https://doi.org/https://doi.org/10.1016/0004-3702\(81\)90024-2](https://doi.org/https://doi.org/10.1016/0004-3702(81)90024-2)

- Huang, H., Song, C., Ting, J., Tian, T., Hong, C., Di, Z., & Gao, D. (2021). A Novel Machine Lip Reading Model. *Procedia Computer Science*, 199, 1432–1437. <https://doi.org/10.1016/j.procs.2022.01.181>
- Jin Ting, C. S. H. H. T. T., Institute of Electrical and Electronics Engineers. Indonesia Section, IEEE Signal Processing Society. Indonesia Chapter, Universitas Telkom, Multimedia University, & Universitas Gadjah Mada. (2021). *A Comprehensive Dataset for Machine-Learning-based Lip-Reading Algorithm*.
- Kurniawan, R. A. T., & Kaswidjanti, W. (2024). *SIBI Dynamic Gesture Translation Using MediaPipe and Long Short-Term Memory in Real-Time* (pp. 49–60). https://doi.org/10.2991/978-94-6463-366-5_6
- Lucas, B., & Kanade, T. (1981). An Iterative Image Registration Technique with an Application to Stereo Vision (IJCAI). *[No Source Information Available]*, 81.
- Ma, P., Wang, Y., Shen, J., Petridis, S., & Pantic, M. (2021). *Lip-reading with Densely Connected Temporal Convolutional Networks*.
- Mahboob, K., Nizami, H., Ali, F., & Alvi, F. (2021). Sentences Prediction Based on Automatic Lip-Reading Detection with Deep Learning Convolutional Neural Networks Using Video-Based Features. *Communications in Computer and Information Science*, 1489 CCIS, 42–53. https://doi.org/10.1007/978-981-16-7334-4_4
- Manalu, H. V., & Rifai, A. P. (2024). Detection of human emotions through facial expressions using hybrid convolutional neural network-recurrent neural network algorithm. *Intelligent Systems with Applications*, 21. <https://doi.org/10.1016/j.iswa.2024.200339>
- Mesbah, A., Berrahou, A., Hammouchi, H., Berbia, H., Qjidaa, H., & Daoudi, M. (2019). Lip reading with Hahn Convolutional Neural Networks. *Image and*

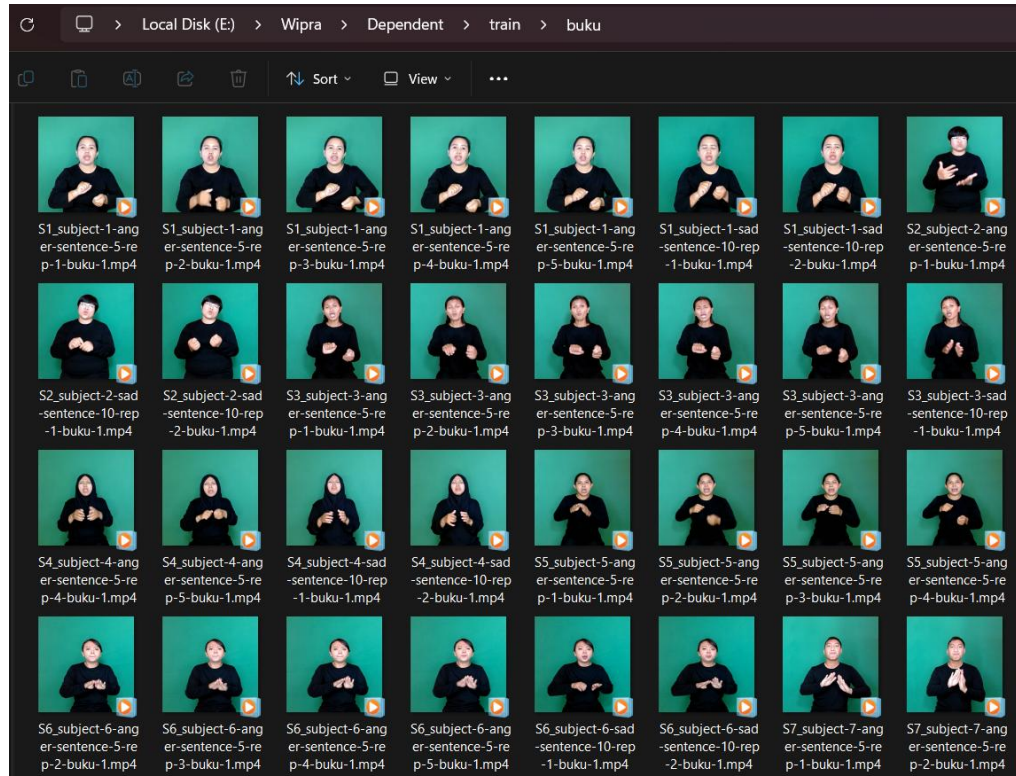
- Vision Computing*, 88, 76–83.
<https://doi.org/https://doi.org/10.1016/j.imavis.2019.04.010>
- Murthy, N. S. (2020). Lip-Reading Techniques: A Review. *INTERNATIONAL JOURNAL OF SCIENTIFIC & TECHNOLOGY RESEARCH*, 9, 2.
www.ijstr.org
- Nayar Shree. (2021, May 3). *Application of Optical Flow | Optical Flow* [Video recording]. First Principles of Computer Vision.
- OZCAN, T., & BASTURK, A. (2019). Lip Reading Using Convolutional Neural Networks with and without Pre-Trained Models. *Balkan Journal of Electrical and Computer Engineering*, 7(2), 195–201.
<https://doi.org/10.17694/bajece.479891>
- Prajwal, K. R., Afouras, T., & Zisserman, A. (2021). *Sub-word Level Lip Reading With Visual Attention*. <http://arxiv.org/abs/2110.07603>
- Rakun, E., Andriani, M., Wiprayoga, I. W., Danniswara, K., & Tjandra, A. (2013). Combining depth image and skeleton data from Kinect for recognizing words in the sign system for Indonesian language (SIBI [Sistem Isyarat Bahasa Indonesia]). *2013 International Conference on Advanced Computer Science and Information Systems, ICACSYS 2013*, 387–392.
<https://doi.org/10.1109/ICACSYS.2013.6761606>
- Samto, Hakim Dimiyati, Bunawan Lani, & Bintoro Totok. (2020). *Kamus Sistem Isyarat Bahasa Indonesia*.
- Shi, C., Zhang, Z., Zhang, W., Zhang, C., & Xu, Q. (2022). Learning Multiscale Temporal-Spatial-Spectral Features via a Multipath Convolutional LSTM Neural Network for Change Detection with Hyperspectral Images. *IEEE Transactions on Geoscience and Remote Sensing*, 60.
<https://doi.org/10.1109/TGRS.2022.3176642>

- Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-K., Woo, W.-C., & Kong Observatory, H. (n.d.). *Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting*.
- Syulistyo, A. R., Hormansyah, D. S., & Saputra, P. Y. (2020). SIBI (Sistem Isyarat Bahasa Indonesia) translation using Convolutional Neural Network (CNN). *IOP Conference Series: Materials Science and Engineering*, 732(1). <https://doi.org/10.1088/1757-899X/732/1/012082>
- Wang, Z. H., Horng, G. J., Hsu, T. H., Aripriharta, A., & Jong, G. J. (2020). Heart sound signal recovery based on time series signal prediction using a recurrent neural network in the long short-term memory model. *Journal of Supercomputing*, 76(11), 8373–8390. <https://doi.org/10.1007/s11227-019-03096-x>
- Weng, X., & Kitani, K. (2019). *Learning Spatio-Temporal Features with Two-Stream Deep 3D CNNs for Lipreading*. <http://arxiv.org/abs/1905.02540>
- Zahara, L., Musa, P., Wibowo, E. P., Karim, I., & Musa, S. B. (2020, April). The Facial Emotion Recognition (FER-2013) Dataset for Prediction System of Micro-Expressions Face Using the Convolutional Neural Network (CNN) Algorithm based Raspberry Pi. *2020 5th International Conference on Informatics and Computing, ICIC 2020*. <https://doi.org/10.1109/ICIC50835.2020.9288560>
- Zhao, Y., Man, K. L., Smith, J., Siddique, K., & Guan, S. U. (2020). Improved two-stream model for human action recognition. *Eurasip Journal on Image and Video Processing*, 2020(1). <https://doi.org/10.1186/s13640-020-00501-x>

LAMPIRAN

Lampiran 1. Dataset

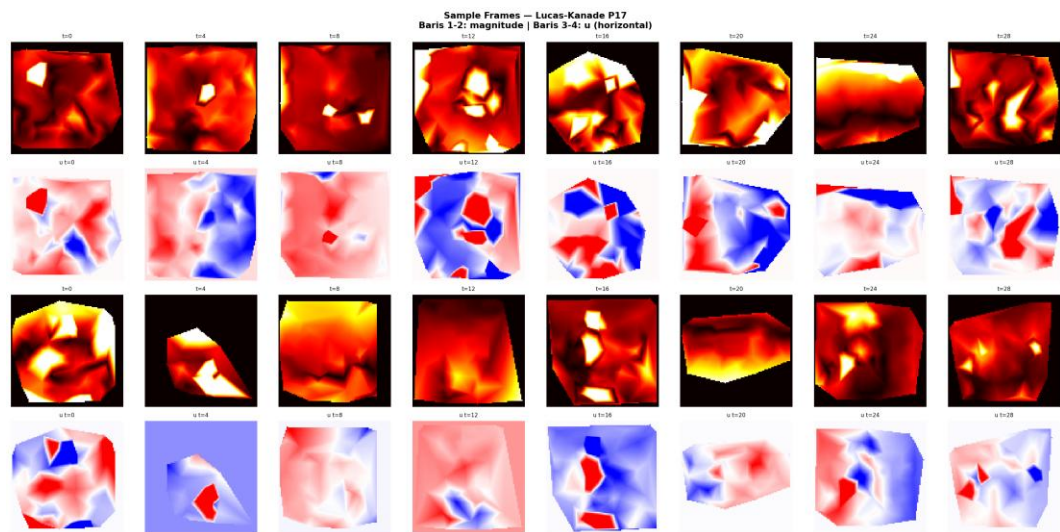
a. Raw Dataset



b. Dataset RGB yang sudah diproses



c. Dataset Flow dari proses Optical Flow



Lampiran 2. Tabel Perbandingan Parameter *Optical Flow* Lucas-Kanade

<i>ID</i>	<i>maxCorners</i>	<i>qualityLevel</i>	<i>minDistance</i>	<i>blockSize</i>	<i>winSize</i>	<i>maxLevel</i>	<i>Acc Train/Test</i>	<i>Loss Train/Test</i>
BaseLK	200	0.01	5	5	15x15	3	60.00 % 62.27 %	1.2840 1.2097
P1	100	0.01	5	5	15x15	3	55.00 % 55.45 %	2.8162 2.8004
P2	300	0.01	5	5	15x15	3	57.95 % 58.64 %	2.2272 2.1726
P3	200	0.005	5	5	15x15	3	63.41 % 53.64 %	1.6444 2.0830
P4	200	0.02	5	5	15x15	3	58.41 % 58.18 %	1.4241 1.4181
P5	200	0.01	7	5	15x15	3	52.95 % 51.36 %	2.7186 2.6363
P6	200	0.01	9	5	15x15	3	56.59 % 55.91 %	1.4398 1.4161
P7*	200	0.01	5	5	15x15	1	55.23 % 60.45 %	2.5653 2.4617
P8	200	0.01	5	5	15x15	2	52.50 % 49.55 %	2.0969 2.2257
P9	100	0.005	5	5	15x15	3	59.32 % 55.91 %	1.3627 1.3841
P10	100	0.02	5	5	15x15	3	53.41 % 48.18 %	2.3374 2.7250
P11	150	0.01	5	5	15x15	3	50.91 % 49.09 %	2.4470 2.3916

P12	150	0.005	5	6	15x15	3	56.59 % 61.82 %	2.6239 2.2906
P13	150	0.02	5	6	15x15		56.82 % 60.45 %	2.1399 2.0026
P14	250	0.01	5	6	15x15		53.86 % 52.73 %	2.9948 2.6461
P15	250	0.005	5	6	15x15		59.77 % 59.55 %	2.1824 2.1642
P16	250	0.02	5	6	15x15		55.91 % 50.00 %	2.0572 2.2542
P17	300	0.005	5	6	15x15		62.05 % 60.45 %	2.0328 1.8257
P18	300	0.02	5	6	15x15		59.09 % 55.45 %	1.9912 1.9249

Lampiran 3. Kode Lengkap Preprosesing Data

```
# Cell 5 - Extract semua frame
def extract_all_frames(video_path, resize_to=(224, 224)):
    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        return []
    frames = []
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(cv2.resize(frame, resize_to))
    cap.release()
    return frames

# Cell 6 - Deteksi bibir di semua frame (Tasks API)
def get_lip_bbox(landmarks, h, w, margin=0.30):
    coords = np.array([[int(landmarks[i].x * w),
                        int(landmarks[i].y * h)] for i in
LIP_LANDMARKS])
    x0, y0 = coords.min(axis=0)
    x1, y1 = coords.max(axis=0)
    bw, bh = x1 - x0, y1 - y0
    x0 = max(0, int(x0 - bw * margin))
    y0 = max(0, int(y0 - bh * margin))
    x1 = min(w, int(x1 + bw * margin))
    y1 = min(h, int(y1 + bh * margin))
    return x0, y0, x1, y1

def detect_lips_all_frames(frames, detector, margin=0.30):
    lip_regions, detect_mask = [], []
    for frame in frames:
        h, w = frame.shape[:2]
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_img = mp.Image(image_format=mp.ImageFormat.SRGB,
data=rgb)
        result = detector.detect(mp_img)

        if result.face_landmarks:
            lm = result.face_landmarks[0]
            x0,y0,x1,y1 = get_lip_bbox(lm, h, w, margin)
            crop = frame[y0:y1, x0:x1]
            if crop.size > 0:
                lip_regions.append(crop)
                detect_mask.append(True)
                continue

            lip_regions.append(None)
            detect_mask.append(False)
    return lip_regions, detect_mask

# Cell 7 - Pilih 30 frame berdasarkan perubahan bibir
def select_top30_by_lip_motion(lip_regions, detect_mask,
target_n=30):
```

```

        valid = [i for i, ok in enumerate(detect_mask) if ok]
        if not valid:
            return None

        thumbs = {i: cv2.cvtColor(cv2.resize(lip_regions[i], (32,
32))),
                                cv2.COLOR_BGR2GRAY).astype(np.float32)
                                for i in valid}

        scores = {}
        prev = None
        for i in valid:
            scores[i] = 0.0 if prev is None else np.abs(thumbs[i] -
thumbs[prev]).mean()
            prev = i

        if len(valid) <= target_n:
            sel = valid.copy()
            while len(sel) < target_n:
                sel.append(valid[-1])
            return sel

        top = sorted(scores, key=scores.get,
reverse=True)[:target_n]
        return sorted(top)

# Cell 8 - Crop & resize 30 frame terpilih
def crop_selected_frames(lip_regions, selected, lip_size=(96,
96)):
    crops = []
    for i in selected:
        c = lip_regions[i]
        crops.append(cv2.resize(c, lip_size) if c is not None
                    else np.zeros((*lip_size, 3),
dtype=np.uint8))
    return np.array(crops, dtype=np.uint8)

```

Lampiran 4. Kode Lengkap Optical Flow Lucas-Kanade P17

```
# — Parameter P17
FEAT_PARAMS = dict(
    maxCorners    = 300,
    qualityLevel  = 0.005,
    minDistance   = 5,
    blockSize     = 6
)
LK_PARAMS = dict(
    winSize       = (15, 15),
    maxLevel      = 3,
    criteria      = (cv2.TERM_CRITERIA_EPS |
cv2.TERM_CRITERIA_COUNT, 30, 0.01),
    minEigThreshold = 1e-4
)
INTERP_METHOD = 'linear'

NUM_CLASSES = len(CLASSES)
IDX_TO_CLASS = {i: c for i, c in enumerate(CLASSES)}

# — Fungsi utama Optical Flow LK
def load_gray(npy_path):
    data = np.load(npy_path)
    return np.array([cv2.cvtColor(f, cv2.COLOR_BGR2GRAY) for f
in data])

def sparse_to_dense(src_pts, flow_vecs, H, W):
    if len(src_pts) < 4:
        return np.zeros((H, W, 3), dtype=np.float32)
    gy, gx = np.mgrid[0:H, 0:W]
    grid = np.column_stack([gx.ravel(), gy.ravel()])
    u = griddata(src_pts, flow_vecs[:, 0], grid,
method=INTERP_METHOD, fill_value=0.0).reshape(H,
W).astype(np.float32)

# — Statistik training set
print('PASS 1: hitung statistik training set (P17)...')
train_files = [f for cls in CLASSES for f in
sorted((RAW_LIPS/'train'/cls).glob('*.npz'))]
print(f'Total train files: {len(train_files)}')

n_px = np.zeros(3, dtype=np.float64)
mean = np.zeros(3, dtype=np.float64)
M2 = np.zeros(3, dtype=np.float64)
failed = []

for f in tqdm(train_files, desc='Computing stats P17'):
    try:
        flows = compute_lk_flow(load_gray(f))
        if flows is None: continue
        for ch in range(3):
```



```

        for x in flows[... , ch].ravel():
            n_px[ch] += 1
            delta = x - mean[ch]
            mean[ch] += delta / n_px[ch]
            M2[ch] += delta * (x - mean[ch])
    except Exception as e:
        failed.append(f.name)

std = np.maximum(np.sqrt(M2 / (n_px - 1)), 1e-6)

print('\nHitung clip percentile...')
buf = []
for f in tqdm(train_files[:150], desc='Percentile',
leave=False):
    try:
        flows = compute_lk_flow(load_gray(f))
        if flows is not None: buf.append(flows[:,::2, ::4, ::4,
:])
    except: pass

if buf:
    cat = np.concatenate(buf, axis=0)
    clip_min = np.percentile(cat, 2, axis=(0, 1, 2))
    clip_max = np.percentile(cat, 98, axis=(0, 1, 2))
else:
    clip_min, clip_max = mean - 3*std, mean + 3*std

stats = {
    'mean': mean.tolist(), 'std': std.tolist(),
    'clip_min': clip_min.tolist(), 'clip_max':
clip_max.tolist(),
    'n_files': len(train_files) - len(failed),
    'channels': ['u', 'v', 'magnitude'],
    'method': 'lucas-kanade',
    'config': 'P17',
    'feat_params': {str(k): v for k, v in FEAT_PARAMS.items()}
}
with open(STATS_FILE, 'w') as f:
    json.dump(stats, f, indent=2)

print(f'\nStatistik P17 ({len(train_files)-len(failed)} file):')
for i, ch in enumerate(['u', 'v', 'magnitude']):
    print(f' {ch:9s}: mean={mean[i]:+.4f}, std={std[i]:.4f},
clip=[{clip_min[i]:+.3f}, {clip_max[i]:+.3f}]')
print(f'Saved: {STATS_FILE}')

v = griddata(src_pts, flow_vecs[:, 1], grid,
method=INTERP_METHOD, fill_value=0.0).reshape(H,
W).astype(np.float32)
return np.stack([u, v, np.sqrt(u**2 + v**2)], axis=-1)

def compute_lk_flow(frames):
    T, H, W = frames.shape
    if T < 2: return None
    flows = []

```

```

        for t in range(T - 1):
            p0 = cv2.goodFeaturesToTrack(frames[t], mask=None,
**FEAT_PARAMS)
            if p0 is None or len(p0) < 4:
                flows.append(np.zeros((H, W, 3), dtype=np.float32));
            continue
            p1, status, _ = cv2.calcOpticalFlowPyrLK(frames[t],
frames[t+1], p0, None, **LK_PARAMS)
            mask = status.ravel() == 1
            if mask.sum() < 4:
                flows.append(np.zeros((H, W, 3), dtype=np.float32));
            continue
            old = p0[mask].reshape(-1, 2)
            new = p1[mask].reshape(-1, 2)
            flows.append(sparse_to_dense(old, new - old, H, W))
        return np.array(flows, dtype=np.float32)

print('fungsi siap')

# — Batch processing
import time as _time
with open(STATS_FILE) as f: stats = json.load(f)
G_MEAN = np.array(stats['mean'],      dtype=np.float32)
G_STD   = np.array(stats['std'],      dtype=np.float32)
G_CMIN  = np.array(stats['clip_min'], dtype=np.float32)
G_CMAX  = np.array(stats['clip_max'], dtype=np.float32)

def normalize(flows):
    out = flows.copy()
    for ch in range(3):
        out[..., ch] = np.clip(out[..., ch], G_CMIN[ch],
G_CMAX[ch])
        out[..., ch] = (out[..., ch] - G_MEAN[ch]) / G_STD[ch]
    return out.astype(np.float32)

for split in SPLITS:
    for cls in CLASSES:
        (FLOWS_DIR / split / cls).mkdir(parents=True,
exist_ok=True)

ok, fail, fail_log = 0, 0, []
t_start = _time.time()
print('PASS 2: proses semua data (P17)...\n')

for split in SPLITS:
    print(f'--- {split.upper()} ---')
    for cls in CLASSES:
        out_dir = FLOWS_DIR / split / cls
        for f in
tqdm(sorted((RAW_LIPS/split/cls).glob('*.npy')),
desc=f'{split}/{cls}', leave=False):
            out_path = out_dir / f.name
            if out_path.exists(): ok += 1; continue

```

```
        try:
            flows = compute_lk_flow(load_gray(f))
            if flows is None or len(flows) < 5: fail += 1;
fail_log.append({'split':split,'class':cls,'file':f.name,'reason':
':flow failed'}); continue
            np.save(out_path, normalize(flows))
            ok += 1
        except Exception as e:
            fail += 1;
fail_log.append({'split':split,'class':cls,'file':f.name,'reason':
':str(e)})

print(f'\nSelesai dalam {(_time.time()-t_start)/60:.1f} menit |
OK: {ok} | Gagal: {fail}')
```

Lampiran 5. Kode Lengkap Model Conv-LSTM Terbaik M02

```

BASE_DIR      = Path('/home/wipra-
ranum/Documents/Skripsi/LipReading')
PROJECT_ROOT  = BASE_DIR / 'Lip_Reading_Gabungan'
RAW_LIPS      = PROJECT_ROOT / 'preprocessed' / 'dependent' /
'raw_lips'
MODELS_DIR    = PROJECT_ROOT / 'models/ofat_model/M02'
RESULTS_DIR   = PROJECT_ROOT / 'results/ofat_model/M02'
MODELS_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

with open(PROJECT_ROOT / 'config.json') as f:
    cfg = json.load(f)

CLASSES       = cfg['data']['classes']
NUM_CLASSES    = len(CLASSES)
IDX_TO_CLASS   = {i: c for i, c in enumerate(CLASSES)}

VARIANT_ID     = 'M02'
CNN_CHANNELS   = [32, 64, 128]
LSTM_HIDDENS   = [128, 128]
DROPOUT        = 0.5
FC_HIDDEN      = 256

BATCH_SIZE     = 8
NUM_EPOCHS     = 100
LR             = 0.001
WEIGHT_DECAY   = 1e-5
SEED           = 42

torch.manual_seed(SEED); np.random.seed(SEED); random.seed(SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed(SEED)

print(f'Variant   : {VARIANT_ID}')
print(f'CNN ch    : {CNN_CHANNELS}')
print(f'LSTM h     : {LSTM_HIDDENS}')
print(f'Dropout    : {DROPOUT} | FC: {FC_HIDDEN}')
print(f'Classes    : {NUM_CLASSES} - {CLASSES}')
print(f'Results    → {RESULTS_DIR}')

class LipDataset(Dataset):
    def __init__(self, root, classes, split):
        self.samples, self.labels = [], []
        for idx, cls in enumerate(classes):
            d = Path(root) / split / cls
            if not d.exists(): print(f' Warning: {d} tidak
ditemukan'); continue
            for f in sorted(d.glob('*.npy')):
                self.samples.append(f); self.labels.append(idx)
        print(f' {split:5s}: {len(self.samples)} samples')

```

```

    def __len__(self): return len(self.samples)

    def __getitem__(self, i):
        seq = np.load(self.samples[i]).astype(np.float32) /
255.0
        seq = torch.from_numpy(seq.transpose(3, 0, 1, 2)) #
[C,T,H,W]
        return seq, torch.tensor(self.labels[i],
dtype=torch.long)

print('Loading datasets...')
train_ds = LipDataset(RAW_LIPS, CLASSES, 'train')
val_ds   = LipDataset(RAW_LIPS, CLASSES, 'val')
test_ds  = LipDataset(RAW_LIPS, CLASSES, 'test')

train_loader = DataLoader(train_ds, BATCH_SIZE,
shuffle=True, num_workers=4, pin_memory=True)
val_loader   = DataLoader(val_ds, BATCH_SIZE, shuffle=False,
num_workers=4, pin_memory=True)
test_loader  = DataLoader(test_ds, BATCH_SIZE, shuffle=False,
num_workers=4, pin_memory=True)

x, y = next(iter(train_loader))
print(f'\nBatch shape : {x.shape} | Label sample:
{[IDX_TO_CLASS[l.item()] for l in y]}')


class ConvLSTMCell(nn.Module):
    def __init__(self, in_ch, h_ch, k=3):
        super().__init__()
        self.h_ch = h_ch
        self.conv = nn.Conv2d(in_ch + h_ch, 4 * h_ch, k,
padding=k // 2)

    def forward(self, x, state):
        h, c = state
        i, f, o, g = torch.split(self.conv(torch.cat([x, h],
dim=1)), self.h_ch, dim=1)
        c_next = torch.sigmoid(f) * c + torch.sigmoid(i) *
torch.tanh(g)
        h_next = torch.sigmoid(o) * torch.tanh(c_next)
        return h_next, c_next

    def init_hidden(self, B, H, W, dev):
        return (torch.zeros(B, self.h_ch, H, W, device=dev),
torch.zeros(B, self.h_ch, H, W, device=dev))

class ConvLSTMModel(nn.Module):
    """
    Input : [B, 3, T, H, W] = [B, 3, 30, 96, 96]
    Output: [B, num_classes]
    cnn_channels : list channel CNN backbone

```

```

    lstm_hiddens : list hidden ConvLSTM (bisa 2 atau 3 layer)
    """
    def __init__(self, num_classes, cnn_channels, lstm_hiddens,
dropout=0.5, fc_hidden=256):
        super().__init__()

        # CNN backbone
        def cblock(ic, oc):
            return nn.Sequential(nn.Conv2d(ic, oc, 3,
padding=1),
                                nn.BatchNorm2d(oc), nn.ReLU(),
nn.MaxPool2d(2))
        layers, prev = [], 3
        for ch in cnn_channels:
            layers.append(cblock(prev, ch)); prev = ch
        self.cnn = nn.Sequential(*layers)

        # ConvLSTM layers
        self.lstm_cells = nn.ModuleList()
        in_ch = prev
        for h_ch in lstm_hiddens:
            self.lstm_cells.append(ConvLSTMCell(in_ch, h_ch))
            in_ch = h_ch

        self.head = nn.Sequential(
            nn.Linear(lstm_hiddens[-1], fc_hidden),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(fc_hidden, num_classes)
        )

    def forward(self, x):
        B, C, T, H, W = x.shape
        x = x.permute(0, 2, 1, 3, 4).reshape(B*T, C, H, W)
        x = self.cnn(x)
        _, c, h, w = x.shape
        x = x.view(B, T, c, h, w)
        states = [cell.init_hidden(B, h, w, x.device) for cell
in self.lstm_cells]
        for t in range(T):
            inp = x[:, t]
            for li, cell in enumerate(self.lstm_cells):
                states[li] = cell(inp, states[li])
                inp = states[li][0]
            out = states[-1][0].mean(dim=[-2, -1]) # GAP
        return self.head(out)

model = ConvLSTMModel(NUM_CLASSES, CNN_CHANNELS, LSTM_HIDDENS,
DROPOUT, FC_HIDDEN).to(device)
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
model_size_mb = total_params * 4 / 1e6
print(f'Variant : {VARIANT_ID}')
print(f'Total params : {total_params:,}')

```

```

print(f'Model size      : {model_size_mb:.2f} MB')

optimizer = optim.Adam(model.parameters(), lr=LR,
weight_decay=WEIGHT_DECAY)
scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=0.5,
patience=5)
criterion = nn.CrossEntropyLoss()

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, preds_all, labels_all = 0.0, [], []
    ctx = torch.enable_grad() if train else torch.no_grad()
    with ctx:
        for x, y in tqdm(loader, leave=False, desc='Train' if
train else 'Eval '):
            x, y = x.to(device), y.to(device)
            out = model(x)
            loss = criterion(out, y)
            if train:
                optimizer.zero_grad(); loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
                optimizer.step()
            total_loss += loss.item() * len(y)
            preds_all.extend(out.argmax(1).cpu().tolist())
            labels_all.extend(y.cpu().tolist())
    return total_loss / len(loader.dataset), 100 *
accuracy_score(labels_all, preds_all), preds_all, labels_all

def per_class_str(labels, preds):
    res = []
    for i, cls in enumerate(CLASSES):
        tot = sum(l == i for l in labels)
        if tot == 0: continue
        acc = round(100 * sum(p == i for p, l in zip(preds,
labels) if l == i) / tot)
        res.append(f'{cls}:{acc}%')
    return ' '.join(res)

history = {'train_loss':[], 'train_acc':[], 'val_loss':[],
'val_acc':[], 'lr':[], 'epoch_time':[]}
best_val = 0.0; best_ep = 0
best_path = MODELS_DIR / f'{VARIANT_ID}_best.pth'
t0_total = time.time()

for ep in range(1, NUM_EPOCHS + 1):
    t0 = time.time()
    tr_loss, tr_acc, _, _ = run_epoch(train_loader,
train=True)
    va_loss, va_acc, va_p, va_l =
run_epoch(val_loader, train=False)
    scheduler.step(va_acc)

```

```

cur_lr = optimizer.param_groups[0]['lr']
ep_t = time.time() - t0

history['train_loss'].append(tr_loss)
history['train_acc'].append(tr_acc)
history['val_loss'].append(va_loss)
history['val_acc'].append(va_acc)
history['lr'].append(cur_lr)
history['epoch_time'].append(ep_t)

mark = ''
if va_acc > best_val:
    best_val = va_acc; best_ep = ep
    torch.save({'model_state': model.state_dict(), 'epoch':
ep,
                'val_acc': va_acc, 'variant': VARIANT_ID},
best_path)
    mark = ' <-- best'

    print(f'{ep:4d} loss={tr_loss:.4f}
tr={tr_acc:.2f}% val={va_acc:.2f}% lr={cur_lr:.2e} {ep_t:.0f}
s{mark}')
    if ep % 5 == 0 or mark:
        print(f'          per-class: {per_class_str(va_l, va_p)}')

total_time = time.time() - t0_total
print(f'\nTraining selesai! Best val={best_val:.2f}% di epoch
{best_ep} | Total={total_time/60:.1f} menit')

# Load best checkpoint → evaluasi test set
ckpt = torch.load(best_path, weights_only=False)
model.load_state_dict(ckpt['model_state'])
te_loss, te_acc, te_p, te_l = run_epoch(test_loader,
train=False)

report = classification_report(te_l, te_p,
target_names=CLASSES, output_dict=True)
f1_macro = report['macro avg']['f1-score'] * 100
f1_weight = report['weighted avg']['f1-score'] * 100
prec_w = report['weighted avg']['precision'] * 100
rec_w = report['weighted avg']['recall'] * 100

class_accs = []
for i in range(NUM_CLASSES):
    tot = sum(l == i for l in te_l)
    class_accs.append(100 * sum(p == i for p, l in zip(te_p,
te_l) if l == i) / tot if tot else 0)

print(f'=== Hasil Test {VARIANT_ID} ===')
print(f'Test Accuracy : {te_acc:.2f}%')
print(f'F1 Weighted : {f1_weight:.2f}%')
print(f'F1 Macro : {f1_macro:.2f}%')

```



```
print(f'Precision (w) : {prec_w:.2f}%')
print(f'Recall (w) : {rec_w:.2f}%')
print(f'Best Epoch : {best_ep}/{NUM_EPOCHS}')
print(f'Best Val Acc : {best_val:.2f}%')
print('\nPer-class accuracy:')
for cls, acc in zip(CLASSES, class_accs):
    flag = '✓' if acc >= 80 else ('Δ' if acc >= 50 else 'X')
    print(f' {flag} {cls:12s}: {acc:.1f}%')
```

Lampiran 6. Kode Lengkap Single-Stream Optical Flow Lucas-Kanade P17

```

BASE_DIR      = Path('/home/wipra-
ranum/Documents/Skripsi/LipReading')
PROJECT_ROOT = BASE_DIR / 'Lip_Reading_Gabungan'
FLOWS_DIR     = PROJECT_ROOT / 'preprocessed' / 'dependent' /
'flows_lk_p17' # input P17
MODELS_DIR    = PROJECT_ROOT / 'models' / 'lk_p17' / 'M02'
RESULTS_DIR   = PROJECT_ROOT / 'results' / 'lk_p17' / 'M02'
MODELS_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

with open(PROJECT_ROOT / 'config.json') as f:
    cfg = json.load(f)

CLASSES       = cfg['data']['classes']
NUM_CLASSES   = len(CLASSES)
IDX_TO_CLASS  = {i: c for i, c in enumerate(CLASSES)}

VARIANT_ID    = 'P17-M02'
CNN_CHANNELS  = [32, 64, 128]
LSTM_HIDDENS  = [128, 128]          # layer-2 dinaikkan dari 64 → 128
DROPOUT       = 0.5
FC_HIDDEN     = 256
SEQ_LEN       = 29                 # T-1 dari 30 frame raw (flow = 29
frame)

BATCH_SIZE    = 8
NUM_EPOCHS    = 100
LR            = 0.001
WEIGHT_DECAY  = 1e-5
SEED          = 42

torch.manual_seed(SEED); np.random.seed(SEED); random.seed(SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed(SEED)

print(f'Variant   : {VARIANT_ID}')
print(f'Input     : {FLOWS_DIR}')
print(f'CNN ch      : {CNN_CHANNELS}')
print(f'LSTM h      : {LSTM_HIDDENS} ← layer-2 = 128 (vs 64
baseline)')
print(f'Seq len    : {SEQ_LEN} frames (optical flow T-1)')
print(f'Classes    : {NUM_CLASSES} - {CLASSES}')
print(f'Results    → {RESULTS_DIR}')

class LKFlowDataset(Dataset):
    """
    Load optical flow .npy [T-1, H, W, 3] → tensor [3, T-1, H,
W].
    Data SUDAH ternormalisasi (clip + z-score) dari Notebook 04c
- jangan dibagi 255!

```

```

    Augmentasi: horizontal flip + negate channel u (semantik-
    benar untuk optical flow).
    """
    def __init__(self, root, classes, split, seq_len=29,
    augment=False):
        self.samples, self.labels = [], []
        self.seq_len = seq_len
        self.augment = augment
        for idx, cls in enumerate(classes):
            d = Path(root) / split / cls
            if not d.exists(): print(f' Warning: {d} tidak
            ditemukan'); continue
            for f in sorted(d.glob('*.npy')):
                self.samples.append(f); self.labels.append(idx)
        print(f' {split:5s}: {len(self.samples)} samples')

    def _fix_len(self, s):
        T = len(s)
        if T == self.seq_len: return s
        if T < self.seq_len:
            pad = np.zeros((self.seq_len - T, *s.shape[1:]),
            dtype=np.float32)
            return np.concatenate([s, pad])
        return s[np.linspace(0, T - 1, self.seq_len, dtype=int)]

    def _aug(self, s):
        if np.random.random() > 0.5:
            s = s[:, :, ::-1, :].copy() # flip horizontal
            s[..., 0] *= -1 # negate u channel
        (wajib!)
        return s

    def __len__(self): return len(self.samples)

    def __getitem__(self, i):
        seq = np.load(self.samples[i]).astype(np.float32) #
        [T-1, H, W, 3] - sudah normalized
        if self.augment: seq = self._aug(seq)
        seq = self._fix_len(seq) #
        [SEQ_LEN, H, W, 3]
        seq = torch.from_numpy(seq.transpose(3, 0, 1, 2)) #
        [3, SEQ_LEN, H, W]
        return seq, torch.tensor(self.labels[i],
        dtype=torch.long)

    print('Loading datasets...')
    train_ds = LKFlowDataset(FLOWS_DIR, CLASSES, 'train', SEQ_LEN,
    augment=True)
    val_ds = LKFlowDataset(FLOWS_DIR, CLASSES, 'val', SEQ_LEN,
    augment=False)
    test_ds = LKFlowDataset(FLOWS_DIR, CLASSES, 'test', SEQ_LEN,
    augment=False)

    train_loader = DataLoader(train_ds, BATCH_SIZE,
    shuffle=True, num_workers=4, pin_memory=True)

```

```

val_loader = DataLoader(val_ds, BATCH_SIZE, shuffle=False,
num_workers=4, pin_memory=True)
test_loader = DataLoader(test_ds, BATCH_SIZE, shuffle=False,
num_workers=4, pin_memory=True)

x, y = next(iter(train_loader))
print(f'\nBatch shape : {x.shape} (expected [8, 3, {SEQ_LEN},
96, 96])')
print(f'Value range : [{x.min():.3f}, {x.max():.3f}] (sudah
ternormalisasi, bukan 0-1)')
print(f'Label sample: {[IDX_TO_CLASS[l.item()] for l in y}]')


class ConvLSTMCell(nn.Module):
    def __init__(self, in_ch, h_ch, k=3):
        super().__init__()
        self.h_ch = h_ch
        self.conv = nn.Conv2d(in_ch + h_ch, 4 * h_ch, k,
padding=k // 2)

    def forward(self, x, state):
        h, c = state
        i, f, o, g = torch.split(self.conv(torch.cat([x, h],
dim=1)), self.h_ch, dim=1)
        c_next = torch.sigmoid(f) * c + torch.sigmoid(i) *
torch.tanh(g)
        h_next = torch.sigmoid(o) * torch.tanh(c_next)
        return h_next, c_next

    def init_hidden(self, B, H, W, dev):
        return (torch.zeros(B, self.h_ch, H, W, device=dev),
torch.zeros(B, self.h_ch, H, W, device=dev))

class ConvLSTMModel(nn.Module):
    """
    Input : [B, 3, T, H, W] - 3 ch = u, v, magnitude (optical
flow P17)
    Output: [B, num_classes]
    cnn_channels : list channel CNN backbone
    lstm_hiddens : list hidden ConvLSTM (fleksibel 2 atau 3
layer)
    """
    def __init__(self, num_classes, cnn_channels, lstm_hiddens,
dropout=0.5, fc_hidden=256):
        super().__init__()

        def cblock(ic, oc):
            return nn.Sequential(nn.Conv2d(ic, oc, 3,
padding=1),
                                nn.BatchNorm2d(oc), nn.ReLU(),
                                nn.MaxPool2d(2))
        layers, prev = [], 3
        for ch in cnn_channels:

```

```

        layers.append(cblock(prev, ch)); prev = ch
    self.cnn = nn.Sequential(*layers)

    self.lstm_cells = nn.ModuleList()
    in_ch = prev
    for h_ch in lstm_hidden:
        self.lstm_cells.append(ConvLSTMCell(in_ch, h_ch))
        in_ch = h_ch

    self.head = nn.Sequential(
        nn.Linear(lstm_hidden[-1], fc_hidden),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(fc_hidden, num_classes)
    )

    def forward(self, x):
        B, C, T, H, W = x.shape
        x = x.permute(0, 2, 1, 3, 4).reshape(B*T, C, H, W)
        x = self.cnn(x)
        _, c, h, w = x.shape
        x = x.view(B, T, c, h, w)
        states = [cell.init_hidden(B, h, w, x.device) for cell
in self.lstm_cells]
        for t in range(T):
            inp = x[:, t]
            for li, cell in enumerate(self.lstm_cells):
                states[li] = cell(inp, states[li])
            inp = states[li][0]
        out = states[-1][0].mean(dim=[-2, -1]) # GAP
        return self.head(out)

model = ConvLSTMModel(NUM_CLASSES, CNN_CHANNELS, LSTM_HIDDEN,
DROPOUT, FC_HIDDEN).to(device)
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
model_size_mb = total_params * 4 / 1e6

with torch.no_grad():
    out_test = model(torch.randn(2, 3, SEQ_LEN, 96,
96).to(device))

print(f'Variant      : {VARIANT_ID}')
print(f'Total params : {total_params:,}')
print(f'Model size   : {model_size_mb:.2f} MB')
print(f'Forward test : [2, 3, {SEQ_LEN}, 96, 96] →
{out_test.shape} ✓')

optimizer = optim.Adam(model.parameters(), lr=LR,
weight_decay=WEIGHT_DECAY)

```

```

scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=0.5,
patience=5)
criterion = nn.CrossEntropyLoss()

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, preds_all, labels_all = 0.0, [], []
    ctx = torch.enable_grad() if train else torch.no_grad()
    with ctx:
        for x, y in tqdm(loader, leave=False, desc='Train' if
train else 'Eval '):
            x, y = x.to(device), y.to(device)
            out = model(x)
            loss = criterion(out, y)
            if train:
                optimizer.zero_grad(); loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
                optimizer.step()
            total_loss += loss.item() * len(y)
            preds_all.extend(out.argmax(1).cpu().tolist())
            labels_all.extend(y.cpu().tolist())
    return total_loss / len(loader.dataset), 100 *
accuracy_score(labels_all, preds_all), preds_all, labels_all

def per_class_str(labels, preds):
    res = []
    for i, cls in enumerate(CLASSES):
        tot = sum(l == i for l in labels)
        if tot == 0: continue
        acc = round(100 * sum(p == i for p, l in zip(preds,
labels) if l == i) / tot)
        res.append(f'{cls}:{acc}%')
    return ' '.join(res)

history = {'train_loss':[], 'train_acc':[], 'val_loss':[],
'val_acc':[], 'lr':[], 'epoch_time':[]}
best_val = 0.0; best_ep = 0
best_path = MODELS_DIR / f'{VARIANT_ID}_best.pth'
t0_total = time.time()

print(f'Training {VARIANT_ID} - {NUM_EPOCHS} epochs |
{NUM_CLASSES} kelas\n')
for ep in range(1, NUM_EPOCHS + 1):
    t0 = time.time()
    tr_loss, tr_acc, _, _ = run_epoch(train_loader,
train=True)
    va_loss, va_acc, va_p, va_l =
run_epoch(val_loader, train=False)
    scheduler.step(va_acc)
    cur_lr = optimizer.param_groups[0]['lr']
    ep_t = time.time() - t0

    history['train_loss'].append(tr_loss)
    history['train_acc'].append(tr_acc)

```

```

history['val_loss'].append(va_loss)
history['val_acc'].append(va_acc)
history['lr'].append(cur_lr)
history['epoch_time'].append(ep_t)

mark = ''
if va_acc > best_val:
    best_val = va_acc; best_ep = ep
    torch.save({'model_state': model.state_dict(), 'epoch':
ep,
                'val_acc': va_acc, 'variant': VARIANT_ID,
                'flow_config': 'P17', 'lstm_hiddens':
LSTM_HIDDENS}, best_path)
    mark = ' <-- best'

    print(f'{ep:4d}  loss={tr_loss:.4f}
tr={tr_acc:.2f}%  val={va_acc:.2f}%  lr={cur_lr:.2e}  {ep_t:.0f}
s{mark}')
    if ep % 5 == 0 or mark:
        print(f'          per-class: {per_class_str(va_l, va_p)}')

total_time = time.time() - t0_total
print(f'\nTraining selesai! Best val={best_val:.2f}% di epoch
{best_ep} | Total={total_time/60:.1f} menit')

ckpt = torch.load(best_path, weights_only=False)
model.load_state_dict(ckpt['model_state'])
te_loss, te_acc, te_p, te_l = run_epoch(test_loader,
train=False)

report    = classification_report(te_l, te_p,
target_names=CLASSES, output_dict=True)
f1_macro  = report['macro avg']['f1-score'] * 100
f1_weight = report['weighted avg']['f1-score'] * 100
prec_w    = report['weighted avg']['precision'] * 100
rec_w     = report['weighted avg']['recall'] * 100

class_accs = []
for i in range(NUM_CLASSES):
    tot = sum(l == i for l in te_l)
    class_accs.append(100 * sum(p == i for p, l in zip(te_p,
te_l) if l == i) / tot if tot else 0)

print(f'=== Hasil Test {VARIANT_ID} ===')
print(f'Test Accuracy   : {te_acc:.2f}%')
print(f'F1 Weighted      : {f1_weight:.2f}%')
print(f'F1 Macro         : {f1_macro:.2f}%')
print(f'Precision (w)    : {prec_w:.2f}%')
print(f'Recall (w)       : {rec_w:.2f}%')
print(f'Best Epoch        : {best_ep}/{NUM_EPOCHS}')
print(f'Best Val Acc      : {best_val:.2f}%')
print(f'\nPer-class accuracy:')

```

```
for cls, acc in zip(CLASSES, class_accs):  
    flag = '✓' if acc >= 80 else ('Δ' if acc >= 50 else 'X')  
    print(f' {flag} {cls:12s}: {acc:.1f}%')
```


Lampiran 7. Kode Lengkap Dual-Stream Optical Flow Lucas-Kanade P17 & M02

```

BASE_DIR      = Path('/home/wipra-
ranum/Documents/Skripsi/LipReading')
PROJECT_ROOT  = BASE_DIR / 'Lip_Reading_Gabungan'
RGB_DIR       = PROJECT_ROOT / 'preprocessed' / 'dependent' /
'raw_lips'
FLOW_DIR      = PROJECT_ROOT / 'preprocessed' / 'dependent' /
'flows_lk_p17'
MODELS_DIR    = PROJECT_ROOT / 'models' / 'dual_stream' / 'M02'
RESULTS_DIR   = PROJECT_ROOT / 'results' / 'dual_stream' / 'M02'
MODELS_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

with open(PROJECT_ROOT / 'config.json') as f:
    cfg = json.load(f)

CLASSES       = cfg['data']['classes']
NUM_CLASSES   = len(CLASSES)
IDX_TO_CLASS  = {i: c for i, c in enumerate(CLASSES)}

# — Parameter M02 (identik di kedua single-stream)
VARIANT_ID    = 'Dual-M02'
CNN_CHANNELS  = [32, 64, 128]
LSTM_HIDDENS  = [128, 128]      # M02: layer-2 = 128 (bukan 64
seperti baseline)
DROPOUT       = 0.5
FC_HIDDEN     = 256
RGB_SEQ_LEN   = 30              # frame RGB
FLOW_SEQ_LEN  = 29              # frame flow = T-1

BATCH_SIZE    = 8
NUM_EPOCHS    = 100
LR            = 0.001
WEIGHT_DECAY  = 1e-5
SEED          = 42

torch.manual_seed(SEED); np.random.seed(SEED); random.seed(SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed(SEED)

print(f'Variant    : {VARIANT_ID}')
print(f'CNN ch     : {CNN_CHANNELS}')
print(f'LSTM h      : {LSTM_HIDDENS} (M02: layer-2 = 128, bukan
64)')
print(f'Dropout    : {DROPOUT} | FC: {FC_HIDDEN}')
print(f'RGB seq     : {RGB_SEQ_LEN} frames | Flow seq:
{FLOW_SEQ_LEN} frames')
print(f'Classes     : {NUM_CLASSES} — {CLASSES}')
```

```

class DualStreamDataset(Dataset):
    """
    Load pasangan (RGB, LK P17 flow) untuk dual-stream M02.

    RGB : raw_lips → [T, H, W, 3] uint8, dibagi 255
    Flow : flows_lk_p17 → [T-1, H, W, 3] float32, sudah z-score
    (P17 params)

    Augmentasi train:
    - Horizontal flip + negate u channel pada flow.
      Negate u wajib karena u merepresentasikan perpindahan
horizontal
    - tanpa negate, arah gerak jadi kebalik semantiknya.
    """

    def __init__(self, rgb_dir, flow_dir, classes, split,
                 rgb_seq=30, flow_seq=29, augment=False):
        self.rgb_dir = Path(rgb_dir)
        self.flow_dir = Path(flow_dir)
        self.rgb_seq = rgb_seq
        self.flow_seq = flow_seq
        self.augment = augment
        cls_map = {c: i for i, c in enumerate(classes)}

        self.samples = []
        for cls in classes:
            for rf in sorted((self.rgb_dir / split /
cls).glob('*.numpy')):
                ff = self.flow_dir / split / cls / rf.name
                if ff.exists():
                    self.samples.append((rf, ff, cls_map[cls]))

        # verifikasi pasangan
        miss = 0
        for cls in classes:
            rn = len(list((self.rgb_dir / split /
cls).glob('*.numpy'))))
            fn = len(list((self.flow_dir / split /
cls).glob('*.numpy'))))
            if rn != fn: miss += abs(rn - fn)
        if miss:
            print(f' Warning: {miss} file tidak berpasangan di
split={split}')
            print(f' {split:5s}: {len(self.samples)} pasangan')

    def _fix_len(self, s, target):
        T = len(s)
        if T == target: return s
        if T < target:
            pad = np.zeros((target - T, *s.shape[1:]),
dtype=np.float32)
            return np.concatenate([s, pad])
        return s[np.linspace(0, T - 1, target, dtype=int)]

    def _aug(self, rgb, flow):

```

```

        if np.random.random() > 0.5:
            rgb = rgb[:, :, ::-1, :].copy()
            flow = flow[:, :, ::-1, :].copy()
            flow[..., 0] *= -1    # negate u (horizontal
displacement)
        return rgb, flow

    def __len__(self): return len(self.samples)

    def __getitem__(self, i):
        rf, ff, label = self.samples[i]
        rgb = np.load(rf).astype(np.float32) / 255.0    # [T, H,
W, 3]
        flow = np.load(ff).astype(np.float32)           # [T-1,
H, W, 3]

        if self.augment: rgb, flow = self._aug(rgb, flow)

        rgb = self._fix_len(rgb, self.rgb_seq)
        flow = self._fix_len(flow, self.flow_seq)

        rgb = torch.from_numpy(rgb.transpose(3, 0, 1, 2))    #
[C, T, H, W]
        flow = torch.from_numpy(flow.transpose(3, 0, 1, 2))
        return rgb, flow, torch.tensor(label, dtype=torch.long)

print('Loading datasets...')
train_ds = DualStreamDataset(RGB_DIR, FLOW_DIR, CLASSES,
'train',
                            RGB_SEQ_LEN, FLOW_SEQ_LEN,
augment=True)
val_ds    = DualStreamDataset(RGB_DIR, FLOW_DIR, CLASSES, 'val',
                            RGB_SEQ_LEN, FLOW_SEQ_LEN,
augment=False)
test_ds   = DualStreamDataset(RGB_DIR, FLOW_DIR, CLASSES, 'test',
                            RGB_SEQ_LEN, FLOW_SEQ_LEN,
augment=False)

kw = dict(batch_size=BATCH_SIZE, num_workers=4, pin_memory=True)
train_loader = DataLoader(train_ds, shuffle=True, **kw)
val_loader    = DataLoader(val_ds,  shuffle=False, **kw)
test_loader   = DataLoader(test_ds,  shuffle=False, **kw)

rgb_b, flow_b, lbl_b = next(iter(train_loader))
print(f'\nRGB batch : {rgb_b.shape}    range
[{rgb_b.min():.3f}, {rgb_b.max():.3f}]')
print(f'Flow batch  : {flow_b.shape}   range
[{flow_b.min():.3f}, {flow_b.max():.3f}]')
print(f'Label sample: {[IDX_TO_CLASS[l.item()] for l in
lbl_b]}')

```

```

# — ConvLSTM Cell
class ConvLSTMCell(nn.Module):
    def __init__(self, in_ch, h_ch, k=3):
        super().__init__()
        self.h_ch = h_ch
        self.conv = nn.Conv2d(in_ch + h_ch, 4 * h_ch, k,
padding=k // 2)

    def forward(self, x, state):
        h, c = state
        i, f, o, g = torch.split(self.conv(torch.cat([x, h],
dim=1)), self.h_ch, dim=1)
        c_next = torch.sigmoid(f) * c + torch.sigmoid(i) *
torch.tanh(g)
        h_next = torch.sigmoid(o) * torch.tanh(c_next)
        return h_next, c_next

    def init_hidden(self, B, H, W, dev):
        return (torch.zeros(B, self.h_ch, H, W, device=dev),
torch.zeros(B, self.h_ch, H, W, device=dev))

# — Stream Encoder (identik M02 tapi output feature, bukan
logit)
class StreamEncoder(nn.Module):
    """
    Encoder per-stream dengan konfigurasi M02:
    CNN [32→64→128] + LSTM [128→128] → GAP → feature [B, 128].

    Perbedaan dari encoder baseline (07c): output 128-dim, bukan
64-dim,
    karena LSTM layer-2 di M02 dinaikkan dari 64 → 128 hidden
channels.
    """
    def __init__(self, in_ch=3, cnn_channels=None,
lstm_hiddens=None):
        super().__init__()
        cnn_channels = cnn_channels or [32, 64, 128]
        lstm_hiddens = lstm_hiddens or [128, 128]

        def cblock(ic, oc):
            return nn.Sequential(nn.Conv2d(ic, oc, 3,
padding=1),
nn.BatchNorm2d(oc), nn.ReLU(),
nn.MaxPool2d(2))
        layers, prev = [], in_ch
        for ch in cnn_channels:
            layers.append(cblock(prev, ch)); prev = ch
        self.cnn = nn.Sequential(*layers)

        self.lstm_cells = nn.ModuleList()
        for h_ch in lstm_hiddens:
            self.lstm_cells.append(ConvLSTMCell(prev, h_ch))
            prev = h_ch
        self.out_dim = lstm_hiddens[-1]

```

```

def forward(self, x):
    B, C, T, H, W = x.shape
    x = x.permute(0, 2, 1, 3, 4).reshape(B*T, C, H, W)
    x = self.cnn(x)
    _, c, h, w = x.shape
    x = x.view(B, T, c, h, w)
    states = [cell.init_hidden(B, h, w, x.device) for cell
in self.lstm_cells]
    for t in range(T):
        inp = x[:, t]
        for li, cell in enumerate(self.lstm_cells):
            states[li] = cell(inp, states[li])
            inp = states[li][0]
    return states[-1][0].mean(dim=[-2, -1])    # [B,
lstm_hiddens[-1]]

```

— Attention Fusion

```

class AttentionFusion(nn.Module):
    """
    Bobot adaptif per stream.
    Karena M02 menggunakan 128-dim feature (vs 64-dim di
baseline),
    gate input adalah concat 256-dim, dan output gate adalah
128-dim.

    g_rgb = sigmoid(W_rgb @ [rgb_feat; flow_feat])
    g_flow = sigmoid(W_flow @ [rgb_feat; flow_feat])
    fused = g_rgb * rgb_feat + g_flow * flow_feat
    """
    def __init__(self, feat_dim=128):
        super().__init__()
        self.w_rgb = nn.Linear(feat_dim * 2, feat_dim)
        self.w_flow = nn.Linear(feat_dim * 2, feat_dim)

    def forward(self, rgb_feat, flow_feat):
        combined = torch.cat([rgb_feat, flow_feat], dim=1)    #
[B, 256]
        gate_rgb = torch.sigmoid(self.w_rgb(combined))        #
[B, 128]
        gate_flow = torch.sigmoid(self.w_flow(combined))      #
[B, 128]
        return gate_rgb * rgb_feat + gate_flow * flow_feat    #
[B, 128]

```

— Model Utama

```

class DualStreamM02(nn.Module):
    """
    Dual-Stream Conv-LSTM M02:
    - RGB stream → StreamEncoder(M02) → rgb_feat [B, 128]
    - Flow stream → StreamEncoder(M02) → flow_feat [B, 128]
    - AttentionFusion → fused [B, 128]
    - Head: FC(128→256) + ReLU + Dropout + FC(256→N)

```

```

"""
    def __init__(self, num_classes, cnn_channels, lstm_hidden,
dropout=0.5, fc_hidden=256):
        super().__init__()
        feat_dim = lstm_hidden[-1] # 128 untuk M02
        self.rgb_enc = StreamEncoder(in_ch=3,
cnn_channels=cnn_channels, lstm_hidden=lstm_hidden)
        self.flow_enc = StreamEncoder(in_ch=3,
cnn_channels=cnn_channels, lstm_hidden=lstm_hidden)
        self.fusion = AttentionFusion(feat_dim=feat_dim)
        self.head = nn.Sequential(
            nn.Linear(feat_dim, fc_hidden),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(fc_hidden, num_classes)
        )

    def forward(self, rgb, flow):
        rgb_feat = self.rgb_enc(rgb)
        flow_feat = self.flow_enc(flow)
        fused = self.fusion(rgb_feat, flow_feat)
        return self.head(fused)

model = DualStreamM02(NUM_CLASSES, CNN_CHANNELS, LSTM_HIDDENS,
DROPOUT, FC_HIDDEN).to(device)
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
model_size_mb = total_params * 4 / 1e6

# inference time
dummy_rgb = torch.randn(1, 3, RGB_SEQ_LEN, 96, 96).to(device)
dummy_flow = torch.randn(1, 3, FLOW_SEQ_LEN, 96, 96).to(device)
model.eval()
with torch.no_grad():
    for _ in range(5): _ = model(dummy_rgb, dummy_flow)
    if torch.cuda.is_available(): torch.cuda.synchronize()
    t0 = time.time()
    for _ in range(50): _ = model(dummy_rgb, dummy_flow)
    if torch.cuda.is_available(): torch.cuda.synchronize()
    infer_ms = (time.time() - t0) / 50 * 1000
model.train()

with torch.no_grad():
    out_test =
model(torch.randn(2,3,RGB_SEQ_LEN,96,96).to(device),
        torch.randn(2,3,FLOW_SEQ_LEN,96,96).to(device))

print(f'Variant          : {VARIANT_ID}')
print(f'CNN ch           : {CNN_CHANNELS}')
print(f'LSTM h             : {LSTM_HIDDENS} (M02: layer-2 = 128)')
print(f'Feat dim           : {LSTM_HIDDENS[-1]} (64→128 dibanding
baseline)')
print(f'Total params      : {total_params:,}')

```

```

print(f'Model size      : {model_size_mb:.2f} MB')
print(f'Inference time: {infer_ms:.2f} ms/sample')
print(f'Forward test   : {out_test.shape}  ✓')

# scheduler patience=8 untuk dual-stream (lebih lama stabil
dibanding single)
optimizer = optim.Adam(model.parameters(), lr=LR,
weight_decay=WEIGHT_DECAY)
scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=0.5,
patience=8)
criterion = nn.CrossEntropyLoss()

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, preds_all, labels_all = 0.0, [], []
    ctx = torch.enable_grad() if train else torch.no_grad()
    with ctx:
        for rgb, flow, y in tqdm(loader, leave=False,
desc='Train' if train else 'Eval '):
            rgb, flow, y = rgb.to(device), flow.to(device),
y.to(device)
            out = model(rgb, flow)
            loss = criterion(out, y)
            if train:
                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
                optimizer.step()
            total_loss += loss.item() * len(y)
            preds_all.extend(out.argmax(1).cpu().tolist())
            labels_all.extend(y.cpu().tolist())
    return total_loss / len(loader.dataset), 100 *
accuracy_score(labels_all, preds_all), preds_all, labels_all

def per_class_str(labels, preds):
    res = []
    for i, cls in enumerate(CLASSES):
        tot = sum(l == i for l in labels)
        if tot == 0: continue
        acc = round(100 * sum(p == i for p, l in zip(preds,
labels) if l == i) / tot)
        res.append(f'{cls}:{acc}%')
    return ' '.join(res)

print('Setup selesai')
print(f'Scheduler patience: 8 (dual-stream perlu lebih lama
stabil)')

```

```

history = {'train_loss':[], 'train_acc':[], 'val_loss':[],
'val_acc':[], 'lr':[], 'epoch_time':[]}
best_val = 0.0; best_ep = 0
best_path = MODELS_DIR / f'{VARIANT_ID}_best.pth'
t0_total = time.time()

print(f'Training {VARIANT_ID} - {NUM_EPOCHS} epochs |
{NUM_CLASSES} kelas')
print(f'RGB: {RGB_SEQ_LEN}fr | Flow (P17): {FLOW_SEQ_LEN}fr |
Feat dim: {LSTM_HIDDENS[-1]}\n')

for ep in range(1, NUM_EPOCHS + 1):
    t0 = time.time()
    tr_loss, tr_acc, _, _ = run_epoch(train_loader,
train=True)
    va_loss, va_acc, va_p, va_l =
run_epoch(val_loader, train=False)
    scheduler.step(va_acc)
    cur_lr = optimizer.param_groups[0]['lr']
    ep_t = time.time() - t0

    history['train_loss'].append(tr_loss)
    history['train_acc'].append(tr_acc)
    history['val_loss'].append(va_loss)
    history['val_acc'].append(va_acc)
    history['lr'].append(cur_lr)
    history['epoch_time'].append(ep_t)

    mark = ''
    if va_acc > best_val:
        best_val = va_acc; best_ep = ep
        torch.save({
            'model_state' : model.state_dict(),
            'epoch'       : ep,
            'val_acc'      : va_acc,
            'variant'      : VARIANT_ID,
            'cnn_channels': CNN_CHANNELS,
            'lstm_hiddens': LSTM_HIDDENS,
            'flow_config' : 'P17'
        }, best_path)
        mark = ' <-- best'

    print(f'{ep:4d} loss={tr_loss:.4f}
tr={tr_acc:.2f}% val={va_acc:.2f}% lr={cur_lr:.2e} {ep_t:.0f}
s{mark}')
    if ep % 5 == 0 or mark:
        print(f' per-class: {per_class_str(va_l, va_p)}')

total_time = time.time() - t0_total
print(f'\nTraining selesai! Best val={best_val:.2f}% di epoch
{best_ep} | Total={total_time/60:.1f} menit')

```



```

ckpt = torch.load(best_path, weights_only=False)
model.load_state_dict(ckpt['model_state'])
print(f'Loaded best model epoch {ckpt["epoch"]}
(val={ckpt["val_acc"]:.2f}%)\\n')

te_loss, te_acc, te_p, te_l = run_epoch(test_loader,
train=False)

report = classification_report(te_l, te_p,
target_names=CLASSES, output_dict=True)
f1_macro = report['macro avg']['f1-score'] * 100
f1_weight = report['weighted avg']['f1-score'] * 100
prec_w = report['weighted avg']['precision'] * 100
rec_w = report['weighted avg']['recall'] * 100

class_accs = []
for i in range(NUM_CLASSES):
    tot = sum(l == i for l in te_l)
    class_accs.append(100 * sum(p == i for p, l in zip(te_p,
te_l) if l == i) / tot if tot else 0)

print('=' * 60)
print(f'  HASIL TEST - {VARIANT_ID}')
print('=' * 60)
print(f'  Test Accuracy   : {te_acc:.2f}%')
print(f'  F1 Weighted      : {f1_weight:.2f}%')
print(f'  F1 Macro         : {f1_macro:.2f}%')
print(f'  Precision (w)    : {prec_w:.2f}%')
print(f'  Recall (w)       : {rec_w:.2f}%')
print(f'  Best Epoch       : {best_ep}/{NUM_EPOCHS}')
print(f'  Best Val Acc     : {best_val:.2f}%')
print('=' * 60)
print('\\nPer-class accuracy:')
for cls, acc in zip(CLASSES, class_accs):
    flag = '✓' if acc >= 80 else ('Δ' if acc >= 50 else 'X')
    print(f'  {flag} {cls:12s}: {acc:.1f}%')

print('\\nClassification Report:')
print(classification_report(te_l, te_p, target_names=CLASSES,
digits=3))

```